

电子科技大学
UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

硕士学位论文

MASTER THESIS



论文题目 基于在网计算的低轨卫星网络链路洪
泛攻击防御研究

学科专业 信息与通信工程

学 号 202321011337

作者姓名 龚科成

指导教师 孙罡 教授

学 院 信息与通信工程学院

分类号 _____ 密级 _____ 公开 _____

UDC^{注1} _____

学 位 论 文

基于在网计算的低轨卫星网络链路洪泛攻击防御研究

(题名和副题名)

龚科成

(作者姓名)

指导教师 _____ 孙昱 _____ 教授 _____
电子科技大学 _____ 成都 _____

(姓名、职称、单位名称)

申请学位级别 _____ 硕士 _____ 学科专业 _____ 信息与通信工程 _____

提交论文日期 _____ 论文答辩日期 _____

学位授予单位和日期 _____ 电子科技大学 _____

答辩委员会主席 _____

评阅人 _____

注 1: 注明《国际十进分类法 UDC》的类号。

Research on In-Network Computing-Based Defense Against Link Flooding Attacks in Low Earth Orbit Satellite Networks

A Master Thesis Submitted to
University of Electronic Science and Technology of China

Discipline **Information and Communication Engineering**

Student ID **202321011337**

Author **Gong Kecheng**

Supervisor **Prof. Sun Gang**

School **School of Information and Communication**
Engineering

独创性声明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得电子科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

作者签名：_____ 日期：_____ 年 _____ 月 _____ 日

论文使用授权

本学位论文作者完全了解电子科技大学有关保留、使用学位论文的规定，同意学校有权保留并向国家有关部门或机构送交论文的复印件和数字文档，允许论文被查阅。本人授权电子科技大学可以将学位论文的全部或部分内容编入有关数据库进行检索及下载，可以采用影印、扫描等复制手段保存、汇编学位论文。

（涉密的学位论文须按照国家及学校相关规定管理，在解密后适用于本授权。）

作者签名：_____ 导师签名：_____

日期：_____ 年 _____ 月 _____ 日

摘 要

大规模低地球轨道卫星巨型星座凭借低时延、广覆盖和全球互联能力，正逐步成为天地一体化信息网络的重要基础设施。然而，低轨卫星的星间链路带宽有限，卫星轨道、拓扑结构和部分链路信息又具有较强的公开性与可预测性，使攻击者更容易围绕关键链路构造隐蔽的链路洪泛攻击。这类攻击通过操纵大量分布式节点向合法目标发送低速但协同的流量，使攻击流在目标链路上汇聚形成拥塞，从而降低合法业务吞吐并破坏网络可用性。

现有防御方法难以直接适用于低轨卫星网络场景。集中式方案受制于星地控制环路时延，难以及时响应动态变化的攻击；面向地面网络设计的可编程数据面方案多依赖聚合速率等单一特征，在多诱饵目的分摊、低速化和脉冲化等隐蔽攻击下容易失效；仅在受害链路附近进行本地处置，也难以减少攻击流在上游链路上的无效传输。针对上述问题，本文面向星载可编程数据面受限环境，从单星精确识别和多星协同抑制两个层面开展系统研究。

针对单星场景下隐蔽链路洪泛攻击难以准确识别的问题，本文提出一种基于多签名在网检测与队列调度的防御方法。该方法采用“Top- K 候选生成 + 多维结构特征提取”的级联式检测架构，在有限状态开销下联合利用聚合速率、扇入度、扇出度和持续性等特征，对典型退化攻击建立更稳定的判别依据，并通过队列优先级调度将检测结果转化为低误伤的处置动作。实验结果表明，该方法能够在资源开销可控的前提下显著提升隐蔽链路洪泛攻击的识别能力，并保持较好的检测鲁棒性与业务保护效果。

针对单星本地处置难以降低上游链路带宽浪费的问题，本文进一步提出一种基于多星协同与逐跳推回的协同防御方法。该方法通过抑制型邻居扩散协议在局部区域内聚合多星攻击证据，并结合逐跳推回机制将防御位置从受害链路前移至上游入口，从而把受害点被动缓解扩展为沿路径主动抑制。实验结果表明，该方法能够在有限协同开销下减少攻击流在网络中的无效占用，改善攻击期间合法业务的吞吐表现，并提升大规模星座环境下防御机制的整体协同效率。本文工作为大规模低轨卫星网络提供了一种兼顾低时延、低资源开销与抗退化能力的在网安全防御优化思路。

关键词：低轨卫星网络；链路洪泛攻击；在网计算；可编程数据平面；协同防御

ABSTRACT

Large-scale Low Earth Orbit (LEO) mega-constellations, with their low latency, wide coverage, and global connectivity, are becoming an important foundation of space-ground integrated information networks. However, LEO constellations rely on bandwidth-limited Inter-Satellite Links (ISLs), while orbital, topological, and partial link information remains highly public and predictable. This makes it easier for attackers to launch stealthy Link Flooding Attacks (LFAs) against critical links. In such attacks, a large number of distributed nodes send low-rate yet coordinated traffic to legitimate destinations, causing attack flows to converge on target links and degrade benign throughput and network availability.

Existing defense mechanisms remain difficult to apply directly to LEO scenarios. Centralized approaches are constrained by the latency of space-ground control loops and cannot respond promptly to evolving attacks. Programmable-data-plane solutions originally designed for terrestrial networks often rely on single features such as aggregate rate and therefore struggle under multi-Decoy dilution, low-rate attacks, and pulsed attacks. Moreover, actions taken only near the victim link cannot prevent wasted bandwidth consumption along upstream paths. To address these issues, this thesis conducts a systematic study from two perspectives: accurate single-satellite identification and cooperative multi-satellite suppression.

To address the difficulty of accurately identifying stealthy LFAs from a single-satellite perspective, this thesis proposes a defense method based on multi-signature in-network detection and queue scheduling. MS-SatShield adopts a cascaded architecture of Top- K candidate generation and multi-dimensional structural feature extraction, jointly exploiting aggregate rate, Fan-in (FI), Fan-out (FO), and Persistence under limited state overhead to establish more reliable detection cues against degraded attacks, while translating detection outputs into differentiated queue scheduling actions. Experimental results show that the proposed method significantly improves the identification of stealthy LFAs, remains robust under typical degraded attacks, and preserves a practical data-plane resource footprint.

To address the limitation that local handling at a single satellite cannot reduce wasted bandwidth on upstream links, this thesis further proposes a cooperative defense method

based on multi-satellite cooperation and hop-by-hop pushback. CoopSatShield aggregates attack evidence within a local region through a suppressed neighbor-gossip protocol and combines it with a hop-by-hop pushback mechanism that moves the defense point from the victim link toward upstream ingress points, thereby extending local mitigation into path-aware suppression. Experimental results show that CoopSatShield can reduce wasted attack occupancy along upstream paths, improve benign throughput during attacks, and maintain low coordination overhead in large-scale constellation environments. Overall, this work provides an in-network security defense optimization framework for large-scale LEO satellite networks that jointly achieves low latency, low resource overhead, and robustness against attack degradation.

Keywords: LEO Satellite Networks; Link Flooding Attack; In-Network Computing; Programmable Data Plane; Cooperative Defense

目 录

第一章 绪论	1
1.1 研究背景与意义	1
1.1.1 卫星网络的发展及战略地位	1
1.1.2 LEO 卫星网络的链路洪泛攻击	2
1.1.3 传统安全防御体系的局限性	3
1.1.4 可编程数据面与在网计算赋能星上安全	4
1.2 国内外研究现状	5
1.2.1 卫星网络安全与动态路由技术	5
1.2.2 链路洪泛攻击检测与缓解技术	6
1.2.3 基于数据面可编程的在网安全防御研究现状	7
1.2.4 现状评述与本文的研究切入点	8
1.3 本文主要研究内容	9
1.3.1 多签名在网检测与单节点缓解	9
1.3.2 多星协同与逐跳推回缓解	10
1.4 本文的结构安排	10
第二章 相关理论与技术基础	11
2.1 大规模低轨卫星网络体系架构与特性	11
2.1.1 巨型星座的网络拓扑与星间链路模型	11
2.1.2 卫星网络的高动态性与快照路由模型	12
2.1.3 面向星上处理的软硬件资源约束	14
2.1.4 高汇聚链路的形成及安全脆弱面	15
2.2 LFA 的攻击模型与对抗分析	15
2.2.1 DDoS 攻击的演进与 LFA 的定位	15
2.2.2 Crossfire 攻击的构造模型	16
2.2.3 LEO 场景下 LFA 的退化策略	17
2.2.4 LFA 攻防中的观测不对称与设计启示	18
2.3 在网计算与 P4 可编程数据面架构	19
2.3.1 数据面可编程能力的演进	19
2.3.2 PISA 架构与匹配-动作流水线	20
2.3.3 数据面有状态处理与存储约束	20

2.3.4 数据面、控制面与星上 Agent 的职责划分	21
2.3.5 面向在网安全的近似数据结构	22
2.4 分布式 Gossip 协议与协同防御基础	23
2.4.1 去中心化协同的必要性与传统共识的代价	23
2.4.2 Gossip 协议的基本原理与传播特性	24
2.4.3 抑制型传播与开销控制	25
2.4.4 Gossip 参数选择与区域协同范围设计	26
2.5 本章小结	27
第三章 基于多签名在网检测与队列调度的 LFA 防御优化	28
3.1 引言	28
3.2 问题定义与方法设计	29
3.2.1 威胁模型与问题表述	29
3.2.2 速率可分性退化与多签名动机	31
3.2.3 系统架构与数据流解释	33
3.2.4 基于 CMS 与候选缓存的一级候选生成	34
3.2.5 多签名可疑度建模与判决	37
3.2.6 FI/FO 在网近似统计结构	38
3.2.7 多签名在网检测与队列调度	40
3.3 实验结果与分析	40
3.3.1 实验平台与基线方案	40
3.3.2 实验参数设置	42
3.3.3 评价指标与绘制方式	44
3.3.4 退化证据链与多签名收益分析	46
3.3.5 消融实验与参数敏感性分析	48
3.3.6 方法适用范围与外推限制	52
3.4 本章小结	52
第四章 基于多星协同与逐跳推回的 LFA 防御优化	54
4.1 引言	54
4.2 问题定义与方法设计	55
4.2.1 问题模型与设计目标	55
4.2.2 CoopSatShield 总体架构	57
4.2.3 抑制型邻居扩散协同机制	59
4.2.4 协同判定与推回机制	63

4.2.5 信令开销与可实现性分析	67
4.3 仿真与分析	68
4.3.1 实验设置	68
4.3.2 结果与分析	69
4.3.3 适用范围与讨论	75
4.4 本章小结	75
第五章 总结与展望	77
5.1 全文总结	77
5.2 未来展望	78
致 谢	79
参考文献	81
攻读硕士学位期间取得的成果	86

图目录

图 3-1 不同 Decoy 数下的聚合速率 CDF。(a) $M = 1$; (b) $M = 10$; (c) $M = 100$; (d) $M = 1000$	32
图 3-2 MS-SatShield 总体架构	34
图 3-3 退化攻击下的检测性能。(a)Rate-only 的一级候选与判决指标; (b) S_0 与 S_1 的 F1 对比	45
图 3-4 脉冲式 on-off 攻击下的时域响应 ($M = 100$)。(a) 目标链路利用率; (b) 一级候选命中的攻击 key 数量; (c) 不同方案的 Recall; (d) 不同方 案的良性吞吐比例	49
图 3-5 候选规模 K 对 Recall 与 F1 的影响	50
图 3-6 位图长度 m 对 FI/FO 估计误差与总内存的影响	51
图 3-7 epoch 长度 Δ 对反应时间与良性吞吐下降的影响	51
图 4-1 CoopSatShield 单节点内部架构	57
图 4-2 EG/PT 协同控制消息格式	60
图 4-3 逐跳推回缓解传播示意	65
图 4-4 不同碎片化程度下的无效带宽占用对比	70
图 4-5 良性吞吐随时间变化 ($M = 100$)	71
图 4-6 协同控制开销与抑制机制效果。(a)EG 消息数随 R 变化; (b) 控制带 宽随 R 变化; (c) 抑制参数 k 的影响	72
图 4-7 碎片化攻击下的表现。(a)F1 与 Recall; (b)Precision; (c) 良性吞吐恢 复; (d) 无效带宽占用	73

表目录

表 2-1 LEO 在网防御体系中不同执行层的职责划分 22

表 2-2 本文所用近似数据结构的功能定位与实现代价对比..... 24

表 2-3 LEO 多星协同候选机制的对比 26

表 3-1 实验主要参数设置（与 SatShield/ICARUS 对齐） 44

表 3-2 退化设定：多 Decoy 分摊（降低按目的聚合速率，放大 FI/FO） 44

表 4-1 Evidence Gossip（EG）消息字段设计 59

表 4-2 Pushback Token（PT）消息字段设计 59

表 4-3 本章实验主要参数设置 69

表 4-4 量化对比基线设置 69

第一章 绪论

1.1 研究背景与意义

1.1.1 卫星网络的发展及战略地位

传统地面通信网络虽然在人口密集区域已形成较成熟的覆盖，但全球仍有约 75% 的陆地面积和逾 90% 的海洋区域处于蜂窝网络无法覆盖的数字盲区^[1]。在卫星通信领域，地球静止轨道（Geostationary Earth Orbit, GEO）卫星部署于约 36 000 km 的赤道正上方，虽然单星可覆盖约三分之一的地球可视面，但其固有的高轨道高度导致星地信号往返传播时延（Round-Trip Time, RTT）不低于约 240 ms，这对实时视频通信、高频金融交易和交互式在线服务等延迟敏感型应用而言是不可接受的。中地球轨道（Medium Earth Orbit, MEO）卫星虽在时延上有所改善，但单星覆盖面积的缩减使其组网成本急剧攀升，至今仅有导航类，如全球定位系统（Global Positioning System, GPS）和 Galileo 及少数通信星座采用该轨道。

低地球轨道（Low Earth Orbit, LEO）的轨道高度通常介于 500km 至 2 000km 之间。在该高度上，单跳星地传播时延可降至 1.7 ms~6.7 ms，往返时延低于 15 ms，首次使卫星通信在延迟指标上具备了与地面光纤网络竞争的潜力。得益于运载火箭可重复使用技术（如 SpaceX Falcon 9 的一级回收）与卫星批量化制造技术的成熟，单颗 LEO 卫星的发射边际成本已从传统 GEO 卫星时代的数亿美元级降至百万美元以下^[1]。这一经济性突破催生了以 Starlink（截至 2025 年已部署逾 7 000 颗）、Amazon Kuiper（计划 3 236 颗）、OneWeb（648 颗）及中国 GW 星座（约 13 000 颗）为代表的巨型星座建设浪潮^[1-3]。

现代巨型星座的一个重要变化在于星间链路（Inter-Satellite Link, ISL）的引入。Starlink 自 v1.5 批次起为每颗卫星安装四部激光 ISL 终端，单链路容量可达 100 Gbps 量级^[4]。通过 ISL，星座中的卫星可直接进行多跳数据中继与路由，无需将每一跳数据下行至地面信关站再上行。这使得 LEO 星座从传统的“空中基站阵列”演变为一张具有独立路由与转发能力的“太空骨干网”^[5]。在某些洲际通信场景中（例如伦敦至纽约），由于光在真空中的传播速度约为光纤中的 1.47 倍，经由 LEO 星间链路中继的路径甚至可以获得比海底光缆更低的端到端时延^[6-9]。正因如此，大规模 LEO 卫星网络已经成为下一代天地一体化信息网络与第六代移动通信愿景中的重要基础设施^[1]。

进入 2025 年后，巨型星座的竞争重点已从部署规模转向持续运营能力。以 Starlink 为例，Direct to Cell 业务继续推进，官方在 2026 年初表示该业务已依托

650 余颗具备直连蜂窝能力的卫星覆盖五大洲。与此同时, SpaceX 于 2026 年发布 Stargaze 空间态势感知系统, 利用星座内近 30 000 个 Star Trackers 每天观测约 3 000 万次过境目标, 可在分钟级给出近碰筛查结果并向外部运营商开放数据。再加上美国联邦通信委员会 (Federal Communications Commission, FCC) 于 2026 年 1 月部分批准 Gen2 Starlink 升级申请, 可以看出 Starlink 的定位已从低轨宽带接入扩展到直连终端和轨道运行保障并行的综合平台^[10-12]。

其他主要参与者同样在推进部署低轨卫星。Amazon Leo (原 Project Kuiper) 于 2025 年 4 月启动全规模部署, 截至 2026 年 2 月已有 200 余颗卫星在轨, 并明确采用高速光学星间链路和 80 余次重型发射的路线。欧盟弹性、互联与安全卫星基础设施计划 (Infrastructure for Resilience, Interconnectivity and Security by Satellite, IRIS²) 已完成特许合同签署, 规划建设由 290 余颗卫星组成的多轨道安全连接体系。Eutelsat OneWeb 已建成 654 颗卫星的一代 LEO 星座, 并把自动化碰撞规避和 2026 年主动清理演示纳入运营计划; 加拿大 Telesat Lightspeed 也在 2026 年推进着陆站建设, 并宣布首星计划于 2026 年 12 月发射^[13-18]。

在中国, 低轨互联网也已从规划转入持续组网。新华社披露, 千帆/Spacesail 星座在 2025 年底已增至 108 颗在轨卫星, 并完成船载网络接入等应用测试; 国网 (Guowang) 星座自 2024 年底首批发射后, 也在 2025 年加快批量入轨。与此同时, 2025 年中国开始推进太空计算星座等面向在轨人工智能 (Artificial Intelligence, AI) 处理和星间组网的新型基础设施, 说明未来 LEO 网络的角色不会停留在宽带转发, 还会继续向通信、计算与感知并存的空间信息系统扩展^[19-23]。

1.1.2 LEO 卫星网络的链路洪泛攻击

作为一个向全球无差别提供接入服务的开放式基础设施, 大规模 LEO 卫星网络不可避免地继承了互联网体系结构固有的安全脆弱性, 并因其独特的物理特征而面临比地面网络更大的攻击风险。具体而言, LEO 星座的物理结构是高度公开和可预测的: 卫星的轨道根数 (Two-Line Element, TLE) 由北美防空司令部和 CelesTrak 等机构向全球公开发布, 其运动轨迹在未来数年内可被精确推算。这种可预测性虽然便利了网络的频率协调与链路预测, 但同时也为恶意攻击者提供了较为精确的目标信息^[24]。

在众多的网络层安全威胁中, 链路洪泛攻击 (Link Flooding Attack, LFA) 对 LEO 星座网络构成了主要威胁之一。LFA 的典型代表为 Crossfire 攻击^[25], 其本质是一种隐蔽性较强的间接拒绝服务攻击变种。与传统分布式拒绝服务 (Distributed Denial of Service, DDoS) 攻击直接向受害服务器倾泻海量流量不同, LFA 的攻击目标并非任何终端设备, 而是网络内部承载大量转发业务的中间链路^[25, 26]。

LFA 的实施机制如下：攻击者首先通过分布式路径探测（如从受控 Bot 向公共服务器发起 Traceroute）绘制目标网络的内部路由拓扑。随后，攻击者从中甄选出一组承载流量最为集中的瓶颈链路集合 $E_{\text{target}} \subset E^{[25]}$ 。在攻击执行阶段，攻击者操纵大量受控僵尸主机（Bot）集合 $\mathcal{B} = \{B_1, B_2, \dots, B_N\}$ ，向分布在全球各地的合法公共服务器，即诱饵目的节点（Decoy）集合 $\mathcal{D} = \{D_1, D_2, \dots, D_M\}$ 发起看似正常的低速（Low-rate）连接。由于路径选择的收敛效应，大量 $B_i \rightarrow D_j$ 通信对的路由路径会在 E_{target} 上重叠汇聚。当汇聚流量的总速率超过瓶颈链路容量 C_e 时，即 $\sum_{(i,j):e \in \text{path}(B_i, D_j)} r_{ij} > C_e$ ，链路 $e \in E_{\text{target}}$ 即发生拥塞，导致途经该链路的所有合法业务遭受较重的丢包与时延退化。

LFA 对 LEO 星座网络的影响更为突出，原因在于以下两点：第一，星间链路的带宽资源相对有限（通常为 10~100 Gbps 量级），远低于地面骨干网的 Tbps 级容量，使得拥塞阈值更容易被突破。第二，由于卫星轨道参数、星座链路结构与部分容量参数可由公开资料推断，攻击者可以在快照尺度预计算攻击路径与流量分配。围绕这一问题，ICARUS 从威胁建模、攻击构造到效果评估对面向 LEO 的链路洪泛攻击进行了系统分析^[24]。更需要注意的是，LEO 拓扑的周期性时变特性使得攻击者可以实施滚动式攻击策略，即随着星座拓扑的演进不断切换攻击目标链路，使防御方难以建立持续有效的防护。近期研究进一步揭示，攻击者还可利用 LEO 拓扑的时变特性，在路径时延切换过程中将分散发送的攻击流量在目标链路上实现时间聚焦，从而以更低的攻击成本达到相同的拥塞效果^[24, 27, 28]。

从可实施性与可检测性的权衡看，ICARUS 进一步给出了可量化指标：攻击成本可由所需总注入带宽衡量，而可检测性可由卫星上行侧最大带宽增量刻画。其结果表明，即使在随机多路径负载分担场景下，攻击者仍可通过增加资源维持较高成功概率。例如，采用更具韧性的路径策略可将中位攻击成本提高约 385%，但会带来较大的路径时延代价。这一结论说明，仅依赖增大路径随机性难以消减 LFA 风险，仍需在数据面部署及时检测与在网缓解机制^[24]。

1.1.3 传统安全防御体系的局限性

面对 LFA 对 LEO 网络构成的持续可用性威胁，直接将地面网络中成熟的安全防御体系迁移至太空环境将遭遇两个维度的基本矛盾。

矛盾一：控制环路时延与攻击自适应速度之间的不对称。传统的网络异常检测与缓解高度依赖软件定义网络（Software-Defined Networking, SDN）的集中式架构。其典型工作流程为：数据面交换机周期性地将流量统计信息（如 sFlow/NetFlow 采样数据）上报至远端控制器，控制器运行检测算法并计算缓解策略后，再将流表规则（Flow Rules）下发至数据面执行。在地面数据中心网络中，控制器与交换

机之间的通信时延通常在 1 ms 量级，这一闭环可以在攻击演化之前完成响应。然而，在 LEO 卫星网络中，星地控制环路的 RTT 受限于物理传播距离，至少为 3.3 ms（500 km 最低轨道），若涉及多跳 ISL 转发或地面网络路由，实际 RTT 通常达到 20~200 ms。相比之下，LFA 攻击者可以在毫秒级时间尺度内通过控制指令重组僵尸主机集合或微调各流的发送速率。这种时延不对称使得依赖地面控制器的“感知—上传—决策—下发”闭环在 LEO 环境中始终处于滞后状态，防御策略抵达卫星时攻击特征已发生漂移^[25, 29]。

矛盾二：星载硬件的物理极限与传统安全引擎的资源需求之间的鸿沟。 LEO 卫星的设计与发射受到尺寸、重量与功耗（Size, Weight, and Power, SWaP）的严格约束。卫星的电力供应主要依赖有限面积的太阳能电池阵列（典型功率预算为数百瓦至千瓦级），热控系统仅能依靠被动辐射散热。因此，星载处理器通常采用高能效比的抗辐射现场可编程门阵列（Field-Programmable Gate Array, FPGA）或宇航级专用集成电路（Application-Specific Integrated Circuit, ASIC），其计算能力和片上存储容量低于地面商用设备。以片上静态随机存取存储器（Static Random Access Memory, SRAM）为例，地面高性能可编程交换芯片（如 Intel Tofino 2）可提供约 ~80 MB 的片上 SRAM，而宇航级芯片的可用 SRAM 通常仅为数 MB 量级^[30]。传统的网络安全引擎（如基于深度学习的异常检测系统或维护 Per-flow 状态表的流量清洗设备），其运行所需的计算量和内存容量远远超出了星载平台的供给能力。在数十 Gbps 的链路吞吐率下，以 Per-flow 粒度维护所有活跃流（可达 $10^6 \sim 10^7$ 条）的精确状态，所需的内存空间约为数百 MB 至 GB 量级，这在星载环境中难以实现^[30, 31]。

2025 年至今的产业实践进一步说明，更多近实时控制闭环正在前移到轨道侧。Starlink 已将空间态势感知与星座运行耦合，Amazon Leo、IRIS²、OneWeb 等系统也在面向企业、政府和基础设施业务提高连续性要求。对安全防御而言，检测与缓解若仍主要留在地面，不仅受传播时延限制，也难与星座正在形成的轨道侧自治运行和多业务并发支撑方式相适配^[11, 15, 16, 32]。

1.1.4 可编程数据面与在网计算赋能星上安全

上述两重矛盾指向一个基本判断：有效的 LEO 防御方案必须满足两个基本约束——（1）在数据面就地完成检测与缓解的闭环，绕过高时延的星地控制环路；（2）算法的计算复杂度与状态空间消耗必须适配星载硬件的 SWaP 约束。近年来，可编程数据平面与在网计算（In-Network Computing, INC）技术的发展为满足上述约束提供了可行的技术基座^[33, 34]。

可编程协议无关报文处理器（Programming Protocol-independent Packet Proces-

sors, P4) 语言赋予了网络交换芯片在维持纳秒级线速转发性能的同时, 执行由用户自定义的包处理逻辑的能力。P4 程序运行于协议无关交换架构 (Protocol Independent Switch Architecture, PISA) 的硬件流水线之上。在该架构中, 数据包经过可编程的解析器 (Parser) 提取报文头字段后, 依次通过由多级匹配-动作单元组成的入口流水线和出口流水线。每个 MAU 级可在单个时钟周期内完成一次 TCAM/SRAM 查表与一组 ALU 算术逻辑操作。通过在流水线中部署寄存器 (Registers)、计数器 (Counters) 等有状态存储元素, 交换芯片能够以 $O(1)$ 的时间复杂度对每个经过的数据包进行状态化处理^[35]。

在网计算的基本思路是: 将特定的高频、轻量级计算任务从终端或控制器卸载至网络内部的交换节点, 使计算与转发在数据路径上融为一体^[33]。将这一理念应用于 LEO 卫星安全防御, 可实现两方面的结构性优势。其一, 星载 P4 交换机可在数据面就地完成流量特征提取、异常判定与丢包缓解的闭环操作, 形成额外时延很小的本地响应能力, 从而规避星地控制环路的时延瓶颈。其二, 基于流水线架构的近似数据结构 (如 Bloom Filter、Count-Min Sketch) 仅消耗数百 KB 至数 MB 的片上 SRAM, 即可实现对大规模流量的近似统计与追踪, 符合星载平台的 SWaP 约束^[30, 31]。

综上, 利用在网计算技术在 LEO 卫星的数据面构建安全检测与缓解能力, 将对抗 LFA 攻击的第一道防线推进至网络内部的转发节点, 是解决上述两重矛盾的主要技术路径。这一判断构成了本文的研究动机。

1.2 国内外研究现状

1.2.1 卫星网络安全与动态路由技术

LEO 卫星网络的路由与安全是两个交织发展但进度严重失衡的研究方向。在路由方面, 由于卫星的高速运动 (LEO 轨道速度约 7.5 km/s , 轨道周期约 $90 \sim 120 \text{ min}$) 导致物理拓扑持续变化, 传统的基于链路状态通告 (Link State Advertisement, LSA) 的路由协议若直接套用, 将产生每分钟数百至数千次的路由重收敛事件, 造成控制面信令风暴。为此, 学术界提出了多种适应性方案: (1) 基于离散时间快照 (Snapshot) 的路由预计算方案, 利用轨道可预测性将连续时间划分为一系列离散时段, 在每个时段开始时加载预计算的转发表, 避免实时路由收敛^[7-9, 36, 37]; (2) 基于虚拟拓扑的方案, 将物理 ISL 拓扑映射为逻辑上相对固定的虚拟节点关系, 在逻辑层面维护路由表; (3) 基于地理位置的分布式寻址路由, 利用卫星的地理坐标作为路由标识, 实现无需全局拓扑信息的贪婪转发。

相比之下, LEO 网络的安全防御研究发展相对滞后。现有研究大量集中于卫

星-地面链路的接入认证（如基于物理层特征的认证、抗量子密钥分发协议）和信道加密，而针对星间链路层面的数据面攻击防御（特别是 DDoS 和 LFA 类流量攻击）的在轨研究仍相对不足^[38]。已有工作开始讨论卫星网络 DDoS 缓解，但多聚焦于传统卫星接入或非 ISL 场景，尚难直接覆盖 LEO 星座内的跨 ISL 路径拥塞攻击^[39]。目前工程实践中的主流做法仍是将可疑流量绕路至地面流量清洗中心，即通过星地链路将攻击流量引导至地面的高性能清洗设施进行过滤后再回注。这一架构存在三个固有缺陷：（1）绕路增加了额外的星地传播时延（至少一个 RTT，约 3~15 ms），与 LFA 的毫秒级自适应能力不匹配；（2）星地下行链路本身的带宽有限，当遭受大规模攻击时，下行回传通道本身可能成为新的拥塞瓶颈；（3）这一方案对地面基础设施的依赖使其在星地链路中断（如极端天气或对抗性干扰）时失去作用^[38, 40]。

近两年，相关研究开始把关注点从事后检测扩展到路由层面的韧性设计。例如，Meng 等人在 2025 年提出 K-Bottleneck Minimize 路由，通过多路径生成过程中规避统计意义上的瓶颈路径，使攻击成本平均/中位分别提升约 13.1% 和 16.6%，同时提高攻击可检测性；Hu 等人在 2026 年提出 PathWeaver，将拓扑设计与随机化路由联合优化，在不同星座规模下最大链路利用率的降幅降最多可达 25.54%，LFA 攻击成本增加最高达 523%。这些结果说明，仅靠攻击发生后的检测仍不够，拓扑、路由与安全之间的联动正在成为新的研究方向^[41, 42]。

1.2.2 链路洪泛攻击检测与缓解技术

针对 LFA 这一威胁，地面互联网领域已发展出多个具有代表性的防御框架，可按其技术路线划分为三大类别。

第一类：基于主动测量的检测方案。以 LinkScope^[43] 为代表，其核心思想是由网络运维方主动向可疑链路注入探测包，通过分析探测包的丢包率、时延抖动等端到端性能指标来推断链路是否正在遭受洪泛攻击。该类方案的主要局限在于：（1）探测流量本身占用链路带宽，在带宽资源极为稀缺的 ISL 上会加剧拥塞；（2）在 LEO 的高动态拓扑下，探测路径会因拓扑切换而频繁失效，导致测量结果不连续；（3）LFA 攻击者可以通过控制攻击强度使其刚好低于探测阈值，从而规避探测。

第二类：基于集中式 SDN 控制器的检测与流量工程缓解方案。以 Woodpecker^[44] 和 RADAR^[45] 为代表。Woodpecker 通过 SDN 控制器集中采集全网交换机的流量统计信息，构建全局流量矩阵（Traffic Matrix, TM），利用矩阵分解与统计异常检测识别受攻击链路，随后通过求解线性规划（Linear Programming, LP）问题计算最优的流量重路由策略。RADAR 在此基础上引入了基于博弈论的自适应对抗模型，将攻防双方建模为零和博弈，通过纳什均衡求解最优防御策略。此

类方案的主要瓶颈在于其对中心控制器的强依赖：（1）全局流量矩阵的收集周期通常为秒级，加上 LP 求解的计算耗时，端到端的响应时延远超 LFA 的自适应速度；（2）在 LEO 环境中，星地控制环路的额外时延进一步恶化了响应速度；（3）控制器本身可能成为单点故障。

第三类：基于 P4 可编程数据面的分布式检测方案。 Ripple^[46] 提出了一种在 P4 交换机上实现的去中心化 LFA 防御框架，通过在每台边缘交换机的数据面部署轻量级的流量监控逻辑，避免了对集中式控制器的依赖，并利用 P4 的线速处理能力实现了毫秒级的响应速度。Mew^[47] 进一步扩展了 P4 数据面的检测能力，引入了基于可编程交换机的大规模动态 LFA 防御机制，支持在交换机间协同共享检测状态。然而，这两个方案均是针对地面数据中心或互联网服务提供商骨干网设计的，其隐含假设包括：（1）交换机具有充裕的 SRAM/TCAM 资源；（2）网络拓扑变化较小，设备间的协同通信可靠且低延迟；（3）可以灵活地通过控制面动态更新 P4 程序和流表。这些假设在 LEO 环境中均不成立。特别是，当面对 LFA 的退化策略（如多靶机分摊、低速微流化）时，现有 P4 方案仅依赖聚合速率（Rate-only）作为单一检测特征，无法有效区分隐蔽的 LFA 流与合法的长尾大流，导致较高的漏报率和误报率^[46, 47]。

1.2.3 基于数据面可编程的在网安全防御研究现状

P4 可编程技术在网络安全领域的应用已取得持续进展，主要集中在以下方向。

在网络遥测方面，带内网络遥测（In-band Network Telemetry, INT）技术利用 P4 的协议无关性，在数据包头中嵌入沿途各交换机的内部状态信息（如入口/出口时间戳、队列占用深度、链路利用率等），实现了对网络状态的线速、逐包、全路径可视化。INT 为异常检测提供了更细粒度的数据源，但其在 LEO 环境中的应用面临挑战：INT 信息的嵌入增加了数据包的开销（每跳通常增加 8~16 Bytes），导致带宽受限的 ISL 运行状态下降^[35]。

在数据面异常检测方面，已有多项工作探索了在 P4 交换机上实现实时 DDoS 检测的可行性。例如，ddosd-p4 项目在 P4 数据面上实现了基于信息熵的异常检测：利用 Count-Sketch 数据结构对源/目的 IP 的频次分布进行近似估计，通过预计算的离散化查找表将频次映射为熵贡献值，并在观测窗口结束时利用指数加权移动平均与指数加权移动平均绝对偏差进行统计异常判定。该方案证明了在 P4 数据面实现统计检测的技术可行性，但其熵检测依赖于攻击流量导致 IP 分布发生可见偏移这一假设。在 LFA 的多靶机分散策略下，攻击流量的目的 IP 分布可能被刻意分散化，使得熵值变化不明显，从而导致漏检^[31, 48]。

在数据面缓解方面，基于 P4 的可编程防火墙和速率限制器已在地面网络中得

到验证^[31, 49]。通过 P4 的 Meter 原语, 交换机可对匹配特定规则的流量执行令牌桶限速或直接丢弃, 实现线速级的攻击流量过滤。然而, 这些方案通常假设攻击流量可以通过五元组或源 IP 前缀等简单规则精确匹配——这一假设在 LFA 场景中并不成立, 因为每条攻击微流都使用合法的源地址和目的地址, 且流五元组与正常业务无异^[25]。此外, Patronum^[50] 和 ACC-Turbo^[49] 分别从体量化检测和聚合拥塞控制的角度进一步验证了可编程交换机在 DDoS 防御中的实用性, 但同样面临 LFA 微流隐蔽性带来的特征匹配难题。

与此同时, P4 生态也在继续扩展。P4 项目自 2024 年起由 Linux Foundation 托管, P4₁₆ 语言规范在 2024 年更新到 v1.2.5, P4Runtime 先在 2024 年更新到 v1.4.1, 并在 2026 年继续演进到 v1.5.0; 可移植网卡架构 (Portable NIC Architecture, PNA) 及其相关实现也在把 P4 扩展到智能网卡 (Smart Network Interface Card, SmartNIC)、数据处理器 (Data Processing Unit, DPU) 和人工智能网卡 (Artificial Intelligence Network Interface Card, AI NIC) 等目标。这些变化说明, P4 更适合作为描述异构数据面的统一抽象, 而不是绑定某一种交换芯片^[51, 52]。

从社区议题看, 2025 年 P4 Workshop 又把重点延伸到 AI 互连、P4 与扩展伯克利包过滤器 (extended Berkeley Packet Filter, eBPF) / 数据平面开发套件 (Data Plane Development Kit, DPDK) 的协同以及 SmartNIC 上的虚拟交换缓存。再结合 2025 年 Intel 将 Tofino P4 软件开源、Cisco 强调 Silicon One 的 P4 可编程性、Xsight Labs 开放 X-Series 交换芯片 ISA, 以及 NVIDIA BlueField/ConnectX-9 和 AMD Pensando Pollara 400 提供的相关目标架构, 可以看到可编程数据面的讨论范围已从交换机扩展到 NIC、交换机与加速器共同参与的系统环境。对本文而言, 这意味着未来星载可编程平台更可能由 FPGA、DPU、NIC 与专用交换模块共同构成, 因此按 P4 抽象层组织检测和缓解逻辑, 会比围绕单一芯片型号构造方案更易迁移^[53-58]。

1.2.4 现状评述与本文的研究切入点

综合上述三个方向的研究现状, 可以提炼出当前 LEO 网络 LFA 防御面临的三重矛盾:

- (1) 时延矛盾: 地面集中式防御方案的端到端响应时延 (秒级) 远慢于 LFA 攻击自适应切换速度 (毫秒级), 且 LEO 星地控制环路的额外时延会进一步恶化该问题。
- (2) 视野矛盾: LFA 攻击流量以碎片化形式分散穿越多条 ISL, 任何单颗卫星仅能观测到途经本节点的局部流量切片, 无法仅凭局部信息准确区分攻击与正常业务。
- (3) 资源矛盾: 有效检测隐蔽 LFA 需要提取多维度流量特征 (如扇入度、扇出

度、持续性等), 而 P4 流水线的计算能力和 SRAM 容量严格受限于 SWaP 约束, 传统的多特征检测方案面临状态爆炸风险。

上述三重矛盾在已有文献中已得到不同程度的揭示^[30, 46, 47]。面对上述三重矛盾, 本文摒弃传统的“感知—上传—中央清洗”范式, 确立了轻量级数据面在网状态近似与不依赖中心控制的多星去中心化协同相结合的技术路线作为研究切入点, 主要解决如下两个问题:

- 问题一 (对应第三章): 如何在 P4 流水线的严格计算与存储约束下, 设计一种兼顾低资源开销与高检测精度的多维度流量特征在网提取方法, 并将检测结果映射为低开销、低误伤的队列调度策略, 实现对隐蔽 LFA 攻击的单星防御优化?
- 问题二 (对应第四章): 如何在无中心控制器参与条件下, 设计一种低信令开销、适应性更强的多星分布式协同机制, 将单星的局部检测结果聚合为全网一致的攻击判定, 并通过从受害链路向攻击源头方向的逐跳推回实现全网防御优化?

1.3 本文主要研究内容

立足上述研究动机与问题定义, 本文利用 LEO 卫星网络高度结构化且可预测的时变拓扑特性, 以在网计算为主要技术手段, 为大规模 LEO 星座构建了一套从单节点检测到多节点协同推回的安全防御体系。主要研究内容包括以下两个方面。

1.3.1 多签名在网检测与单节点缓解

本部分研究针对单节点在 P4 数据面上进行 LFA 检测时面临的特征退化困境, 提出了一种基于多签名在网检测与队列调度的 LFA 防御优化方法。该方法的主要设计是一种级联式的两级检测流水线架构: 第一级采用“计数最小草图 (Count-Min Sketch, CMS) 频率估计 + 固定容量候选缓存”的 sketch-based 候选生成机制, 利用 CMS 以固定状态维护全局近似频次, 并由显式候选缓存保留当前受监视的 key 身份, 从海量流量中近似恢复速率异常候选集合; 第二级在候选集上部署双缓冲位图结构, 对每个候选流在线提取扇入度、扇出度和历史持续性等多维结构特征签名。多签名的联合评分机制使得即便攻击者通过退化策略使单一维度的特征隐身, 仍可因其他维度的异常被识别。在防御优化侧, 本方法设计了一种高/低优先级两队列的自适应降权策略, 根据多签名综合可疑度对可疑流进行差异化的队列优先级调整, 在保护合法业务的同时对攻击流施加吞吐抑制。

1.3.2 多星协同与逐跳推回缓解

本部分研究针对单星检测视野局限性与“被动就地丢弃”模式的不足，提出了一种基于多星协同与逐跳推回的 LFA 防御优化方法。该方法包含两个组件：（1）抑制型流言传播协议，通过在 LEO 星座的 Mesh 拓扑中设计一种受控的、基于事件触发且带冗余抑制的邻居间信息扩散机制，在严格限制信令开销的前提下实现局部区域内多颗卫星之间的攻击证据交叉验证与聚合；（2）逐跳推回机制，一旦区域内的多星协同判定达成共识，受攻击链路的卫星节点将生成携带攻击特征签名的推回令牌，沿攻击流的反向路径逐跳回传至上游节点，指示路径上的各卫星在其数据面对匹配的攻击流执行限速或丢弃操作。这一机制使防御阵线从被动的受害点丢弃转变为向源头方向推回，减少了攻击流在网络中的无效传输，提高了全网的带宽利用效率。

1.4 本文的结构安排

本论文共分为五章，各章内容安排如下：

第一章绪论。阐述了大规模 LEO 卫星网络的发展现状及其面临的 LFA 安全威胁，分析了传统防御体系在 LEO 环境中的基本局限，引出了基于在网计算的星上内生安全防御思路。在此基础上，系统综述了卫星网络安全、LFA 防御和 P4 在网安全三个方向的国内外研究现状，梳理了现有工作的不足与研究空白，明确了本文的研究切入点和主要研究内容。

第二章相关理论与技术基础。详细介绍了 LEO 巨型星座的网络拓扑模型与快照路由机制，深入分析了 LFA 的攻击构造模型与退化策略，系统阐述了 P4/PISA 可编程数据面架构的工作原理、计算约束与有状态处理能力，并引入了分布式 Gossip 协议及其抑制型变种的理论基础。

第三章基于多签名在网检测与队列调度的 LFA 防御优化。阐述 MS-SatShield 的系统设计、算法细节与实验验证。重点介绍基于 Top- K 候选锁定与多签名在网提取的级联检测流水线，以及基于动态降权的队列调度策略，并通过大规模仿真实验验证其在多种 LFA 退化策略下的检测精度与吞吐恢复效果。

第四章基于多星协同与逐跳推回的 LFA 防御优化。阐述 CoopSatShield 的协同框架设计与实验评估。重点介绍抑制型 Gossip 协议的信令开销控制机制、多星证据交叉验证的判定逻辑，以及逐跳推回令牌的生成与传播机制，并通过跨区域仿真实验验证其不同攻击规模和网络状态下的协同效率与可扩展性。

第五章总结与展望。总结全文的研究成果与主要贡献，并对 LEO 网络在网安全防御的未来研究方向进行展望。

第二章 相关理论与技术基础

本论文所提出的多签名在网检测和多星协同推回缓解的设计与实现，依赖于四个领域的理论支撑：大规模 LEO 星座的网络拓扑与约束模型、链路洪泛攻击的构造机理与对抗博弈、P4 可编程数据面的架构原理与计算约束、以及分布式 Gossip 协议的传播理论与开销控制。本章将逐一展开论述，为后续章节建立可直接引用的理论基础。

2.1 大规模低轨卫星网络体系架构与特性

2.1.1 巨型星座的网络拓扑与星间链路模型

现代 LEO 巨型星座的网络拓扑可从两个层面进行形式化描述：物理构型层面与网络图论层面。

在物理构型层面，巨型星座的卫星分布遵循 Walker 星座模型。以 Walker Delta 构型为例（如 Starlink 第一阶段主体），其参数化表示为 $W(\delta : T/P/F)$ ，其中 δ 为轨道倾角， T 为卫星总数， P 为轨道平面数， F 为相邻轨道间的相位因子。对于 Starlink Shell 1 ($\delta = 53, T = 1584, P = 72, F = 1$)，每个轨道平面承载 $T/P = 22$ 颗卫星，轨道高度 $h = 550$ km。Walker Star 构型（如 Telesat Lightspeed, $\delta = 99.5$ ，近极地轨道）则在极地地区存在轨道交叉区，形成拓扑上的特殊汇聚点。两种构型的共同特征是：在同一轨道平面内，相邻卫星之间通过同轨星间链路相连，链路长度变化较小；不同轨道平面之间，通过异轨星间链路相连，其长度随卫星纬度位置的变化而周期性变化^[4]。在高纬度区域（特别是极圈附近），异轨 ISL 由于相邻轨道平面间距过大或卫星相对速度过高，可能周期性中断^[1, 59]。

在网络图论层面，LEO 星座在任意给定时刻 t 的网络拓扑可建模为时变有向图 $G(t) = (V(t), E(t))$ ，其中 $V(t)$ 为当前在轨运行的卫星节点集合， $E(t)$ 为当前活跃的 ISL 集合。对于典型的四端口 ISL 设计（前后两条同轨 ISL、左右两条异轨 ISL），除极地交叉区外，每个卫星节点的网络度数约为 4。这种规则的格状拓扑结构意味着：（1）任意两颗卫星之间的最短路径跳数可由其在网格中的曼哈顿距离近似估计；（2）当某条主干 ISL 发生拥塞时，替代路径的跳数增量相对可控，但同时也意味着拥塞可能沿网格的行或列方向传播，引发连锁效应^[8, 24]。

每条星间链路 $e \in E(t)$ 具有有限的链路容量 C_e （通常为 10~100 Gbps 量级）。定义链路 e 在时刻 t 的利用率为 $\rho_e(t) = \lambda_e(t)/C_e$ ，其中 $\lambda_e(t)$ 为 e 上的瞬时聚合流量速率。当 $\rho_e(t) > 1$ 时，链路进入拥塞状态，数据包在输出缓冲队列中排队等待

或被丢弃。LFA 攻击的目标正是通过人为制造 $\rho_{e^*}(t) > 1$ ($e^* \in E_{\text{target}}$) 来使目标链路 e^* 瘫痪^[25]。这一量化模型为第三章的检测阈值设定和第四章的推回触发条件提供了形式化基础。

从 2025 年至今的工程实践看, LEO 星座的拓扑与链路设计目标已由低时延互联网接入扩展到统一承载多类业务。Starlink 的 Direct to Cell 已面向现网长期演进 (Long Term Evolution, LTE) 终端提供直连蜂窝覆盖, Amazon Leo 进一步面向企业和政府场景提供私网互连、直达云接入和千兆级终端, 欧盟 IRIS² 与 Eutelsat OneWeb 则在体系设计中引入了主权连接、政府业务连续性以及自动化空间交通安全等要求^[10, 14-16, 32]。这意味着, 同一套 ISL 拓扑不会长期承载近似平均的互联网流量, 业务形态和运行策略的差异会逐步拉开链路热点与风险分布。

进一步地, SpaceX 在 2026 年公布的 Stargaze 系统以及 OneWeb 的自动化避碰实践表明, 巨型星座已经把通信转发、空间态势感知和空间交通管理放进同一运营闭环^[11, 16]。因此, 在网络建模时, LEO 图 $G(t)$ 不能只停留在可达性和最短路层面, 还应分析哪些链路会同时承担更重的业务转发、运维控制和安全暴露压力。后文围绕高汇聚链路展开的检测和缓解设计, 正是基于这一理解。

从网络安全视角看, Walker 型巨型星座的风险分布并不均匀。尽管同轨与异轨 ISL 在物理构型上高度规则, 但在业务需求分布、信关站部署、低时延路由偏好以及极区链路可用性波动的共同作用下, 不同链路被端到端路径选中的概率仍会出现差异。若将某一快照时刻的活跃端到端路径集合记为 $\mathcal{P}(t)$, 则可定义链路 e 的路径承载比:

$$\eta_e(t) = \frac{1}{|\mathcal{P}(t)|} \sum_{p \in \mathcal{P}(t)} \mathbf{1}(e \in p), \quad (2-1)$$

其中 $\mathbf{1}(\cdot)$ 为指示函数。对长期处于高 $\eta_e(t)$ 区域的链路而言, 即便其物理容量并未低于其他链路, 也更容易因路径汇聚而成为业务热点和攻击目标。LEO 星座中的高风险链路, 突出特征在于它们会在路由层面反复被最短路或低时延路径选中, 即使其物理层资源并不更为稀缺^[5, 8, 24]。

2.1.2 卫星网络的高动态性与快照路由模型

LEO 卫星以约 7.5 km/s 的速度运行 (对于 $h = 550$ km 的圆轨道, 轨道周期约 $T_{\text{orbit}} = 2\pi\sqrt{(R_E + h)^3/\mu} \approx 95.6$ min, 其中 $R_E = 6371$ km 为地球半径, $\mu = 3.986 \times 10^{14} \text{ m}^3/\text{s}^2$ 为地心引力常数), 这一高速运动带来两个层面的动态性^[1]。

第一层面为星地覆盖的动态性: 每颗卫星的地面覆盖区持续移动, 地面终端需要每隔数分钟执行一次卫星切换。第二层面为星间拓扑的动态性: 异轨 ISL 的物理可用性和链路质量随卫星间相对位置的变化而波动; 在极地区域, 异轨 ISL 的

周期性中断导致网络在该区域的连通性呈现间歇性特征。

为应对上述动态性，学术界广泛采用离散时间快照模型。其基本思路是利用 LEO 轨道运动的可预测性，将连续时间轴划分为一系列等长的离散时段 $\{[\tau_k, \tau_{k+1}) | k = 0, 1, 2, \dots\}$ ，时段长度 $\Delta\tau = \tau_{k+1} - \tau_k$ （典型值为 10~30 s，如 Starlink 约 15 s）。在每个时段 $[\tau_k, \tau_{k+1})$ 内，网络拓扑 $G(\tau_k)$ 被视为静态不变，卫星在时段开始时加载由地面网络管理系统预先计算并上注的转发信息表（Forwarding Information Base, FIB）。时段结束时，所有路由状态同步更新至下一时段的 FIB^[8, 60]。

ICARUS 的攻击建模进一步说明了该快照假设的安全含义：在链路最短路变化以秒级发生、而单次端到端转发时延远小于快照长度的条件下，攻击者可将连续时变系统近似为一系列静态快照，并提前计算多时段攻击流分配。因此，拓扑动态性本身并不会自动抬高攻击门槛，决定因素仍是防御端能否在相近时间尺度内完成检测和响应闭环^[24]。

快照模型会直接影响本文的防御设计。第三章中的在网检测算法可以将状态累积周期（检测窗口 W ）与快照时段 $\Delta\tau$ 对齐，在每次拓扑切换时对检测状态表进行清洗，从而避免跨拓扑期间的状态污染，并保持窗口内统计的一致性。第四章则利用同一快照时段内路由路径确定这一条件，使推回令牌能够沿已知的反向路径精确回传，无需实时路径发现机制。

快照路由模型之所以能够成为 LEO 网络研究中的主流抽象，一方面是因为它便于仿真，另一方面是因为它较好贴合了该类网络中的多时间尺度分离事实：单个数据包的端到端转发时延通常处于毫秒级，检测窗口和局部协同过程多位于数百毫秒到数秒量级，而由轨道运动驱动的拓扑变化则常发生在秒级到十秒级。因而在多数工程场景下可近似满足：

$$T_{\text{pkt}} \ll W \lesssim \Delta\tau \ll T_{\text{orbit}}, \quad (2-2)$$

其中 T_{pkt} 表示包级转发时间尺度， W 表示检测或聚合窗口， $\Delta\tau$ 表示快照长度， T_{orbit} 表示轨道周期。该不等式意味着：在单个快照内部，防御系统通常可以把转发路径视为近似静态，从而在局部完成状态累积、异常判定与缓解动作，而不必在每个数据包层面重新面对拓扑不确定性^[8, 24, 60]。

然而，快照模型也带来两个常被忽略的安全问题。若检测窗口 W 远大于 $\Delta\tau$ ，窗口中的流量统计就可能跨越两套不同转发表，同一个键（key）在窗口内部的统计含义会发生漂移，进而破坏阈值判决的一致性。若 FIB 切换并非所有卫星同时完成，而是存在短暂的不一致阶段，攻击者又可能利用切换时刻制造瞬时的路径重分布，使局部观测进一步碎片化。基于这一点，本文后续设计中强调将检测窗口、

位图双缓冲翻转、协同证据寿命和推回软状态寿命尽量与快照时隙（epoch）对齐，以减少状态仍属于旧拓扑、策略却已作用于新拓扑的错位现象。

2.1.3 面向星上处理的软硬件资源约束

为了将安全检测与缓解逻辑部署于卫星平台，航天工程的 SWaP 约束是计算与存储资源的上限^[1, 30]。

在功耗维度，LEO 卫星的电力供应主要依赖太阳能电池阵列。以典型的中型通信卫星为例，其太阳能帆板的发电功率约为 1~5 kW，而这一功率预算需在通信载荷（星载天线、射频前端）、计算单元、姿态控制系统和热控系统之间分配。分配给网络交换与处理单元的功耗预算通常不超过数十瓦。作为对比，地面数据中心的高性能可编程交换芯片（如 Intel Tofino 2, 12.8 Tbps）的典型功耗约为 200~300 W，仅此一项即可能超出整颗小型卫星的总功率预算^[1, 35]。

在存储维度，地面 Tofino 2 芯片提供约 80 MB 的片上 SRAM 和 40 Mb 的 TCAM。而宇航级 FPGA（如 Xilinx Virtex-5QV 系列抗辐射器件）的片上块随机访问存储器（Block Random Access Memory, BRAM）容量通常仅为数 MB 量级。如果以 Per-flow 粒度维护流状态表（假设每条流需要 ~64 Bytes 的状态信息，活跃流数量为 N_f ），则精确状态维护所需的内存为 $M_{\text{exact}} = 64N_f$ Bytes。当 $N_f = 10^6$ （中等规模流量）时， $M_{\text{exact}} = 64$ MB，这已远超宇航级芯片的片上 SRAM 容量^[30, 35]。

在计算维度，PISA 流水线的每个 MAU 级仅能在单个时钟周期内执行有限的 ALU 操作：哈希计算、整数加减法、位移、按位逻辑运算（AND/OR/XOR）。不支持浮点运算、乘除法、条件循环（for/while）或递归调用。这意味着任何需要迭代收敛（如梯度下降）或复杂数学运算（如对数、三角函数）的算法均无法在数据面直接实现^[61]。

上述约束共同构成了本文算法设计的限制条件：所有在网检测逻辑必须是单遍（Single-pass）、无循环的，且状态空间消耗必须严格控制在 MB 量级以内。这一约束驱动了第三章采用 CMS 频率估计、固定容量候选缓存与 Bitmap 组合而成的级联两级检测架构^[30]。

需要说明的是，本文讨论的星载可编程数据面并不是将传统卫星广域网的全部控制逻辑下沉到 P4 数据面，而是仅将 LFA 防御中少量、固定复杂度的安全功能放入转发快路径。SatShield 的原型实现已经验证了这一思路的可行性：其在 Intel Tofino Wedge 100BF32X 上实现 P4 数据面原型，支持记录 top 10k heavy flows，资源开销约为 6.11% SRAM、3.50% VLIW、5.82% ALU 和 12.5% hash units，相关逻辑分布在约 12 个硬件 stage 中。与仅执行转发的 P4Hello 基线相比，SatShield 额外引入约 1.45 μ s 的流水线处理时延，但二者吞吐均约为 99.48 Gb/s，说明该类窄功

能安全逻辑在满足流水线资源约束时可以保持线速处理^[30]。本文沿用这一工程边界：数据面只承担常数复杂度的逐包统计与队列标记，复杂评分、阈值更新和 rank 计算由星上轻量 Agent 在 epoch 边界完成，从而避免全量 per-flow 状态和复杂控制逻辑进入转发快路径。

2.1.4 高汇聚链路的形成及安全脆弱面

巨型星座的几何拓扑虽然规则、稀疏且低度数，但业务转发并不会平均铺开，网络中会逐渐出现一批长期承压的高汇聚链路。端到端需求矩阵本身就存在空间不均衡：跨洲际流量、热点城市对业务、海洋与偏远区域回传，以及大型信关站附近的聚合流量，都会持续抬高某些区域路径的使用频度。低时延优先的路由策略又会反复选择跳数更短、传播距离更小或地面绕行更少的路径，使部分跨轨道或跨区域链路长期处于高复用状态。再加上极区异轨链路的周期性中断会削弱局部路径多样性，更多流量被迫通过仍可用的替代链路中继，热点效应会进一步放大^[4, 7, 8]。

为了描述这一长期趋势，可定义链路 e 在连续 K 个快照内的平均压力：

$$\bar{\rho}_e = \frac{1}{K} \sum_{k=1}^K \rho_e(\tau_k), \quad (2-3)$$

其中 $\rho_e(\tau_k)$ 为快照 τ_k 时刻链路利用率。若某条链路在多个快照上持续表现为高 $\bar{\rho}_e$ 、高路径承载比和较弱的替代路径弹性，则可将其视为结构性脆弱点。这类链路平时未必拥塞，更多时候只是长期负载偏高；一旦遭遇利用路径汇聚的 LFA，有限的带宽余量会很快被耗尽，链路也最先失稳。

这一机制与地面骨干网中的热点链路现象相似，但在 LEO 场景中更突出。一方面，星座拓扑可预测，攻击者可以在快照尺度上提前推测热点链路的演化轨迹，而不必像地面互联网那样面对高度不透明的域间路由。另一方面，LEO 星座的平均节点度数较低，单条链路失效或拥塞后，替代路径的跳数膨胀与负载转移会更突出，局部链路也更容易出现级联拥塞。因此，LFA 在 LEO 中除了使目标链路达到饱和，还可能借由绕行效应拖累相邻链路。

2.2 LFA 的攻击模型与对抗分析

2.2.1 DDoS 攻击的演进与 LFA 的定位

分布式拒绝服务（Distributed Denial of Service, DDoS）攻击的演进可划分为三个世代。第一代是体积型攻击，以 UDP 反射放大（如 DNS/NTP/Memcached 反射）和 SYN Flood 为代表，其目标是以绝对的流量体量耗尽被攻击端的接入带宽或处

理能力。此类攻击特征明显（流量突增、协议异常），可被基于简单阈值和五元组黑名单的流量清洗中心有效拦截。第二代为应用层攻击，如 HTTP GET/POST Flood 和 Slowloris，其流量体量较小但精准打击应用层资源（如 Web 服务器的并发连接数或数据库查询能力）。此类攻击需要更细粒度的应用层检测逻辑，但攻击流量仍然汇聚于被攻击终端，终端侧的异常可被直接感知^[38]。

LFA 代表了第三代攻击范式的变化。它与前两代攻击的主要差异在于，被攻击对象不再是终端节点，而是网络内部的中间链路。由于攻击流量的源地址和目的地址均为合法地址，且每条流的速率和行为模式与正常业务无异，终端侧无论是 Bot 还是诱饵目的节点（Decoy）都难以感知异常。真正出现拥塞的位置位于中间链路，而这恰恰是传统终端侧安全设备的监控盲区。这一特点使 LFA 成为当前隐蔽性较强的网络层攻击手段之一^[25, 26]。

在 LEO 场景下，评估 LFA 攻防效果通常至少需要两个互补指标：一是攻击成本，即达到目标拥塞所需的总注入带宽；二是可检测性代理指标，如攻击导致的上行最大增量 maxUp。前者反映攻击者的资源门槛，后者对应运维侧能够观测到的异常幅度，有助于避免仅凭单一成功率指标得出片面结论^[24]。

2.2.2 Crossfire 攻击的构造模型

Crossfire^[25] 是 LFA 的经典实例，其攻击过程可形式化为三阶段模型。

阶段一：拓扑测绘与瓶颈链路识别。攻击者控制分布在全球各地的僵尸主机集合 $\mathcal{B} = \{B_1, \dots, B_N\}$ ，从每个 B_i 向目标区域附近的公共服务器，即诱饵目的节点集合 $\mathcal{D} = \{D_1, \dots, D_M\}$ （Decoy，如 CDN 节点、DNS 根服务器）执行 Traceroute，从而获取 $N \times M$ 条路径信息。通过对这些路径的重叠分析，攻击者可构建出一张目标网络的链路使用频率图，并从中识别出边介数中心性最高的链路子集 E_{target} 。对于任意链路 e ，其边介数中心性定义为：

$$\text{BC}(e) = \sum_{(i,j)} \frac{\sigma_{ij}(e)}{\sigma_{ij}}, \quad (2-4)$$

其中 σ_{ij} 为 B_i 到 D_j 的最短路径总数， $\sigma_{ij}(e)$ 为其中经过链路 e 的路径数。BC(e) 值越高，表明该链路承载的路径汇聚度越大，则为 LFA 目标^[25]。

阶段二：Bot-Decoy 通信对分配。攻击者需要为每个 Bot B_i 分配一组目标诱饵目的节点 $\{D_{j_1}, D_{j_2}, \dots\}$ ，使得所有 Bot-Decoy 通信对在目标链路 $e^* \in E_{\text{target}}$ 上的汇聚流量总和超过其容量。形式化地，攻击者需要求解以下可行性问题：

$$\sum_{\substack{(i,j): \\ e^* \in \text{path}(B_i, D_j)}} r_{ij} \geq C_{e^*}, \quad (2-5)$$

其中 r_{ij} 为 B_i 向 D_j 发送的流量速率。为了保持隐蔽性，每条流的速率需满足 $r_{ij} \leq r_{\text{thresh}}$ （典型值为数十 KB/s 至数百 KB/s），使其不触发终端侧或中间节点的单流速率告警^[25]。

阶段三：协同流量投送。攻击者通过控制信道同步启动所有 Bot 的流量发送。在 LEO 场景中，由于星座拓扑的快照切换周期 $\Delta\tau$ 通常为 10~30 s，攻击者可将攻击持续时间与快照周期对齐，在每个快照时段内保持稳定的攻击流配置，随拓扑切换同步调整 Bot-Decoy 分配方案^[24, 25]。

进一步地，若攻击者掌握拓扑、路由策略及 Bot 地理位置等先验信息，其控制指令可按快照序列提前异步下发，无需在每个时隙实时重算并广播全量命令。该特性降低了攻击侧控制信令暴露，也使针对控制指令的实时拦截在 LEO 场景中更具挑战性^[24]。

2.2.3 LEO 场景下 LFA 的退化策略

在 LEO 场景中，攻击者可利用星座拓扑的公开性和规律性设计针对性的退化策略，以规避各类防御机制。本文重点关注以下三种退化策略，它们直接驱动了第三章多签名检测方案的特征维度选择。

退化策略一：低速微流化（Low-rate Dilution）。攻击者通过扩大僵尸主机规模 N ，将总攻击带宽分摊至更多的 Bot，使每条流的速率 r_{ij} 降至极低值。设目标链路容量为 C_{e^*} ，正常背景流量为 λ_{bg} ，则攻击成功所需的 Bot 总流量为 $\Lambda_{\text{atk}} = C_{e^*} - \lambda_{bg}$ 。若 N 足够大，则 $r_{ij} = \Lambda_{\text{atk}} / (N \cdot \bar{M}) \rightarrow 0$ （ $\bar{M}$ 为每个 Bot 的平均 Decoy 数），使得基于聚合速率的 Heavy-Hitter 检测失去作用^[25, 46]。

退化策略二：多靶机分散（Fan-in/Fan-out Relaxation）。攻击者增大每个 Bot 的目标诱饵目的节点（Decoy）的数量和地理分散度（即放大扇出度 Fan-out），同时增大指向同一 Decoy 的 Bot 数量（即放大扇入度 Fan-in），使攻击流量在源端和目的端均呈高度分散分布。这种分散性破坏了基于目的 IP 熵变化或目的前缀聚合的检测假设^[25]。

退化策略三：脉冲式滚动攻击（on-off rolling）。攻击者将攻击划分为交替的开启阶段（on）和静默阶段（off），在开启阶段集中发送攻击流量，在静默阶段暂停发送，使链路利用率回落至正常水平。通过控制 on-off 的周期与占空比，攻击者可以使基于时间窗口平均值的检测算法难以捕捉瞬时突增。此外，攻击者还可以在不同快照时段切换目标链路，形成时间和空间上的双重游击^[24, 28]。

需要注意的是，关于动态性是否必然放大攻击收益，现在学界还没有达到共识。ICARUS 的初步结果显示：利用路径切换形成脉冲叠加在原理上可行，但可利用窗口在多数场景下并不充足；相比之下，若攻击脉冲与链路负载自然波动（load

surge) 时刻重合, 则可能以更低代价触发拥塞。该问题仍依赖更细粒度的包级分析, 属于当前研究边界而非既定定论^[24, 27]。

上述三种退化策略的共同效果是: 使 LFA 攻击流量在单一特征维度上与正常业务流量难以区分。这一分析直接驱动了第三章中引入多维度特征签名(速率、扇入度/扇出度、持续性)的设计决策。任何单一维度均可被退化策略规避, 但多维度的联合异常仍然可以被检测到^[30]。

2.2.4 LFA 攻防中的观测不对称与设计启示

在前述攻击模型与退化策略基础上, 还需要看到, 低轨卫星场景下 LFA 的难点不只在攻击流量隐蔽, 还在于攻防双方掌握的信息和动作位置并不对称, 因而形成了观测不对称。这一不对称性主要体现在以下四个方面。

- (1) **拓扑知识不对称**。攻击者可以利用公开的轨道根数、已知的路由偏好和多轮探测结果, 在快照尺度上重建全局近似拓扑; 而单颗卫星在默认情况下只能持续观测经过本节点的局部切片, 缺乏同等粒度的全局视角^[24, 25]。
- (2) **统计粒度不对称**。攻击者在组织攻击时掌握 Bot、Decoy 与目标链路之间的全链路映射关系, 可以按路径粒度进行流量编排; 防御方若仅依赖本地聚合速率, 往往只能获得某条链路截面上的投影统计, 难以直接重构攻击编队全貌。
- (3) **执行位置不对称**。攻击者可从源侧发起流量, 在链路上游拥有动作发起点; 而防御方若只在被攻击链路附近检测与丢弃, 则往往只能在攻击流已经穿越多跳 ISL 之后被动响应, 导致大量无效占用已经产生。
- (4) **时间尺度不对称**。攻击者可以依据快照切换和负载波动调整注入节奏, 而防御方若依赖中心收集、离线分析或大窗口平均, 便会在时间上持续滞后于攻击侧的重构速度。

这种观测不对称决定了, 单一特征配合单点缓解在 LEO 环境中往往不够。单一特征只能在有限的局部切片上寻找一类异常, 而攻击者可以通过多 Decoy 分散、低速微流化和脉冲滚动等手段把该异常摊薄; 单点缓解也只能在本地收缩损害, 无法沿路径方向消除上游带宽浪费。因此, 本文在后续章节中分别从两个方向削弱这种不对称: 第三章通过多签名检测提升局部切片中的可分性, 第四章通过局部协同与逐跳推回扩大观测和执行范围。

2.3 在网计算与 P4 可编程数据面架构

2.3.1 数据面可编程能力的演进

软件定义网络通过解耦控制平面与数据平面，首次实现了对网络转发行为的集中式可编程控制。其代表性南向接口协议 OpenFlow 定义了一组固定的匹配字段集合（如 12-tuple 和后续扩展的 OXM 匹配字段），控制器通过下发流表规则指示数据面对匹配的数据包执行预定义的动作（转发、丢弃、修改字段等）。OpenFlow 的主要局限在于协议依赖性：可匹配的协议头字段由标准预先定义并硬编码于交换芯片中，无法支持自定义协议头的解析与处理。当需要处理新的协议（如 INT 遥测头）或自定义封装格式时，必须等待标准更新和芯片厂商的硬件支持，这严重制约了数据面的创新速度^[62]。

P4 语言^[63] 的出现缓解了这一问题。P4 的设计理念主要包含两项原则：协议无关性，即程序员可以自行定义数据包的解析逻辑和匹配字段，交换芯片不再绑定于特定协议；目标无关性，即同一份 P4 程序可以通过不同的编译器后端编译为不同硬件平台（如 ASIC 交换芯片、FPGA SmartNIC、软件交换机 BMv2）的可执行代码。P4 程序定义了数据包处理流程：从比特流中解析出报文头，经过多级匹配-动作处理，最终将修改后的报文头重新组装为比特流。

这种自顶向下的编程范式使得网络运营者和研究者能够根据需求灵活定义数据面的处理逻辑，无需受限于芯片硬编码的功能集。对于本文的研究而言，P4 的协议无关性意味着可以在 LEO 卫星的交换芯片上自定义用于多签名检测的协议头字段和状态更新逻辑，而不必依赖芯片厂商的专门支持^[35, 63]。

产业侧的变化进一步印证了这一点。2025 年 Intel 将 Tofino P4 软件开源，Cisco 明确表示 Silicon One 自架构之初即融入 P4 可编程性，Xsight Labs 则开放了 X-Series 交换芯片 ISA；与此同时，NVIDIA DOCA 给出了面向 BlueField/ConnectX 平台的 P4-based target architecture，AMD Pensando Pollara 400 也继续沿 fully programmable P4 engine 的路线演进^[53-57]。再结合 2025 年 P4 Workshop 对 AI 互连、P4 NIC、eBPF/DPDK 协同和异构硬件编译的讨论，可以看到可编程数据面已经不再局限于交换机局部能力，而是在向跨 NIC、交换机与加速器的系统接口发展^[58]。对本文而言，未来星载可编程平台更可能由 FPGA、DPU、NIC 与专用交换模块组合而成，因此按 P4 抽象层组织检测和缓解逻辑，比围绕单一芯片型号构造方案更稳妥。

2.3.2 PISA 架构与匹配-动作流水线

P4 程序运行于协议无关交换架构（Protocol Independent Switch Architecture, PISA）之上^[61]。PISA 的硬件流水线由以下四个功能模块依次级联构成：

（1）**可编程解析器（Programmable Parser）**。解析器是一个有限状态机（Finite State Machine, FSM），负责将输入的原始比特流按照 P4 程序定义的协议格式依次提取各层报文头字段，并将提取结果写入包元数据（Packet Metadata）和包头向量（Parsed Header Vector, PHV）中。解析器支持条件分支（如根据 EtherType 字段决定下一层协议的解析方式），但不支持循环——每个数据包的解析路径必须是有向无环的。解析器的状态数和转换规则受硬件资源限制，典型 PISA 芯片支持的最大解析深度约为 1000 Bytes。

（2）**入口匹配-动作流水线（Ingress Match-Action Pipeline）**。由 S_{ing} 个（典型值 12~20）MAU 级（Stage）级联组成^[35, 61]。在每个 MAU 级 s ($1 \leq s \leq S_{\text{ing}}$)，数据包的 PHV 中的指定字段被用作匹配键（Match Key），在该级的匹配表（Match Table，实现为 TCAM 或 SRAM 哈希表）中进行查找。匹配成功后，执行对应的动作（Action），包括修改 PHV 字段、读写有状态存储元素、设置出口端口等。每个 MAU 级的处理在单个时钟周期内完成，且各级之间严格串行，不存在回路或反馈——这是 PISA 实现线速处理的主要约束。

（3）**出口匹配-动作流水线（Egress Match-Action Pipeline）**。结构与入口流水线类似，由 S_{egr} 个 MAU 级组成，在数据包经过排队调度后执行。出口流水线通常用于数据包的最终修改（如更新校验和、添加遥测字段）。

（4）**逆解析器（Deparser）**。将经过流水线处理后的 PHV 重新序列化为比特流，写入数据包的报文头区域，完成数据包的重组后发送至物理端口。

PISA 架构的主要约束可总结为：无循环——数据包在流水线中严格单向前进，不可回退；固定时延——每个数据包的处理时间恒定，不依赖于包内容或流状态；有限级数——可用的 MAU 级数有限，复杂的处理逻辑需在有限级数内完成^[35, 61]。这些约束直接影响了第三章中多签名提取算法的流水线映射设计。

2.3.3 数据面有状态处理与存储约束

在 PISA 流水线中实现安全检测需要维护跨包的统计信息，这依赖于 P4 提供的有状态存储原语。P4 定义了三类基本有状态元素^[35, 63]：

Register: 可在 MAU 的 ALU 中被读取和更新的整数数组。Register 是 P4 中最灵活的有状态存储单元，支持读-修改-写（Read-Modify-Write, RMW）操作，如递增、赋值、条件更新等。但存在一个主要约束：同一个 Register 数组的同一个索

引在同一个流水线通过中只能被一个 MAU 级访问，以避免读写竞争。

Counter: 仅支持递增操作的计数器，适用于统计匹配特定规则的数据包数量或字节数。

Meter: 实现令牌桶算法的速率测量器，可对流量进行三色标记，用于速率限制和流量整形。

上述有状态元素均驻留在 PISA 芯片的片上 SRAM 中。如前文所述，宇航级芯片的片上 SRAM 容量严格受限，通常为数 MB 量级。若第一级采用一个 $d \times w$ 的 CMS 计数矩阵（每个 Counter 占 b_{ctr} 比特），并在第二级仅对 K 个显式候选条目维护精细多签名状态（每个候选条目占用 b 字节），则状态空间总消耗可近似写为：

$$M_{\text{state}} \approx \frac{dw b_{\text{ctr}}}{8} + K \times b \text{ Bytes.} \quad (2-6)$$

以第三章的代表性部署口径为例，若取 $d = 4$ ， $w = 2048$ ， $b_{\text{ctr}} = 16$ 比特，则 CMS 矩阵约占 16 KB；若 $K = 1024$ ， $b = 128$ ，则候选级多签名状态约占 128 KB；再加上队列映射、双缓冲控制位与辅助寄存器，总预算仍处于数百 KB 量级，与第三章约 200 KB 的统一部署预算保持一致。这正是两级级联架构的设计优势：通过第一级的 Sketch 频率估计与候选过滤，将需要进行精细多签名提取的流数量从全量（ $\sim 10^6$ ）压缩至 K （ $\sim 10^3$ ），从而将第二级的状态空间消耗控制在可接受范围内^[30]。

此外，PISA 的 ALU 所支持的运算类型严格受限于整数加减法、位移、按位逻辑运算和哈希计算。不支持浮点运算、乘法（部分高端芯片支持有限的硬件乘法器）、除法和索引间接寻址。这一计算约束要求所有在网算法的数学操作必须可被分解为上述基本操作的组合。例如，第三章中的可疑度评分函数需要避免使用除法和指数运算，转而采用基于位移的近似计算或预计算查找表^[61]。

2.3.4 数据面、控制面与星上 Agent 的职责划分

在介绍 PISA 流水线的的能力约束之后，还需进一步回答一个工程实现层面的划分问题：哪些任务应当留在数据面逐包执行，哪些任务应当交由星上 Agent 或地面控制系统完成？这一划分对于后续方案的可实现性具有决定性意义。常见误区是试图将所有检测、聚合与决策逻辑都压入 P4 程序中，但这既会迅速耗尽有限的流水线级数与寄存器资源，也会使程序难以适应快照切换、阈值重配置和协同消息处理等慢时间尺度任务。本文采用的是三层职责划分：快路径放在数据面，窗口级闭环留给 Agent，长期优化交由地面。

上述三层划分的本质，是将不同时间尺度、不同复杂度和不同可信假设下的任务放到最合适的执行位置。数据面负责逐包、常数复杂度且需要立即生效的动

表 2-1 LEO 在网防御体系中不同执行层的职责划分

执行层	时间尺度	典型任务	在本文中的对应功能
数据面 (P4/PISA)	ns- μ s / 每包	报文解析、哈希、寄存器更新、位图置位、队列标记、生存时间递减。	第三章中的 CMS 更新、候选缓存刷新、FI/FO 位图置位，以及第四章中的本地轻量消息处理。
星上 Agent	ms-s / 每 epoch	快照切换、阈值装载、位图读取、聚合评分、协同消息合并、推回软状态维护。	第三章中的分数计算和 rank 映射更新，第四章中的证据扩散消息/推回令牌处理、抑制判定与推回安装。
地面控制 / 离线管理	min-hour	轨道/路由规划、FIB 预计算、长期参数调优、软件升级、策略审计。	快照路由生成、长期统计分析、参数基线生成与离线评估。

作，例如计数更新、位图置位与队列标记；星上 Agent 负责允许按窗口或按快照处理、但仍要求驻留卫星本地闭环的任务，例如分数计算、消息聚合与推回规则下发；地面控制系统则只承担长期规划与离线优化，避免重新回到每次异常都必须走星地闭环的处理模式^[35, 61, 63]。

这一职责划分也解释了本文为何采用数据面与星上 Agent 协同的架构。若只依赖数据面，则难以承载对数计算、窗口级读出与协同消息聚合；若只依赖 Agent 或地面控制，又会牺牲包级响应的实时性。两者结合，才更符合 LEO 场景对轻量级在网计算闭环的要求。

2.3.5 面向在网安全的近似数据结构

上述存储与计算约束使得精确的 Per-flow 状态维护在星载环境中不可行，驱动了近似数据结构在在网安全检测中的广泛应用。本文涉及的主要近似数据结构包括以下四种。

(1) **布隆过滤器 (Bloom Filter, BF)** ^[64]。BF 由一个长度为 m 比特的位数组和 k 个独立的哈希函数 $h_1, h_2, \dots, h_k$ 组成。插入元素 x 时，将位数组中 $h_1(x), h_2(x), \dots, h_k(x)$ 对应的位置置为 1。查询元素 y 时，检查 $h_1(y), \dots, h_k(y)$ 对应的位是否全为 1。BF 不存在假阴性，但存在假阳性，假阳性率约为 $(1 - e^{-kn/m})^k$ ，其中 n 为已插入元素数量。本文第四章的协同消息去重与 Seen-set 维护可采用 BF 实现，从而在局部协同范围内以常数级状态开销实现查重。

(2) **Count-Min Sketch (CMS)** ^[65]。CMS 由 d 行 w 列的二维计数器矩阵组成，配合 d 个独立的哈希函数 $h_1, \dots, h_d$ 。更新元素 x 的计数时，对每行 i ，将

$CMS[i][h_i(x)]$ 递增 1。查询 x 的频次时, 返回 $\hat{f}(x) = \min_{1 \leq i \leq d} CMS[i][h_i(x)]$ 。CMS 的估计具有单边误差性质: $\hat{f}(x) \geq f(x)$ (始终高估), 且 $\Pr[\hat{f}(x) - f(x) > \epsilon N] < \delta$, 其中 N 为总元素数, $\epsilon = e/w, \delta = e^{-d}$ 。CMS 的优点在于: 每个包仅需执行固定次数的哈希与计数更新, 适合 P4 流水线的常数复杂度约束。其局限在于只保存频次矩阵而不显式保存 key 身份, 因此若要恢复 Top- K 候选, 还需要与显式候选缓存或堆结构配合。本文第三章将 CMS 与固定容量候选缓存结合, 用作一级候选生成器。

(3) **固定容量候选缓存 (Candidate Cache)**。候选缓存由容量为 K 的显式条目表组成, 每个条目保存候选 key 的标识、当前估计频次以及与后续二级特征提取相关的元数据 (如位图槽位索引、计数寄存器索引等)。与纯 CMS 不同, 候选缓存显式保留了 key 身份, 因此可为后续的 FI/FO 位图、Persistence 统计和队列映射提供可索引对象; 与需要维持全局严格有序列表的 Top- K 排序不同, 候选缓存只需在固定容量下执行命中刷新、未命中按阈值晋升或替换等局部更新, 更便于在 P4 数据面或 Agent 协同控制中实现。第三章正是通过 CMS 负责全局频次估计、候选缓存负责显式 key 恢复的分工, 近似恢复 Top- K 候选集合。

(4) **位图估计 (Bitmap / Linear Counting)** [66]。使用一个 m 比特的位数组, 通过哈希函数将元素映射到对应位并置位。集合基数的估计基于空位数量 V : $\hat{n} = -m \ln(V/m)$ 。位图估计在精度可控的前提下将基数统计问题转化为简单的位操作, 适合 P4 的 ALU 约束。在第三章中, 位图被用于扇入度 (Fan-in) 和扇出度 (Fan-out) 的在线近似统计。对候选缓存中的 key 维护独立的源 IP 位图和目的 IP 位图, 每个数据包到达时仅需执行一次哈希计算和一次位置位操作。

从工程映射角度看, 上述四类近似数据结构并非彼此替代, 而是共同构成了一条由粗筛、显式恢复、细粒度统计到消息去重组成的功能链。CMS 负责在全量流空间中给出低成本的频率估计, 候选缓存负责恢复 key 身份并承接后续精细特征, Bitmap 负责在候选范围内提取不同对端数量这类结构特征, 而 Bloom Filter 则适合承担第四章中 seen-set 这类存在性判断任务。它们适合在星载场景联合使用, 原因在于每个包经过时只需要执行常数次哈希、读写和位操作, 而无需任何复杂迭代或全表扫描。

2.4 分布式 Gossip 协议与协同防御基础

2.4.1 去中心化协同的必要性与传统共识的代价

第三章的单星检测方案解决了本地闭环响应的问题, 但 LFA 攻击的碎片化特性使得任何单颗卫星只能观测到途经本节点的局部流量切片。在退化策略下, 每

表 2-2 本文所用近似数据结构的功能定位与实现代价对比

结构	主要操作	误差特性	状态/更新代价	在本文中的定位
Bloom Filter	插入、存在性查询。	无假阴性，存在假阳性。	位数组 + 常数次哈希。	第四章协同消息去重、Seen-set 维护。
Count-Min Sketch	频次更新、点查询。	单边高估。	d 次计数器更新。	第三章全量流空间中的一级频率估计。
Candidate Cache	命中、刷新、替换。	近似恢复 Top- K 候选。	固定容量显式状态。	恢复 key 身份并承接二级特征统计。
Bitmap / Linear Counting	置位、空位计数、基数估计。	接近饱和时误差增大。	一次置位 + 窗口末估计。	第三章 FI/FO 的候选内不同对端数量统计。

颗卫星本地观测到的单流速率可能接近正常，但多颗卫星上的观测结果汇聚后才能揭示出异常的汇聚模式^[25, 30]。因此，多星之间的检测证据共享与协同判定是有效对抗 LFA 的必要条件。

直觉上，多节点协同可以通过集中式方案实现——即设置一个或多个中心协调节点，各卫星将本地检测结果上报至中心节点，由后者做出全局判定。但在 LEO 环境中，这一方案面临三个固有问题：（1）中心节点的指定与维护在高动态拓扑下本身就是一个非平凡问题（需要类似 Leader Election 的机制）；（2）中心节点到各卫星的通信路径可能跨越多个 ISL 跳数，引入额外时延；（3）中心节点本身可能是 LFA 攻击的目标，或其接入链路可能因遭受攻击而瘫痪^[25]。

另一种方案是采用分布式强一致性共识协议（如 Paxos、Raft），使所有参与节点就攻击判定达成一致。但此类协议的通信复杂度和时延代价在 LEO 环境中不可接受：以 Raft 为例，一次共识需要至少两轮消息交换（提案-确认），涉及过半节点的参与。在 n 个节点组成的集群中，单次共识的消息复杂度为 $O(n)$ ，如果区域内有数十颗卫星参与协同，每次判定需要交换数十至数百条控制消息。在 ISL 带宽已被 LFA 攻击流量大量侵占的情况下，这些控制消息本身可能因拥塞而丢失或严重延迟，导致协议无法收敛。

全网广播泛洪是最简单的信息共享方式，但其消息复杂度为 $O(|V| \cdot |E|)$ ，在大规模星座（ $|V| > 1000$ ）中将产生灾难性的信令风暴，会耗尽剩余可用带宽，并可能触发二次拥塞。

2.4.2 Gossip 协议的基本原理与传播特性

Gossip 协议（又称流行病协议）^[67] 提供了一种介于强一致性共识与盲目泛洪之间的折中方案。其基本机制为：每个节点周期性地（周期为 T_{gossip} ）随机选取一

个或少数几个邻居节点，将本地持有的信息（如攻击检测证据）发送给对方；接收方将收到的信息与本地信息进行合并，并在后续周期中继续向其邻居传播。

Gossip 协议的传播动力学可类比为传染病模型中的易感者—感染者—康复者（Susceptible-Infected-Recovered, SIR）模型。设网络中有 n 个节点，初始时有 i_0 个节点持有某条信息并处于已感染状态，每个已感染节点在每个 Gossip 周期中以概率 p 将信息传播给一个随机邻居。在完全图上，信息到达所有节点所需的轮次为 $O(\log n)$ ，消息总量为 $O(n \log n)$ 。在稀疏图上，传播速度取决于图的连通性和度数，但仍然保持对数级别的轮次复杂度^[67]。

Gossip 协议的主要优势在于：（1）去中心化——无需指定或维护中心协调节点；（2）容错性——即使部分节点或链路失效，信息仍可通过替代路径传播，最终到达所有可达节点，实现最终一致性；（3）可扩展性——每个节点在每个周期中仅与少量邻居通信，单节点的通信开销与网络规模无关。这些特性使 Gossip 协议适合在高动态、部分链路可能被攻击瘫痪的 LEO 环境中进行多星协同^[67]。

2.4.3 抑制型传播与开销控制

标准 Gossip 协议的一个固有问题是冗余消息的过度生成：已经获知某条信息的节点仍然可能反复收到相同信息的推送，造成带宽浪费。在 LFA 攻击已导致 ISL 带宽紧张的环境中，这种冗余尤其不可接受。

抑制型传播机制通过引入冗余检测与主动静默来控制 Gossip 的开销。这一思路源自物联网领域的 Trickle 算法^[68]，可形式化为以下流程。

对于每条待传播的消息 m ，节点维护三个状态变量：传播计时器 T_m （在区间 $[T_{\min}, T_{\max}]$ 内动态调整）、冗余计数器 c_m （记录在当前周期内收到的与 m 内容一致的消息数量）、以及冗余抑制阈值 K 。在每个传播周期开始时，节点将 c_m 重置为 0 并启动计时器 T_m 。在计时器运行期间，每当节点从邻居收到一条与 m 内容一致的消息时，将 c_m 递增 1。当计时器到期时，若 $c_m < K$ ，节点判断其邻域内尚未充分获知该信息，遂主动向邻居发送 m ；若 $c_m \geq K$ ，节点判断其邻域已充分获知该信息，遂抑制本次发送。若信息在一段时间内未再听到新的更新， T_m 倍增至 $T_{\max}$ ，节点进入静默状态^[68]。

此外，通过为每条消息设置生存时间（Time-To-Live, TTL）跳数限制，可将其传播范围限制在发起节点的 h -hop 感兴趣区域（Region of Interest, ROI）内，避免信息向无关区域扩散。在第四章的 CoopSatShield 设计中，ROI 的跳数半径 h 被设定为与 LFA 攻击路径的典型跳数相匹配，确保协同判定的参与节点恰好覆盖攻击路径经过的卫星群组^[30]。

表 2-3 LEO 多星协同候选机制的对比

机制	组织方式	典型时延	信令开销	对 LEO 的适配性
中心化汇聚	单中心或少量 中心节点收集 证据	中到高	中等	易受长路径时延、 单点故障与中心链 路拥塞影响。
强一致性共 识	多节点多数派 确认后再决策	高	高	一致性强，但在高 动态和拥塞环境下 代价偏大。
全网广播泛 洪	所有节点无差 别扩散消息	低到中	极高	实现简单，但极易 引发广播风暴。
抑制型 Gossip	局部随机扩散 + 冗余抑制 + TTL 裁剪	中低	低到中	去中心化、容错性 好，适合 LEO 局部 协同。

2.4.4 Gossip 参数选择与区域协同范围设计

在将 Gossip 协议用于 LEO 多星协同时，系统表现更多取决于参数配置。与地面大规模互联网环境不同，LEO 场景中的协同传播需要同时满足三个目标：攻击持续期间尽快把局部证据扩散到足够多的相关节点，将控制消息限制在真正与攻击路径有关的局部区域内，并在带宽已经紧张的 ISL 上避免冗余传播制造新的拥塞。因此，TTL 半径 h 、最小/最大计时器 $T_{\min}$, $T_{\max}$ 以及冗余阈值 K 都直接决定着协同系统在传播速度、覆盖精度和控制开销之间的平衡^[67, 68]。

一般而言， h 主要决定协同的空间范围： h 过小，证据传播无法覆盖攻击路径上的相关上游节点； h 过大，则会让无关卫星被卷入协同，平白增加控制开销。 $T_{\min}$ 决定首轮扩散速度，应足够小以追上攻击状态变化，但又不能小到使多个节点同时争抢发送、造成过多碰撞； $T_{\max}$ 决定平稳期的静默程度，通常用于在没有新证据时逐步降低控制频率。冗余阈值 K 反映的是本地是否已经获得足够重复证据：较小的 K 会更积极地抑制重复消息，较大的 K 则更偏向于保证覆盖度。若以 h 跳局部区域为协同边界，且每轮扩散受 $T_{\min}$ 限制，则消息覆盖到 h 跳邻域的时间通常处于 $O(hT_{\min})$ 的量级；这一定性关系也解释了为什么第四章中需要在覆盖范围与传播时延之间寻找平衡。

对本文方案而言，一个重要原则是，Gossip 的作用不在于做全网通告，而在于围绕被攻击区域聚合和扩散证据。它不需要让所有卫星都知道某条证据，只需要让那些最可能共享攻击路径、最有机会执行推回动作的卫星尽快获知该证据。基于这一点，第四章采用了局部区域限制、冗余抑制和软状态寿命控制的组合设计，从而将协同开销压到远低于业务带宽的水平，同时保证证据能够沿最有价值的局

部方向扩散。

2.5 本章小结

本章从四个方面介绍了后续研究所需的理论基础。在网络架构层面，建立了 LEO 巨型星座的时变图模型 $G(t) = (V(t), E(t))$ 和快照路由模型，明确了 ISL 容量约束 C_e 与拓扑切换周期 $\Delta\tau$ 的量化参数。在威胁模型层面，形式化描述了 Crossfire 型 LFA 的三阶段攻击流程和三种退化策略，说明了多维特征检测的必要性。在计算平台层面，详细分析了 PISA 流水线的无循环、固定级数、有限 ALU 等约束，以及片上 SRAM 的容量上限，给出了在网算法设计必须遵守的资源约束。在协同机制层面，引入了 Gossip 协议的传播理论与抑制型变种的开销控制模型，说明了其在 LEO 高动态、部分链路受损环境下的适用性。这些基础将在第三章（多签名在网检测）和第四章（多星协同推回）中被直接引用并进一步展开。

第三章 基于多签名在网检测与队列调度的 LFA 防御优化

3.1 引言

大规模低轨卫星网络通过星间链路与星地链路为全球提供低时延、广覆盖的互联网接入。然而，与地面骨干网类似，低轨卫星网络（Low Earth Orbit Satellite Network, LSN）作为重要通信基础设施同样面临拒绝服务类攻击风险，其中以链路洪泛攻击较为突出。LFA 的典型策略是利用分布式受控终端（Bot）发起大量看似正常的低速流量，将其沿着特定路径汇聚到目标链路，从而在链路上形成拥塞，甚至阻断重要区域间通信。ICARUS 系统化刻画了面向 LSN 的 LFA 攻击面与攻击构造过程，指出攻击者可以借助公开轨道信息推测路由，并在拓扑动态变化下持续调整攻击流集合以维持链路拥塞，从而对 LSN 的可用性形成实质威胁^[24]。

从防御角度看，LSN 的网络形态使传统的控制面集中检测、再下发策略的闭环方案难以满足实时性。一方面，卫星网络控制通道的带宽和时延都较为受限，跨星或星地的控制闭环通常达到毫秒到十毫秒量级；另一方面，攻击流集合可能在亚秒级甚至更短时间尺度内变化，控制面统计汇聚与策略更新一旦滞后，防御效果就会下降。SatShield 针对这一问题提出将检测与缓解尽量下沉到数据平面：以地址聚合速率作为区分特征，在数据平面持续维护近似 Top-K 重流候选，并通过队列调度实现被动丢弃，从而达到近实时的在网缓解^[30]。

这一问题在 2025 年以来更加突出。随着 Starlink Direct to Cell 开始直接承载手机终端的接入业务，Amazon Leo 推出面向企业与政府场景的私网互连和千兆级终端，以及 IRIS² 等体系把基础设施连接纳入星座目标，LEO 网络上合法的大流、回传流和云互连流都在增多^[10, 15, 32]。继续把重流直接视为可疑对象，误判风险会更高，因此检测逻辑需要把速率与扇入度、扇出度和跨窗口持续性等结构特征综合考虑。本章提出 MS-SatShield，用于应对这种业务多样化背景下的可分性下降问题。

尽管 SatShield 已证明聚合速率在现有卫星系统约束下具有较好的可分性，但只依赖速率特征仍会遇到两类难以回避的问题。真实网络中的正常业务本身就存在长尾与大流现象，例如热门服务和突发上行，仅用速率很难区分合法大流与攻击聚合。与此同时，攻击者还可以通过扩大 Bot 规模、降低单 Bot 速率，或将攻击流量分摊到更多诱饵目的节点（Decoy），削弱单一聚合维度上的速率差异，使一级候选生成质量下降，进而诱发漏检或误检。SatShield 在讨论部分也指出，随着 Bot 规模和业务形态继续演进，引入 Fan-in/Fan-out 等结构性签名是提高适应性的

一个方向^[30]。地面网络中, Ripple^[46] 和 Mew^[47] 已展示了基于可编程数据面的 LFA 防御潜力, 但它们依赖于地面交换芯片更充裕的存储与计算资源, 无法直接移植到 SWaP 受限的星载环境。

基于上述背景, 本章在 SatShield 的一级候选生成与队列调度框架的基础上, 提出一种基于多签名在网检测与队列调度的 LFA 防御优化方法 (Multi-Signature Satellite Shield, MS-SatShield)。该方法以计数最小草图 (Count-Min Sketch, CMS) 频率估计与固定容量候选缓存构成一级候选生成器, 避免对全体活跃 key 的不同对端数量做精确统计带来的状态爆炸; 同时在候选集合上引入扇入度 (Fan-in)、扇出度 (Fan-out) 与持续性 (Persistence) 等结构性特征, 以提升对多诱饵目的节点分摊、脉冲式 on-off 和速率下沉等退化策略的检测适应性, 并将多签名可疑度映射为队列优先级, 实现及时、低误伤的防御优化。本章有意将研究范围限定在单星、单节点视角下的本地闭环: 检测、评分和队列调度都发生在观测到拥塞或可疑聚合 key 的卫星节点内部, 其目标是尽快识别攻击相关 key, 并在本地队列层面降低对良性业务的冲击。

3.2 问题定义与方法设计

3.2.1 威胁模型与问题表述

将 LSN 在时刻 t 表示为动态图 $G(t) = (V, E(t))$, 其中 V 为卫星节点集合, $E(t)$ 为随时间变化的星间链路集合。每条链路 $e \in E(t)$ 具有容量 C_e 。在 LFA 场景中, 攻击者控制一组分布式终端 (Bot) 集合 $\mathcal{B}$, 并选择一组诱饵目的节点 (Decoy) 集合 $\mathcal{D}$, 使得从 $\mathcal{B}$ 到 $\mathcal{D}$ 的攻击流量在转发路径上穿越目标链路 e^* , 从而使 e^* 的负载超过其容量并发生拥塞。与地面网络中常见的五元组拆分类似^[25], 攻击者可通过多对多配对 (一个 Bot 对多个目的、一个目的由多个 Bot 访问) 将单条五元组流的速率降低到接近正常水平, 但这会改变流量的结构性特征, 例如源的 Fan-out、目的的 Fan-in 以及可疑 key 的持续出现模式^[30]。

这里需要明确区分业务产生位置和防御执行位置。卫星通信网络本身并不产生用户业务流, Bot 与 Decoy 均位于地面用户、信关站后方网络、云服务或其他地面终端侧; 卫星节点只作为接入、星间中继和转发路由器承载这些业务流。本文所谓“星上检测攻击流”, 指的是卫星转发节点在数据路径上观测并处置从地面入口进入、经星间链路中继的业务流, 而不是假设卫星自身主动产生攻击流量。因此, 本文的 DDoS 场景不同于传统地面网络中直接打垮某个服务器或接入链路的终端型 DDoS: 攻击目标主要是 LEO 拓扑中的瓶颈 ISL 或星地转发链路, 诱饵目的节点可以仍然表现为合法接收端, 真正异常发生在中间转发路径的链路汇聚处。

这也是本文必须把检测逻辑放到星载转发数据面附近的原因。

为与 ICARUS 的攻击评估语义保持一致，本章将攻击有效性分为两类目标：一类关注目标链路是否达到拥塞阈值，另一类关注达成这一目标需要付出多大的可见代价。前者由目标链路是否超容表征，后者分别由攻击成本（达到拥塞所需注入总带宽）与可检测性代理指标（上行侧最大增量）刻画。本文聚焦检测与缓解机制本身，不直接求解最优攻击流分配，但在指标解释上沿用这一语义，以便与 ICARUS/SatShield 的结果保持可对齐性^[24]。

为避免符号歧义，本文先区分方法的一般形式与后续实验设置。一般而言，key 是交换机用于统计和处置的流量标识，可以由五元组、源地址、目的地址、源宿地址对或地址前缀对生成。本文后续实验默认采用目的地址作为 key，即对数据包 p 有 $a = \text{dst}(p)$ 。因此，后文的 CMS 统计、候选缓存、多签名评分和队列调度，默认都围绕目的地址 a 进行。

为了与可编程交换机的周期性统计与配置机制对齐，本章采用固定时间窗口 epoch 建模。记第 t 个 epoch 的时间长度为 Δ ，在该窗口内对 key 进行聚合统计。对任意 key a ，定义其在 epoch t 内的聚合字节数与聚合速率分别为：

$$\text{bytes}_t(a) = \sum_{p \in \mathcal{P}_t(a)} \text{len}(p), \quad (3-1a)$$

$$R_t(a) = \frac{\text{bytes}_t(a)}{\Delta}, \quad (3-1b)$$

其中 $\mathcal{P}_t(a)$ 表示 epoch t 内地址字段取值为 a 的数据包集合， $\text{len}(p)$ 为包长。

为刻画结构性特征，记每个包还包含一个与 key 对应的对端字段 x ：当以源地址作为 key 时， x 为目的地址；当以目的地址作为 key 时， x 为源地址。则可定义 key 的不同对端数量：

$$F_t(a) = |\{x \mid (a, x) \text{ 在 epoch } t \text{ 内出现}\}|, \quad (3-2)$$

其中 $F_t(a)$ 在源 key 模式下对应 Fan-out (FO)，在目的 key 模式下对应 Fan-in (FI)。在本文默认设置下， $a = \text{dst}(p)$ 、 $x = \text{src}(p)$ ，因此 $F_t(a)$ 表示在一个 epoch 内有多少不同源地址访问同一个目的地址，也就是目的地址的 Fan-in。

本文选择目的地址作为默认 key，是因为 LFA 的隐蔽性主要来自五元组层面的低速化和分散化。单条 Bot-Decoy 连接可能看起来接近正常连接，但大量 Bot 指向同一 Decoy 时，仍可能在目的地址层面留下聚合速率和 Fan-in 的组合特征。

此外，考虑 LSN 拓扑与流量均具有动态性，单一窗口内的一级候选结果可能出现波动。为减小这种波动，引入持续性特征，表示某 key 在最近 k_p 个 epoch 中

进入候选集合的次数:

$$P_t(a) = \sum_{i=1}^{k_p} \mathbf{1}[a \in \mathcal{H}_{t-i}], \quad (3-3)$$

其中 $\mathcal{H}_t$ 表示基于 epoch t 窗口统计结果导出的一级候选 key 集合, $\mathbf{1}[\cdot]$ 为指示函数。

需要指出的是, 速率退化与结构性特征放大之间并非彼此独立。考虑 $|\mathcal{B}|$ 个 Bot 向 M 个诱饵目的节点发起攻击, 记每个 Bot 平均访问 d 个 Decoy ($1 \leq d \leq M$), 并假设 Bot 的发送速率在其访问的 Decoy 上近似均分。令总攻击注入量为 $A = \sum_{b \in \mathcal{B}} r_b$, 则单个 Decoy 的聚合速率仍约为 $R_{\text{atk}} \approx A/M$, 而结构特征满足:

$$\text{FO}_{\text{Bot}} = d, \quad (3-4a)$$

$$\mathbb{E}[\text{FI}_{\text{Decoy}}] = \frac{|\mathcal{B}| d}{M}. \quad (3-4b)$$

特别地, 当每个 Bot 仅访问一个 Decoy ($d = 1$) 时, 有 $\text{FO}_{\text{Bot}} = 1$, 且 $\mathbb{E}[\text{FI}_{\text{Decoy}}] = |\mathcal{B}|/M$ 。因此, 在固定总攻击预算 A 的条件下, 攻击者虽然可以通过增大 M 压低按目的地址聚合后的速率, 但往往会在 Fan-in 或 Fan-out 的至少一维上保留结构性痕迹。更准确地说, 攻击者并非在任何条件下都无法同时压低速率与结构特征; 但在本文考虑的攻击预算、Bot 规模与终端速率约束下, 压低 per-key 聚合速率通常会推动 FI/FO 中至少一维偏离良性区间, 因此多签名检测相较于单一速率检测具有更强的适应性。

本章关注的问题是: 在星载可编程交换机片上存储和流水线算力都受限的条件下, 如何在每包线速处理的前提下, 利用 sketch 频率估计及显式候选缓存来在线恢复当前窗口内的重 key 候选集合, 并在该候选集合上提取 $R_t(a)$ 、 $F_t(a)$ 、 $P_t(a)$ 等多维特征, 进一步为可疑 key 分配不同的队列优先级, 实现对 LFA 的及时缓解, 同时尽量降低对良性业务的误伤。

3.2.2 速率可分性退化与多签名动机

在给出正式系统设计之前, 本文先用一个预实验说明单一速率特征在退化攻击下如何失效, 并据此引出后续的多签名设计。SatShield 的一个重要观察是, 在现有 LEO 系统约束下, 攻击者若要使目标 ISL 达到拥塞, 必须注入接近链路容量量级的总攻击流量; 但受单终端上行能力、单星可服务用户数等约束, 攻击流按源地址或目的地址聚合后, 仍会呈现 Mb/s 量级的重尾分布, 因此与绝大多数良性聚合速率之间存在可分性^[30]。基于这一观察, 本章采用与 SatShield 一致的 LEO 仿真与流量构造思路: 使用 Starlink Shell 1 星座拓扑 (1584 颗卫星) 和 ISL 容量 20 Gb/s

作为基本网络模型，良性背景流量由 CAIDA 匿名互联网流量轨迹数据（CAIDA Anonymized Internet Traces 2018 数据集，采集自 Equinix-Chicago 监测点）经重放得到，并通过人口分布模型映射到城市对之间的通信需求（映射方法详见文献 [30] 第 IV-A 节）。该流量轨迹数据采集于 2018 年，与当前互联网流量分布可能存在差异，但本文关注的是攻击侧与正常侧的相对可分性，良性分布的绝对形状对结论影响有限。

图 3-1 展示了不同 Decoy 数 M 下，良性与攻击流量在每个地址 key 聚合速率维度上的经验累积分布函数（Cumulative Distribution Function, CDF）。绘制过程如下：先在仿真中固定 epoch 长度 Δ ，对每个 epoch 统计所有 key 的 $\text{bytes}_t(a)$ ，再由式 (3-1b) 计算 $R_t(a)$ ；再分别对良性与攻击 key 集合绘制 CDF 曲线。当 $M = 1$ 时，良性 $p_{99} \approx 485 \text{ Mb/s}$ 与攻击聚合速率 $10,024 \text{ Mb/s}$ 之间存在约 21 倍间隙； $M = 10$ 时这一间隙缩小到约 2 倍； $M = 100$ 时攻击 key 速率下降到 100 Mb/s ，已经落入良性分布的长尾区间； $M = 1000$ 时攻击 key 与良性 key 基本重叠。该图展示了可分性随 M 增大而退化的变化过程，也为后续多签名检测（图 3-3）提供了动机。

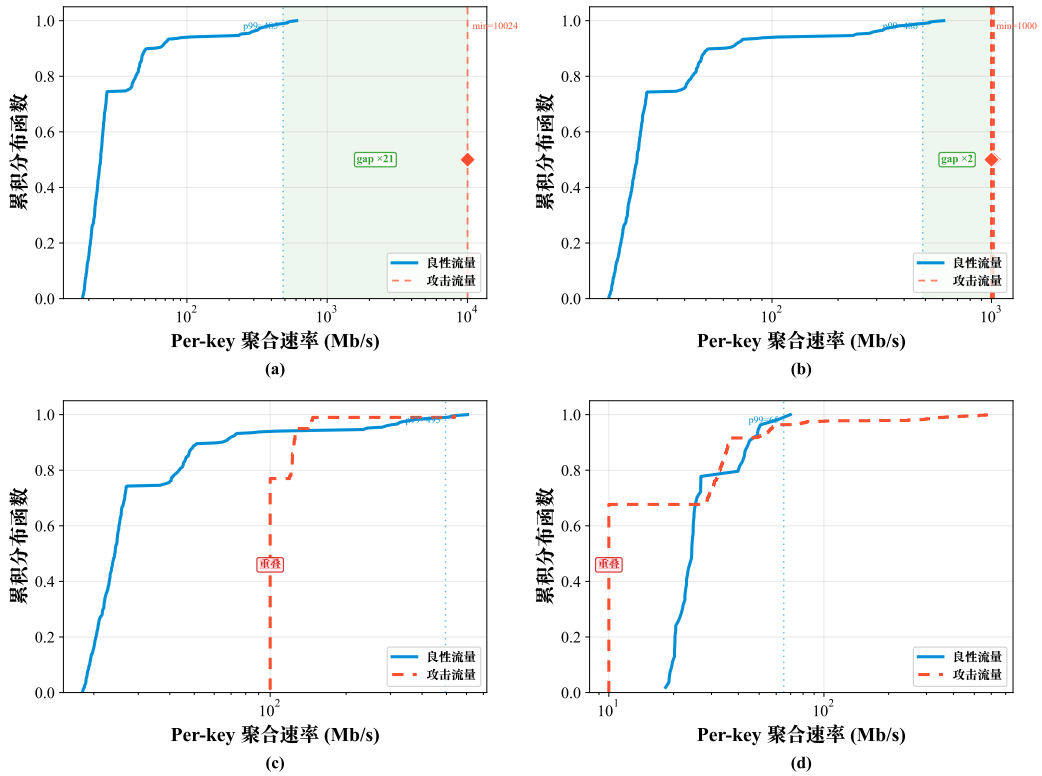


图 3-1 不同 Decoy 数下的聚合速率 CDF。(a) $M = 1$ ；(b) $M = 10$ ；(c) $M = 100$ ；(d) $M = 1000$

图中蓝色实线表示良性 key 的经验 CDF，红色虚线或竖线表示攻击 key 的经验 CDF。绿色阴影对应仍存在可分性间隙的区间；出现 overlap 标注时，则表示攻

击分布已经进入良性长尾并发生交叉。

需要说明的是，图 3-1 中的可分性并非在任何条件下都成立。当攻击者采用更强的隐蔽策略时，速率特征可能退化并与良性长尾区间发生重叠。为明确方法的适用范围，并给出后续实验的退化路径，本章构造了三类典型设定：增大 Bot 规模并降低单 Bot 速率、将攻击流量分摊到更多诱饵目的节点，以及采用脉冲式 on-off 策略使攻击 key 在窗口切换处附近间歇出现。它们都满足 LFA 的基本约束，即总攻击注入量需要与目标链路剩余容量处于同一量级^[25]：

$$\sum_{b \in \mathcal{B}} r_b \approx C_{e^*} - U_{e^*}^{\text{benign}}, \quad (3-5)$$

其中 r_b 为 Bot b 的攻击发送速率， $U_{e^*}^{\text{benign}}$ 为目标链路上良性占用带宽。

式 (3-5) 说明：如果希望将每个 key 的聚合速率压低到良性长尾附近，通常需要相应扩大可用 Bot 数量或增大 Decoy 分摊规模。如式 (3-4a) 和式 (3-4b) 所示，这一过程中 Fan-in/Fan-out 至少一维往往会被放大，从而为多签名检测保留可利用的结构性痕迹。

3.2.3 系统架构与数据流解释

图 3-2 展示了 MS-SatShield 的总体架构。系统由数据平面与星上轻量控制平面（星上 Agent）协同完成闭环：数据平面负责以线速维护基于 CMS 与候选缓存的一级候选生成摘要，并对活动候选执行轻量特征更新；星上 Agent 以 epoch 为粒度拉取候选统计并计算多签名可疑度，将结果映射为队列优先级配置，再写回交换机用于下一轮调度。

为了便于从处理流程理解图 3-2，下面分别说明包级数据流与 epoch 级控制流。为避免时序歧义，本文保留 $\mathcal{H}_t$ 与 $\text{rank}_t(\cdot)$ 表示基于 epoch t 统计结果导出的观测候选集与观测优先级，并约定：

$$\mathcal{H}_t^{\text{act}} = \mathcal{H}_{t-1}, \quad (3-5a)$$

$$\text{rank}_t^{\text{act}} = \text{rank}_{t-1}. \quad (3-5b)$$

因此，在 epoch t 的包处理中，交换机实际使用的是上一窗口导出的活动配置 $(\mathcal{H}_t^{\text{act}}, \text{rank}_t^{\text{act}})$ ；而 epoch t 结束后，代理根据当前窗口统计结果计算新的 $(\mathcal{H}_t, \text{rank}_t)$ ，并将其写回交换机，于 epoch $t+1$ 生效。

包级处理在交换机流水线中顺序发生。每个到达数据包 p 在解析后得到地址 key a 、对端 x 与包长 len 。系统先更新 CMS 计数矩阵，并依据当前估计频次刷新固定容量候选缓存；这一步对应一级候选生成，其目标是在固定内存内近似恢复当前窗口的重 key 集合及其显式 key 身份。对于驻留在候选缓存中的 key，数据平

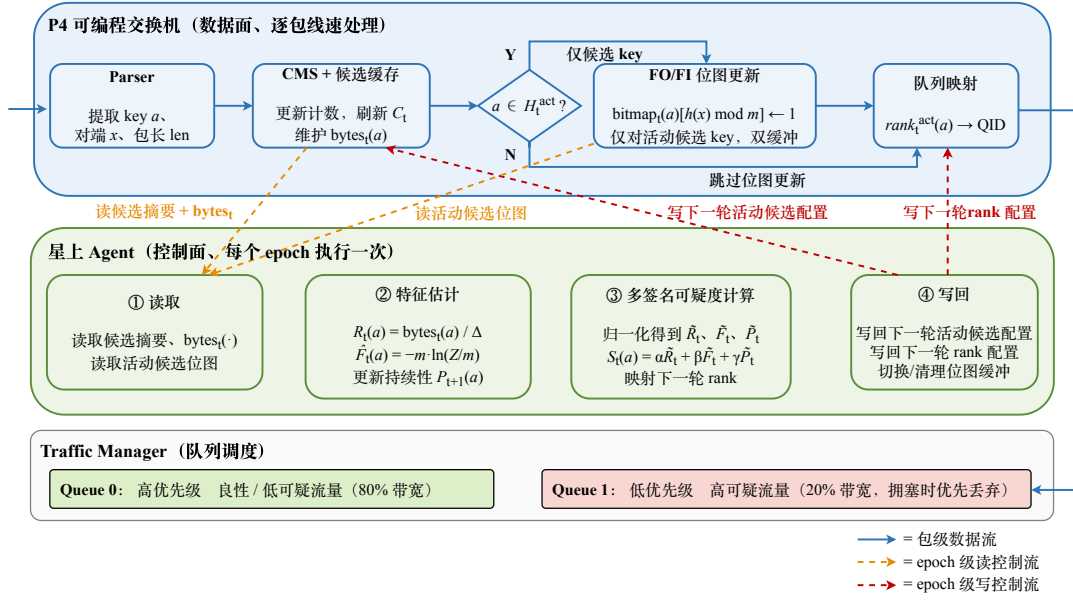


图 3-2 MS-SatShield 总体架构

面同时维护候选级聚合字节计数器 $\text{bytes}_t(a)$ 。随后系统判断该 key 是否命中活动候选集：若 $a \in \mathcal{H}_t^{\text{act}}$ ，则进入 FI/FO 近似统计更新模块，将 x 映射到位图索引并置位；若 $a \notin \mathcal{H}_t^{\text{act}}$ ，则跳过 FI/FO 更新以节省状态访问。最后，系统依据当前已安装的队列优先级映射表 $\text{rank}_t^{\text{act}}(\cdot)$ 选择队列并入队。这样安排后，CMS 更新与候选缓存刷新仍是所有数据包都必须执行的基础操作，而 FI/FO 更新只发生在活动候选集上，因此去重计数的状态成本可以限制在 $O(K \cdot m)$ 。

epoch 级控制流由星上 Agent 触发。Agent 按固定周期读取当前窗口的 CMS/候选缓存摘要与候选级字节计数，形成观测候选集合 $\mathcal{H}_t$ ；再读取活动候选 $\mathcal{H}_t^{\text{act}}$ 对应的位图，用线性计数估计 $\hat{F}_t(a)$ ，并更新持续性 $P_{t+1}(a)$ ；随后将 $R_t(a)$ 、 $\hat{F}_t(a)$ 、 $P_t(a)$ 归一化并融合为可疑度 $S_t(a)$ ，再映射为下一轮活动优先级 $\text{rank}_{t+1}^{\text{act}}(a)$ 并写回交换机。候选筛选、排序、归一化、对数或倒数等复杂运算都放在 Agent 侧完成，数据平面只承担寄存器读写与简单计算，符合 PISA 流水线的可实现约束^[61]。

3.2.4 基于 CMS 与候选缓存的一级候选生成

需要说明的是，本文使用 CMS 并不意味着将 LFA 简化为传统重流检测问题。LFA 的低速性主要体现在单条五元组连接层面，而本文的统计对象是前文定义的聚合 key。在默认目的地址 key 设置下，多个低速 Bot-Decoy 微流可能在同一目的地址上形成可观的聚合速率或结构性 Fan-in 特征。CMS 与候选缓存的作用，是在有限数据面状态下恢复这些值得进一步分析的候选 key，而不是直接给出攻击判定。

与精确逐流状态不同，CMS 用固定大小的计数矩阵维护全局频率摘要，每包只需执行固定次数的哈希与计数更新，适合映射到 P4/PISA 流水线；但 CMS 本身不显式保存 key 身份，因此不能直接输出当前 Top- K 的 key 列表。为同时兼顾全局频率估计和显式 key 恢复，MS-SatShield 在一级候选生成中采用 CMS 与固定容量候选缓存的组合：CMS 负责提供近似频率估计，候选缓存负责维护显式 key 集合及其候选级元数据。

具体地，设 CMS 由 d 行、每行宽度为 w 的计数矩阵 $\mathbf{S}_t \in \mathbb{N}^{d \times w}$ 构成，第 i 行对应哈希函数 $h_i(\cdot)$ 。当包 p （长度为 len ）携带 key a 到达时，数据平面执行：

$$\mathbf{S}_t[i, h_i(a)] \leftarrow \mathbf{S}_t[i, h_i(a)] + len, \quad i = 1, \dots, d. \quad (3-6)$$

随后，以各行对应计数器的最小值作为 key a 在当前窗口的近似聚合字节估计：

$$\widehat{\text{bytes}}_t^{\text{cms}}(a) = \min_{1 \leq i \leq d} \mathbf{S}_t[i, h_i(a)], \quad (3-7a)$$

$$\widehat{R}_t^{\text{cms}}(a) = \frac{\widehat{\text{bytes}}_t^{\text{cms}}(a)}{\Delta}. \quad (3-7b)$$

式 (3-7a) 和式 (3-7b) 给出的是全局频率摘要，而非显式候选集合。为将其转化为后续多签名判决可直接使用的 key 列表，本文引入固定容量候选缓存：

$$\mathcal{C}_t = \{(a, \text{bytes}_t(a), \text{meta}_t(a))\}, \quad (3-8a)$$

$$|\mathcal{C}_t| \leq K, \quad (3-8b)$$

其中每个缓存项保存显式 key 身份、候选级聚合字节计数 $\text{bytes}_t(a)$ 以及状态元数据 $\text{meta}_t(a)$ （如有效位、时间戳或缓存索引等）。实现上， $\text{bytes}_t(a)$ 由候选缓存项中的字节寄存器承载，并作为后续式 (3-1b) 中速率特征 $R_t(a)$ 的在网观测值。

候选缓存的更新规则如下：若到达 key a 已存在于 $\mathcal{C}_t$ ，则直接刷新该项并累加 $\text{bytes}_t(a)$ ；若 $a \notin \mathcal{C}_t$ 且缓存未满，则插入新项；若缓存已满，则以 CMS 估计值作为替换依据。将当前缓存中的最小 CMS 估计值记为：

$$\theta_t = \min_{u \in \mathcal{C}_t} \widehat{\text{bytes}}_t^{\text{cms}}(u), \quad (3-9)$$

当 $\widehat{\text{bytes}}_t^{\text{cms}}(a) > \theta_t$ 时，以 a 替换当前缓存中估计最小的候选项。这里追求的是在固定容量 K 下尽量恢复当前窗口中值得进入二级判别的重 key 候选，而非在数据平面精确维护全局严格排序。因此，本文后续所称的 Top- K 候选，均指 CMS 与候选缓存联合恢复出的近似重流候选集合，其排序和覆盖都带有概率性质。

从资源角度看，一级候选生成器的状态预算可分解为

$$M_{\text{CMS}} = \frac{dw b_{\text{ctr}}}{8}, \quad (3-10a)$$

$$M_{\text{cache}} = \frac{K (b_{\text{key}} + b_{\text{bytes}} + b_{\text{meta}})}{8}, \quad (3-10b)$$

其中 b_{ctr} 为 CMS 计数器位宽， b_{key} 、 b_{bytes} 与 b_{meta} 分别表示候选缓存中 key 标识、字节计数与元数据的位宽。在本文默认配置下，一级候选生成的开销通常为几十 KB，低于后续 FI/FO 位图区域的状态占用，因此整体资源占用仍主要由位图部分决定。

关于 key 状态的保存方式，本文并不在数据面维护全量显式 key 表。CMS 只是一个由 $d \times w$ 个计数器组成的近似统计结构。每个数据包到达时，数据面根据 key a 的多个哈希位置更新对应计数器，因此 CMS 可以估计某个 key 的聚合字节数，但并不保存该 key 的显式身份，也不能枚举当前所有活跃 key。

显式 key 只保存在容量为 K 的候选缓存中。可以把候选缓存理解为 K 个固定编号的表项：当某个 key 进入候选缓存后，它会占用其中一个表项，该表项保存 key 标识、候选级字节计数 $\text{bytes}_t(a)$ 和元数据。这里的候选级字节计数不是 CMS 哈希桶计数，而是绑定到该候选 key 的字节寄存器，用于计算该候选在当前 epoch 内的观测速率。FI/FO 位图、rank 映射和队列调度规则也只围绕这些候选表项维护。当候选 key 被替换时，对应表项中的字节计数、位图和调度状态随之重置。

持续性特征 $P_t(a)$ 也不要求数据面保存所有历史 key。数据面只在当前 epoch 产生候选集合，星上 Agent 在 epoch 边界读取候选集合，并在控制侧维护最近 k_p 个 epoch 的候选历史，用于更新 $P_t(a)$ 和下一轮 rank 配置。这样可以避免在逐包快路径中为所有曾出现过的 key 保存历史记录。

这一设计把候选恢复与最终判定拆开处理。CMS 在全量 key 空间中提供频率线索，候选缓存只显式保留少量需要继续观察的 key，从而避免维护全量 key 表带来的状态和查找开销。一级候选生成并不直接判定某个 key 是否为攻击对象，而是在固定状态预算下尽量恢复需要进入第二级分析的 key 身份。最终是否降权调度，仍由第二级的 Rate、FI/FO 和 Persistence 联合评分决定。

从工程实现看，这一设计还给第二级特征统计计划定了访问范围。由于 FI/FO 位图只对活动候选集维护，第二级的 Bitmap、Persistence 与 queue mapping 状态都被限制在 K 个对象之内。 K 因而不只是检测参数，也决定了整个数据面的内存上界与读写次数上界：一旦 K 固定，候选集内不同对端数量统计、rank 表和辅助寄存器的规模也随之固定。这也是后续消融实验要专门讨论 K 的原因。

3.2.5 多签名可疑度建模与判决

MS-SatShield 将一级候选生成与可疑判决分开处理：CMS 与候选缓存负责从海量 key 中筛选可疑候选集合 $\mathcal{H}_t$ ，多签名模型再在候选集合上区分攻击相关 key 与合法大流 key。为此，定义候选 key a 的综合可疑度为：

$$S_t(a) = \alpha \cdot \tilde{R}_t(a) + \beta \cdot \tilde{F}_t(a) + \gamma \cdot \tilde{P}_t(a), \quad (3-11)$$

其中 α 、 β 、 γ 为权重， $\tilde{(\cdot)}$ 为归一化后的特征值。权重的设计遵循如下原则： α 与 β 为基础检测权重，满足 $\alpha + \beta = 1$ ，使得仅由 rate 与 FI/FO 组成的基础分数 $S_t^{\text{base}}(a) = \alpha \tilde{R}_t(a) + \beta \tilde{F}_t(a) \in [0, 1]$ ； γ 为持续性的加性奖励权重，独立于基础权重之外，其作用是为反复出现的可疑 key 提供额外加分。当 $\gamma = 0$ 时退化为双签名方案 S1；当 $\gamma > 0$ 时， $S_t(a)$ 的值域扩展为 $[0, 1 + \gamma]$ ，但由于 γ 较小（本文取 $\gamma = 0.2$ ），阈值 $\tau = 0.5$ 的判决语义不受实质影响—— τ 仍位于基础分数值域 $[0, 1]$ 的中点，持续性仅为已超过 τ 的 key 提供额外置信度提升。本文默认取 $\alpha = \beta = 0.5$, $\gamma = 0.2$ 。

归一化的目的在于使不同量纲、不同尺度的特征在同一分数空间可加。本文采用分位数归一化：以良性流量历史统计得到的分位点作为尺度基准，例如：

$$\tilde{R}_t(a) = \min \left(1, \frac{R_t(a)}{Q_{0.99}(R^{\text{benign}})} \right), \quad (3-12a)$$

$$\tilde{F}_t(a) = \min \left(1, \frac{\hat{F}_t(a)}{Q_{0.99}(F^{\text{benign}})} \right), \quad (3-12b)$$

其中 $Q_{0.99}(\cdot)$ 表示 0.99 分位点。 $Q_{0.99}$ 可通过离线分析良性 trace 获得并作为配置参数写入代理；在线场景下亦可采用滑动窗口分位数估计，但需确保窗口长度覆盖多个 epoch 以避免被攻击流量污染。持续性 $\tilde{P}_t(a)$ 可直接按窗口长度 k_p 归一化为 $\tilde{P}_t(a) = P_t(a)/k_p$ 。

在给出检测判决之后，系统还需要将分数空间映射到队列优先级空间。结合当前实验与实现口径，本文采用高/低优先级两队列体系，并定义下一轮活动优先级为：

$$\text{rank}_{t+1}^{\text{act}}(a) = g(S_t(a)) \in \{0, 1\}, \quad (3-13)$$

其中较小的 rank 表示较高优先级。本文采用二值阈值映射：

$$g(S_t(a)) = \begin{cases} 0, & \text{如果 } S_t(a) < \tau, \\ 1, & \text{如果 } S_t(a) \geq \tau, \end{cases} \quad (3-14)$$

其中 $S_{\max} = 1 + \gamma$ 为可疑度的理论上界（ $\gamma = 0$ 时 $S_{\max} = 1$ ）， τ 为检测阈值。当 $S_t(a) < \tau$ 时，key 被分配到高优先级队列并保留高优先级带宽份额；当 $S_t(a) \geq \tau$

时, **key** 被分配到低优先级队列, 在拥塞时优先承受丢弃。若将来扩展到多级优先级队列, 可在此基础上将 $g(\cdot)$ 泛化为多段映射, 但本文实验不展开该扩展。

3.2.6 FI/FO 在网近似统计结构

在网环境中对每个 **key** 精确维护式 (3-2) 的不同对端数量会导致状态空间与更新开销不可接受。为保持可实现性, MS-SatShield 仅对候选集合 $\mathcal{H}_t$ 维护 FI/FO 的近似统计结构。本章以位图 (Bitmap) 作为主线实现^[66], 原因在于其数据平面更新简单 (置位操作) 且内存预算可直接计算。

对每个候选 **key** a , 分配一个长度为 m 的位图 $\text{Bitmap}_t(a) \in \{0, 1\}^m$ 。每当观察到关系对 (a, x) , 计算

$$i = h(x) \bmod m, \quad (3-15a)$$

$$\text{Bitmap}_t(a)[i] \leftarrow 1, \quad (3-15b)$$

其中 $h(\cdot)$ 为哈希函数。epoch 末在星上 Agent 侧读取位图, 令 Z 为位图中 0 的个数, 则可用线性计数^[66] 估计去重后的对端数量:

$$\hat{F}_t(a) = -m \cdot \ln \left(\frac{Z}{m} \right). \quad (3-16)$$

式 (3-16) 的计算包含对数运算, 因此由代理完成, 数据平面只负责置位更新。为避免每个 epoch 全量清零位图带来额外开销, 同时减少攻击者利用窗口切换时刻的空间, 本文采用双缓冲位图: 奇偶 epoch 交替写入不同缓冲区, 读取后由代理对上一缓冲区批量清零, 从而避免数据平面执行大规模清理操作。具体而言, 代理在每个 epoch 末通过控制平面 API 翻转一个单比特寄存器 `epoch_parity`, 数据平面据此选择写入的位图实例, 无需流水线自行判断窗口切换。

若按部署口径同时为 FI 与 FO 两个视角各维护一组双缓冲位图, 则位图状态预算为:

$$M_{\text{Bitmap}}^{\text{deploy}} = \frac{4Km}{8} \text{ Bytes}, \quad (3-17)$$

其中系数 4 分别对应 FI/FO 两个视角与奇偶双缓冲两组实例。在此基础上, 总数据面状态预算可近似写为:

$$M_{\text{total}} \approx M_{\text{CMS}} + M_{\text{cache}} + M_{\text{bytes}} + M_{\text{Bitmap}}^{\text{deploy}} + M_{\text{rank}} + M_{\text{aux}}. \quad (3-18)$$

需要说明的是, 位图结构的误差会随着真实基数接近或超过 m 而快速增大, 即出现饱和。但在本章场景中, FI/FO 主要用于区分正常情况下较小的 FI/FO 与攻击导致的较大增幅, 并不要求精确估计超大规模基数。因此, 在有限内存下位图

依然可作为可用的判别签名。若需要覆盖更大的 FI/FO 范围，可采用 HLL-lite 结构提高抗饱和能力。本文聚焦 Bitmap 实现，暂不展开对比实验。

选择位图加线性计数，是面向星载实现条件做出的取舍。对本文来说，FI/FO 只要能把正常情况下较小的扇入度/扇出度与攻击造成的较大增幅区分开即可，无需在极大基数范围内追求最优无偏估计。这样一来，数据面只需完成哈希和置位，对数运算留给 Agent 侧处理，逐包快路径仍保持常数次寄存器访问。若直接在数据面实现 HLL 或更复杂的去重计数逻辑，寄存器读写与比较操作都会增加，也更难适配星载 PISA 的约束。

算法 3-1: MS-SatSHIELD 多签名在网检测与队列调度流程

输入:

- (1) 当前到达数据包 $p = \langle a, x, len \rangle$;
- (2) 当前窗口数据面状态: CMS 计数矩阵 S_t 、候选缓存 C_t 、候选级字节计数 $bytes_t(\cdot)$ 、活动候选集 $\mathcal{H}_t^{\text{act}}$ 、位图 $\text{Bitmap}_t(\cdot)$;
- (3) 控制面状态与参数: 持续性 $P_t(\cdot)$ 、epoch 长度 Δ 、位图长度 m 、评分权重 α 、 β 、 γ 。

输出: 当前包的队列优先级决策，以及下一轮活动候选配置 $(\mathcal{H}_{t+1}^{\text{act}}, \text{rank}_{t+1}^{\text{act}}(\cdot))$ 。

/* 数据平面逐包更新一级候选与候选级统计 */

- 1 解析得到地址 key a (源地址或目的地址)、对端地址 x (目的地址或源地址) 以及包长 len ;
 - 2 更新 CMS 计数矩阵 (以 (a, len) 更新 d 个 counter) 并按当前估计值刷新候选缓存 C_t ;
 - 3 **if** $a \in C_t$ **then**
 - 4 累加候选级字节计数 $bytes_t(a) \leftarrow bytes_t(a) + len$;
 - /* 对活动候选更新 FI/FO 位图并完成当前包入队 */
 - 5 **if** $a \in \mathcal{H}_t^{\text{act}}$ **then**
 - 6 计算 $i = h(x) \bmod m$ ，并置位 $\text{Bitmap}_t(a)[i] \leftarrow 1$;
 - 7 根据当前映射表 $\text{rank}_t^{\text{act}}(\cdot)$ 确定队列优先级并入队;
 - /* 星上 Agent 在 epoch 末生成下一轮活动配置 */
 - 8 从数据平面读出当前窗口的 CMS/候选缓存摘要与 $bytes_t(\cdot)$ ，形成观测候选集合 $\mathcal{H}_t$;
 - 9 **if** $\gamma > 0$ **then**
 - 10 将 $\{a \mid P_t(a) \geq 2, a \notin \mathcal{H}_t\}$ 合并到评分候选集 $\mathcal{H}_t$ (持续性扩展);
 - 11 读取 $\mathcal{H}_t^{\text{act}}$ 的位图并由式 (3-16) 估计 $\hat{F}_t(\cdot)$ ，更新持续性 $P_{t+1}(\cdot)$;
 - 12 由式 (3-1b)–(3-11) 计算多签名可疑度 $S_t(\cdot)$ 并由式 (3-14) 映射为下一轮活动优先级 $\text{rank}_{t+1}^{\text{act}}(\cdot)$;
 - 13 令 $\mathcal{H}_{t+1}^{\text{act}} \leftarrow \mathcal{H}_t$ ，并将 $\mathcal{H}_{t+1}^{\text{act}}$ 与 $\text{rank}_{t+1}^{\text{act}}(\cdot)$ 写回交换机表项，使其在 epoch $t + 1$ 生效;
 - 14 切换/清理位图缓冲区以开始下一 epoch;
 - 15 **return** $(\mathcal{H}_{t+1}^{\text{act}}, \text{rank}_{t+1}^{\text{act}}(\cdot))$ 。
-

双缓冲不只可以方便清零。它把窗口切换时的大规模状态重置从逐包快路径

中剥离出来，避免攻击者利用窗口切换处制造瞬时状态不一致；同时，奇偶缓冲交替写入保证当前 epoch 的统计始终对应一块干净位图，而上一块位图可以在 Agent 侧按批次读取和清理。这种时序安排与前文定义的 epoch 级控制流保持一致。

3.2.7 多签名在网检测与队列调度

算法 3-1 给出了 MS-SatShield 的整体流程。该流程与图 3-2 一一对应，并明确区分了数据平面逐包处理与星上 Agent 逐 epoch 更新。

下面给出一个以目的地址为 key 的单个 epoch 的运行示例。设某诱饵目的地址 a_d 在 epoch t 内同时接收来自大量 Bot 的低速访问。对经过被攻击卫星的数据包，数据面先根据 a_d 的哈希位置更新 CMS 中的多个计数器，并用这些近似计数判断 a_d 是否应进入候选缓存；一旦 a_d 占用候选缓存中的某个表项，该表项对应的字节寄存器 $\text{bytes}_t(a_d)$ 开始持续累加。由于当前实验默认以目的地址为 key，数据面对每个不同源地址 x 仅做一次位图置位，因此窗口结束时会形成较大的 $\hat{F}_t(a_d)$ 。如果 a_d 在前若干窗口中也多次进入候选集，则其持续性 $P_t(a_d)$ 也会较高。epoch 结束后，Agent 读取这些统计量并计算 $S_t(a_d)$ ；当其超过阈值 τ 时， a_d 会在 epoch $t+1$ 被映射到低优先级队列，后续所有发往 a_d 的流量在拥塞时都会优先承受丢弃。相对地，正常热门目的地址虽然可能具有较高的 $R_t(a)$ ，但若其 FI 低于攻击类诱饵目的地址，或者只在短时间内进入候选集，则综合得分仍可能低于阈值，从而保留在高优先级队列。这个例子表明，MS-SatShield 的效果来自频率筛选、结构判别、历史记忆和队列执行之间的配合，而不是某个单独特征。

3.3 实验结果与分析

3.3.1 实验平台与基线方案

为验证 MS-SatShield 的检测与缓解效果，本文搭建了由 LEO 攻击仿真、P4/PISA 约束下的数据面原型设计和流量重放评估组成的实验框架。LEO 层面采用与 ICARUS/SatShield 一致的仿真参数与星座模型（基于 Hypatia^[60] 仿真框架），以保证攻击与背景流量分布具有可对齐的现实依据^[24, 30]；数据平面层面按 P4^[63] 的可编程流水线约束实现 CMS 频率估计、固定容量候选缓存更新、候选位图更新与两队列优先级映射；星上 Agent 在每个 epoch 周期性读取寄存器并下发队列优先级映射。

需要说明的是，本文实验中的“P4/PISA 约束下的数据面原型”并不等同于已经在某一具体星载芯片上完成流片或上星验证。其实验目的，是检验算法能否被约束在 P4/PISA 允许的操作集合之内：数据面逐包只执行固定次数的哈希、计数、

寄存器读写、位图置位和队列标记；需要排序、归一化、线性计数对数估计和参数刷新等较复杂步骤均在星上 Agent 的 epoch 级控制闭环中完成。结合消融实验给出的状态预算，默认配置下统一部署开销约为 200 KB，低于典型地面 P4 ASIC 的片上 SRAM，也处于未来星载 FPGA/DPU/可编程交换模块可考虑的量级。因而本文证明的是“轻量级在网安全逻辑在工程约束下具备部署可能性”，而不是声称现有所有卫星均已具备完整 P4 数据面能力。

每轮实验按四个顺序步骤执行。在给定 epoch 长度 Δ 与当前拓扑快照 G_t 的条件下，仿真器先生成该窗口内的链路状态、最短路径与目标链路位置，并据此确定背景流与攻击流的转发路径。随后，流量重放模块将良性 trace 映射为地面终端对之间的业务请求，再叠加满足式 (3-5) 的攻击流，形成带标签的逐包输入序列。接着，输入序列按时间顺序送入在网处理程序，由数据平面逐包执行 CMS 更新、候选缓存刷新、活动候选位图置位与队列映射，同时由星上 Agent 在 epoch 末读取寄存器摘要并生成下一轮 rank 配置。最后，离线评估脚本依据攻击生成器输出的标签文件，对 detector 级和 identifier 级结果分别计算 Precision、Recall、F1，并汇总反应时间与良性吞吐等缓解指标。

这样拆分之后，观测、判决和生效三个时序阶段被明确分开。对任意一个 epoch，数据平面实际执行的是上一窗口导出的活动配置 $(\mathcal{H}_t^{\text{act}}, \text{rank}_t^{\text{act}})$ ，当前窗口只负责生成下一轮配置。这一时序与前文的双缓冲位图和 epoch 级控制流定义保持一致，也意味着后文讨论的 reaction time 表示攻击出现、当前窗口统计完成以及下一窗口缓解生效在内的闭环生效时间，而不是单次表项写入延迟。

基线对比方案主要包括：

- (1) SatShield (Rate-only, 记为 S0)：仅使用归一化聚合速率 $\tilde{R}_t(a)$ 作为可疑度（即 $\alpha = 1, \beta = \gamma = 0$ ），通过阈值 $\tau = 0.5$ 进行二元判决。该基线是 SatShield 原文^[30] 在本文框架内的等效实现，保留了其一级候选生成与速率阈值判决的基本逻辑；
- (2) MS-SatShield (rate + FI/FO, 记为 S1)：在候选集合上加入 $\hat{F}_t(a)$ ，取 $\alpha = \beta = 0.5, \gamma = 0$ ；
- (3) MS-SatShield (rate + FI/FO + Persistence, 记为 S2)：进一步加入持续性 $P_t(a)$ ，取 $\alpha = \beta = 0.5, \gamma = 0.2$ ，用于对抗脉冲式攻击与候选波动。

本章三组基线只在第二级判别特征上存在差异，其余实验条件保持一致：采用相同的拓扑与路由快照、相同的背景与攻击输入、相同的一级候选生成器（CMS + 候选缓存）、相同的 key 视角（默认目的地址）、相同的二值队列结构，以及相同的检测阈值 $\tau = 0.5$ 。因此，S0、S1、S2 的比较实际发生在“仅速率、速率 + 结构、速率 + 结构 + 历史”三种设置之间，用来考察每个新增签名带来的边际收益。这

样设置后，一旦结果发生变化，就可以更直接地归因到新增特征本身，而不是候选生成、队列策略或实现细节的差异。

三种方案均使用相同阈值 $\tau = 0.5$ 。由于 $S0$ 与 $S1$ 的可疑度值域均为 $[0, 1]$ ， $\tau = 0.5$ 的判决语义一致； $S2$ 的 $\gamma = 0.2$ 是加性奖励（见 §3.2.5），不会改变基础判决逻辑，因此共用 τ 仍然公平。这样的对比也与后续消融实验的三个层级保持一致。

此外，ICARUS 在同一星座配置下采用了空载网络、基于国内生产总值的引力模型与基于人口分布的引力模型的三类背景流量场景，并通过地理网格离散化评估区域级连通性影响。本文不复现其全量攻击优化流程，而是沿用 SatShield 的带标签流量回放与在网检测、缓解这一实验主线，用来回答单节点检测器在退化策略下的区分能力问题。两类实验关注的重点不同：ICARUS 更强调攻击可达性与代价-可检测性权衡，本文更强调在网检测与缓解精度，因此两者结果不作直接的数值优劣比较^[24, 30]。

3.3.2 实验参数设置

良性背景流量采用 CAIDA 匿名 trace 进行重放，并依据人口分布模型映射为城市对之间的通信需求；同时保证背景流量不会使任意链路长期超过 90% 利用率，以避免“未攻击已拥塞”的非预期干扰^[30]。攻击流量由 LFA 生成器构造：在每个攻击实验中随机选取城市对作为被攻击通信对，选择目标链路 e^* 并生成满足式 (3-5) 的攻击注入速率。表 3-1 总结了主要的网络与终端参数。

需要说明的是，ICARUS 为了分离攻击最低资源门槛与背景噪声干扰，常采用无良性流量的保守场景作为上界评估；本文则采用含良性背景的重放场景，更接近部署侧的观测条件。两种设定并不矛盾：前者用于估计最坏情况下的攻击可行性，后者用于检验检测器在真实流量长尾存在时的稳健性。本文报告的检测收益不直接外推为最低攻击成本结论，而是作为给定背景流量下的防御性能证据^[24, 30]。

（1）良性背景流量的构造与映射

选择 CAIDA 匿名互联网 trace 作为良性背景，主要基于两点。该数据集包含真实互联网主干链路中的长尾分布、突发性与热门目的地效应，比人工合成的 Poisson/CBR 流更容易暴露 Rate-only 检测器在合法大流与攻击聚合之间的区分困难；同时，SatShield 已在同类 LEO-LFA 评估中采用相近的数据来源与映射思路，便于与既有结果建立可对齐的比较口径。本文并不依赖 CAIDA 在绝对业务占比上的精确刻画，而是利用其提供的真实长尾结构，检验检测器在存在正常重流时是否仍具有区分能力。

具体构造时，原始 **trace** 先按链路容量尺度进行比例缩放，使重放后的背景负载与 20 Gb/s 量级的 ISL 场景相匹配；再依据人口加权的城市对模型，将 **trace** 中的通信需求映射为 LEO 地面终端之间的源宿请求。这样的两阶段映射可以减少直接把互联网 **trace** 原样投射到星座链路所带来的失真：前一阶段保证负载量级与星座容量约束一致，后一阶段将业务空间分布与地理人口分布联系起来，使热点城市之间更容易形成较高的正常通信强度。为避免背景流量本身持续压满链路，本文进一步约束任意链路的长期良性利用率不超过 90%，从而保证后续观测到的拥塞主要由攻击注入触发，而不是由背景配置不当造成。

（2）攻击流生成与真实标签（Ground Truth）定义

攻击流量的生成遵循 LFA 的基本构造逻辑：先在当前快照上选定目标链路 e^* ，再从其上游可达区域选择 Bot、从下游可达区域选择 Decoy，使 Bot 到 Decoy 的转发路径在统计意义上尽可能汇聚到 e^* 上。为保证不同退化设定之间具有可比性，本文在比较 M 或 on-off 节律时，尽量保持总攻击注入量 A 处于同一量级，只改变每个目的地址承载的攻击聚合速率，以及这些 key 在相邻 epoch 中是否连续出现。这样可以更明确地将实验结论归因于特征退化方式本身，而不是攻击总强度的变化。

由于本文实验默认按目的地址进行检测，评估标签也以目的地址为单位。具体来说，如果某个目的地址被攻击流用作 Decoy，并且这些攻击流经过目标链路 e^* 形成拥塞贡献，则该目的地址记为攻击相关 key；其他目的地址记为正常 key。这样可以保证检测输出、缓解对象和 Precision/Recall/F1 的统计单位一致。若某一目的地址同时承载背景流量与攻击流量，只要其在观测窗口内存在攻击贡献，仍将其视为攻击相关 key。

为了展示速率特征退化后多签名如何恢复检测能力，本文在保持总攻击注入量 A 基本不变的前提下，构造了多 Decoy 分摊的退化设定。表 3-2 给出了这一设定的参数组。表中包含 $M = 1$ 的下界情形，它仅作为对照，用来展示当每个目的聚合速率极大时，Rate-only 可以工作；随着 M 增大，单 Decoy 的聚合速率约为 A/M ，并逐步逼近正常业务的长尾区间，从而使 Rate-only 的候选质量下降，也引出 FIFO 的必要性。

表 3-1 实验主要参数设置
(与 SatShield/ICARUS 对齐)

参数	取值
星座拓扑	Starlink Shell 1 (1584 颗卫星)
ISL 容量 C_e	20 Gb/s
单终端上行/下行峰值	64 Mb/s(上行) / 384 Mb/s (下行)
单星最大服务终端数	2080
背景流量来源	CAIDA trace 重放并按人口分布模型映射
epoch 长度 Δ	1.0 s
候选规模 K	$\text{clip}(0.3 \cdot N_{\text{keys}}, 50, 1000)$ (自适应) ^a
位图长度 m	256 bits
持续窗口 k_p	5 epochs
可疑度阈值 τ	0.5
活动队列数 Q	2 (高/低优先级)
高优先级带宽比	0.8 (80% 带宽保留给良性/低可疑流量)

^a N_{keys} 为上一 epoch 中一级候选生成器 (CMS + 候选缓存) 报告的活动 key 估计数量。

表 3-2 退化设定: 多 Decoy 分摊
(降低按目的聚合速率, 放大 FI/FO)

组别	$ \mathcal{B} $	r (Mb/s)	A (Gb/s)	M	单 Decoy 聚合速率 A/M
B1	2000	5	10	1	10 Gb/s
B2	2000	5	10	10	1 Gb/s
B3	2000	5	10	100	100 Mb/s
B4	2000	5	10	1000	10 Mb/s

3.3.3 评价指标与绘制方式

检测侧采用 Precision、Recall、F1 作为主要指标。令真实攻击相关 key 集合为 F , 算法输出集合为 $\hat{F}$, 则:

$$\text{Precision} = \frac{|F \cap \hat{F}|}{|\hat{F}|}, \quad (3-19a)$$

$$\text{Recall} = \frac{|F \cap \hat{F}|}{|F|}, \quad (3-19b)$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (3-19c)$$

其中 F 的定义需与攻击生成器一致: 若以源地址为 key, 则 F 定义为 Bot 源地址集合; 若以目的地址为 key, 则 F 定义为诱饵目的地址集合。缓解侧采用反应时

间（reaction time）与良性吞吐下降（throughput drop）等指标，与 SatShield 的定义保持一致：reaction time 表示从观测到首个攻击包到系统开始生效缓解的时间间隔；throughput drop 表示攻击发生前后良性吞吐的下降比例^[30]。

本章同时统计 detector 级指标与最终识别级指标，因为 MS-SatShield 具有清晰的两级结构：第一级回答攻击相关 key 是否进入候选集，第二级回答进入候选后能否被正确赋予较低优先级。若只报告最终 F1，就无法区分性能下降究竟来自候选覆盖不足，还是来自候选内判别失败。将两者拆开后，便可以像图 3-3(a) 那样，更准确地定位 Rate-only 方案的失效位置。

除时域曲线外，本文对吞吐与反应时间的统计都采用攻击期口径，而不是全时段口径。具体来说，良性吞吐比例按攻击持续阶段的 epoch 级结果计算并求平均，以避免将预热阶段和攻击结束后的恢复阶段纳入统计后，削弱对攻击期间缓解效果差异的体现；reaction time 则以首个攻击包进入系统为起点，以首批低优先级缓解规则在数据面实际生效为终点，因此反映的是包含一个 epoch 观测与配置延迟在内的闭环响应时间，而不是单次表项写入延迟。对于 on-off 场景，文中另外给出时域曲线，用于展示切换处的波动与恢复过程。

图 3-3 的绘制流程如下：对每组实验参数，先利用仿真器与流量重放器生成带标签的包序列；再将包序列输入在网处理程序，完成逐包处理与逐 epoch 统计；最后在离线评估脚本中按式 (3-19a)–(3-19c) 计算 Precision、Recall 和 F1 并绘制曲线。为与 SatShield 的复现逻辑对齐，本文同时给出一级候选捕获准确性（对应 detector）与攻击相关 key 识别准确性（对应多签名判决）两类 F1 结果，从而将候选生成质量与候选内判别能力分开分析，避免混淆误差来源。

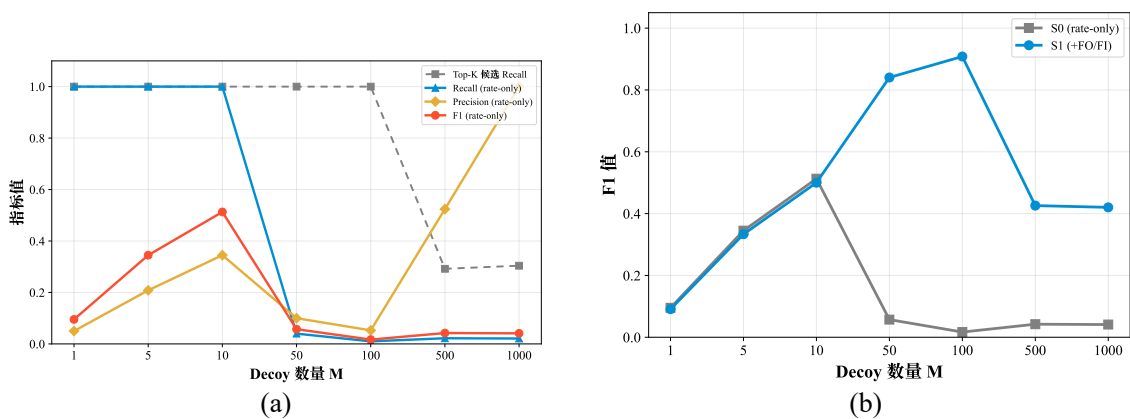


图 3-3 退化攻击下的检测性能。(a)Rate-only 的一级候选与判决指标；(b)S0 与 S1 的 F1 对比

图 3-3 的真实标签统一按诱饵目的地址口径定义，因此图 3-3(a) 中的一级候选

Recall 与图 3-3(b) 中的 F1 都是在同一标签空间下统计得到的。

3.3.4 退化证据链与多签名收益分析

(1) 可分性分析与一级候选诊断

图 3-1 从分布层面验证了速率可分性随 M 退化的变化过程。 $M = 1$ 时，良性 $p_{99} \approx 485$ Mb/s 与攻击聚合速率 10,024 Mb/s 之间存在约 21 倍间隙，一级候选生成器可轻松捕获攻击 key。随着 M 增大到 100，攻击 key 的 per-key 速率降至 100 Mb/s，已基本落入良性长尾区间，速率特征失去区分力。

图 3-3(a) 从检测管线角度进一步诊断了 Rate-only 方案的失效模式。该图同时给出一级候选 Recall、阈值判决 Recall、Precision 与 F1 四条曲线。当 $M \leq 100$ 时一级候选 Recall 保持 1.0，说明候选生成模块在中等退化下仍具备攻击 key 捕获能力；然而阈值判决 Recall 在 $M = 50$ 时骤降至 0.04，F1 全程不超过 0.52，表明 Rate-only 方案的瓶颈在于判决阈值无法区分攻击 key 与良性重流 key。当 $M \geq 500$ 时，一级候选 Recall 本身亦降至约 0.3，意味着大量攻击 key 的聚合速率已低于良性 key 而被候选生成器排除。需要说明的是， $M \geq 500$ 时 Rate-only 的 Precision 出现回升（ $M = 1000$ 时达 1.0），但这一现象源于 Recall 极低时（ ~ 0.02 ）判为正的 key 集合 $|\hat{F}|$ 极小，分母缩减导致 Precision 的数值放大，并不反映检测能力的实质改善。

从图 3-3(a) 还可以看到，Rate-only 方案的失效不是一步发生的，而是先出现候选仍在、判决先失效，随后再演变为候选与判决同时失效。前一阶段说明，单一速率特征的困难在于即使候选已被捕获，也难以把它们从良性重流中分离出来；后一阶段则表明，当 per-key 速率继续下降后，任何依赖重 key 假设的方案都会先撞上候选覆盖率上限。区分这两类失效模式，有助于解释本文为何把改进重点放在第二级多签名判别上，而非一味扩展一级候选规模。

(2) FI/FO 在静态退化场景下的作用

图 3-3(b) 对比了 S0 (Rate-only, $\alpha = 1$) 与 S1 (+FI/FO, $\alpha = \beta = 0.5$) 的 F1 随 M 的变化，展示了 FI/FO 特征在静态退化场景下的作用。S1 的 F1 曲线大致呈现三段变化：

- **重叠段** ($M \leq 10$)：S0 $\approx$ S1，两条线几乎重合。此时攻击 key 聚合速率 ≥ 1 Gb/s，Rate-only 已能检出全部攻击 key，FI/FO 不提供额外增益。
- **分离段** ($M = 50 \sim 100$)：S1 优于 S0。 $M = 50$ 时 S0 的 F1 值骤降至 0.057，而 S1 跃升至 0.84； $M = 100$ 时 S1 达峰值 0.908，S0 仅 0.017。其机理在于：虽然攻击 per-key 速率（100 $\sim$ 200 Mb/s）已接近良性 p_{99} ，但每个 Decoy 的

Fan-in ($FI = B/M = 20 \sim 40$) 仍远高于良性 key 的典型 FI ($1 \sim 5$), FI/FO 特征提供了判别能力。

- **衰减段** ($M \geq 500$): S1 的 FI 值下降至 0.42。攻击 per-key 速率降至 $20 \sim 35$ Mb/s, 部分攻击 key 脱离一级候选集 (一级候选 Recall 约 0.3), 候选集的覆盖率瓶颈成为 S1 的 FI 值的天花板。此外, 此时 $FI = |\mathcal{B}|/M = 2000/500 = 4$, 已接近良性 key 的典型 FI 上界 ($1 \sim 5$), FI 的区分能力同步退化, 进一步限制了 S1 的判别空间。

需要指出的是, 在所有 M 值下都存在约 20 个良性重流 key (rate 为 $290 \sim 606$ Mb/s, FI 为 $13 \sim 26$) 被误判为可疑。这些 key 对应真实的热门目的地, 其 rate 与 FI 都落在良性分布的长尾区域, 评分自然会超过阈值 $\tau = 0.5$, 因此构成了当前参数配置 ($\tau = 0.5, \alpha = \beta = 0.5$) 下基于 “rate+FI” 检测器的误报基线。调整 τ 可以改变该数值, 但也会影响 Recall (详见消融实验 §3.3.5)。这部分误报造成的影响, 不在这里预设为可接受或不可接受, 而是通过图 3-4(d) 的良性吞吐比例结果加以量化。

目的地址级 key 的一个直接代价是缓解粒度较粗: 当某个目的地址同时承载攻击流量和正常流量时, 发往该目的地址的正常流量也可能受到影响。本文没有对可疑 key 执行无条件硬丢弃, 而是通过低优先级队列和带宽保留机制降低其调度优先级。因此, 这类误伤不会被正文假设为不存在, 而是通过 Precision、误报 key 数量以及良性吞吐比例共同反映。

图 3-3(b) 表明, FI/FO 的优势主要集中在这样一段区间: 速率特征已经开始失真, 但攻击相关 key 还没有跌出候选集。这说明 MS-SatShield 的收益主要来自候选内结构判别能力的恢复, 而不是对极端碎片化攻击的充分免疫。当 M 继续增大, FI 也回落到良性区间时, 本方法仍会受到一级覆盖率与结构可分性两方面限制。

(3) 持续性特征在动态攻击场景下的价值

图 3-3(b) 仅展示 S0 与 S1 的对比, 未包含 S2 (+Persistence)。其原因在于: 在静态退化场景中, 攻击 key 在每个 epoch 均进入一级候选集, 导致持续性 $\tilde{P} \approx 1.0$; 同时约 20 个良性重流 key 也几乎在每个 epoch 进入候选集, $\tilde{P}$ 同样趋近 1.0, 持续性特征对两者加分相同, 无法提供额外区分能力。

持续性的作用主要体现在动态退化场景中。为此, 图 3-4 给出了脉冲式 on-off 攻击下的时域响应。实验固定退化强度为 $M = 100$, 前 5 个 epoch 为 warmup, 随后攻击按 on-off 各 10 个 epoch 的节律周期性切换, 总仿真时长为 100 个 epoch。图中红色阴影表示攻击 on-phase; 图 3-4(a) 给出目标链路利用率, 图 3-4(b) 给出一级候选命中的攻击 key 数量, 图 3-4(c) 和图 3-4(d) 分别给出 S0、S1、S2 三种方案的

Recall 与良性吞吐比例时域曲线。

图 3-4(a) 表明, 攻击 on-phase 到来时目标链路利用率迅速升至超载区间, off-phase 又回落至正常水平, 说明该设定确实对应时域上的动态退化, 而不是前文的静态退化场景。图 3-4(b) 则表明, 一级候选集在 on-off 切换附近存在可见波动: on-phase 内命中数接近攻击 key 总数, 但在切换处会短暂回落, 这正是持续性特征要补偿的部分。进一步看检测结果, 图 3-4(c) 中 S0 在 $M = 100$ 下几乎失效 (on-phase 平均 Recall ≈ 0.01); S1 借助 FI/FO 可以实现高召回检测 (on-phase 平均 Recall ≈ 0.990), 但在切换处仍会因候选波动出现轻微起伏; S2 则进一步将 on-phase 平均 Recall 提升到 0.999。需要说明的是, 在攻击关闭阶段不存在真实攻击 key, 此时 Recall 的分母为空。为保持时域曲线连续, 本文约定该阶段的 Recall 记为 1.0。

S2 的提升来自两个机制叠加。持续性采用加性奖励权重 ($\alpha = \beta = 0.5, \gamma = 0.2$), 即 γ 作为额外加分而不从基础权重中借用, 因此 S2 的基础检测能力与 S1 保持一致。与此同时, 当 $\gamma > 0$ 时, 系统会将历史上 $P \geq 2$ 但未进入当前一级候选集的 key 也纳入评分候选集 (见算法 3-1 中的持续性扩展步骤), 从而为短暂跌出候选集的可疑 key 保留一段记忆。这个机制在缓解侧也有体现: 图 3-4(d) 显示 S0 的良性吞吐仅为 0.572, S1 恢复到 0.881, S2 进一步提升到 0.885。此外, 由于本文取 $k_p = 5$, 在 off 持续 10 个 epoch 后, 所有攻击 key 的 $P_t(a)$ 都会衰减为 0, 持续性分数随之归零, 因此不会把历史攻击残留带入后续判决。

从时域角度看, S2 的改进也主要集中在 on-off 切换处, 而不是稳态阶段。换句话说, Persistence 不会把原本不可分的静态样本变得可分, 它做的是用跨 epoch 的弱记忆平滑短时波动, 减少候选短暂丢失带来的性能下降。这与图 3-3(b) 中 S2 在静态场景下几乎没有额外收益的结果是一致的: Persistence 解决的是时间连续性问题, 而不是静态结构可分性问题。

3.3.5 消融实验与参数敏感性分析

为进一步说明仅对一级候选维护 FI/FO 时的资源与精度权衡, 本节对候选规模 K 、位图长度 m 、epoch 长度 Δ 三个参数进行敏感性实验。实验固定退化设定为 $M = 100$ (速率特征开始失效的分界点), 采用 S1 ($\alpha = \beta = 0.5$) 方案进行评估。

(1) 候选规模 K

图 3-5 展示了在 $M = 100$ 、S1 方案下, Recall 与 F1 随 K 的变化。当 $K < 500$ 时, Recall 与 F1 几乎同步上升, 这说明此区间内一级候选缓存容量过小, 大量攻击 key 尚未进入候选集, 是检测性能的主要瓶颈。K 增至 1000 时 Recall 饱和至 1.0,

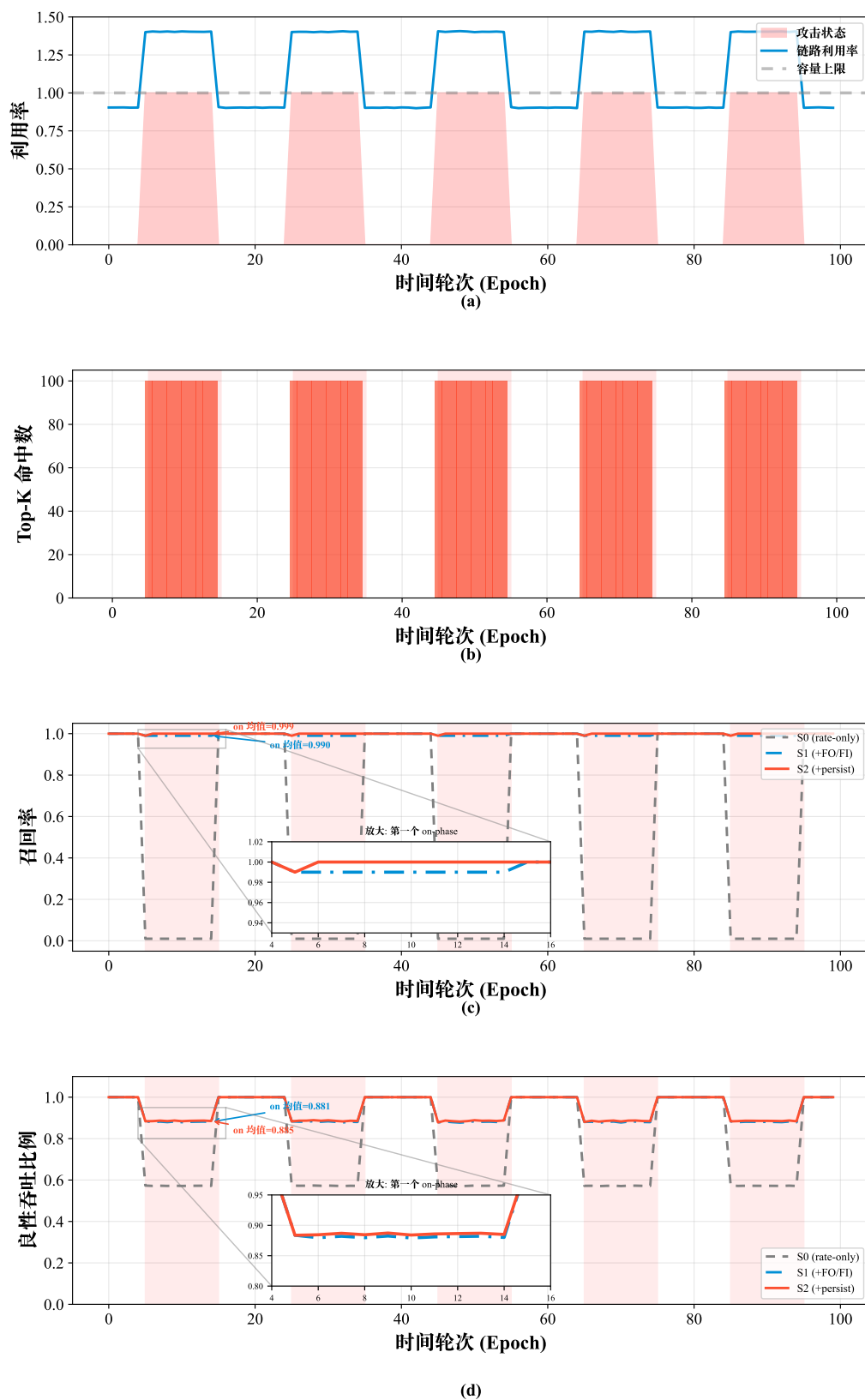
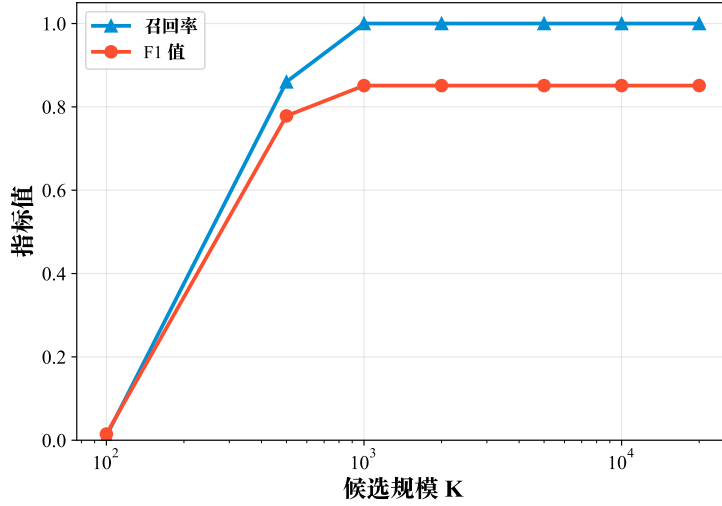


图 3-4 脉冲式 on-off 攻击下的时域响应 ($M = 100$)。 (a) 目标链路利用率; (b) 一级候选命中的攻击 key 数量; (c) 不同方案的 Recall; (d) 不同方案的良性吞吐比例

图 3-5 候选规模 K 对 Recall 与 F1 的影响

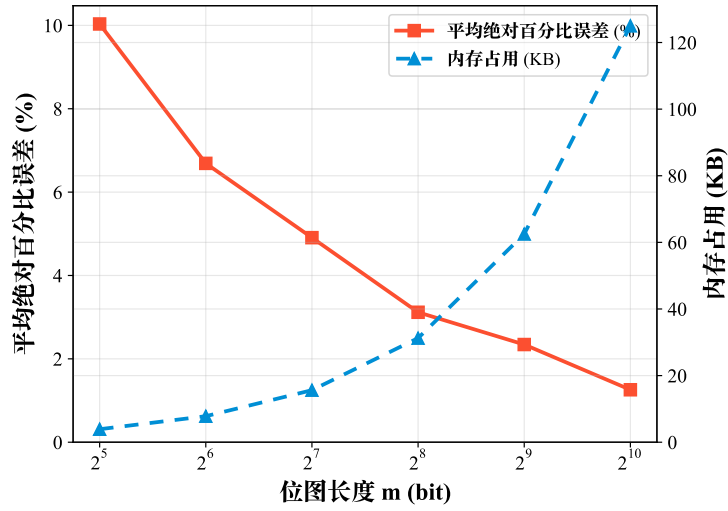
F1 达到约 0.91。继续增大 K （2000 ~ 20000）后 Recall 保持在 1.0，F1 略有下降（趋近 0.90），原因是更多良性 key 涌入候选集增加了误报基数。从资源角度看，若仅计单个 256 bit 位图，则 $K = 1000$ 时位图本体约为 $K \times m/8 = 32$ KB；但按本文统一的部署口径，同时计入 CMS 矩阵、候选缓存、FI/FO 双视图、双缓冲位图、字节计数器、rank 表与辅助寄存器后，总状态预算约为 200 KB，仍显著低于典型可编程交换机的片上 SRAM 容量。因此 $K \approx 1000$ （或等效地 $K \approx 0.3 \cdot N_{\text{keys}}$ ）是本文实验设置下的经验较优折中。

（2）位图长度 m

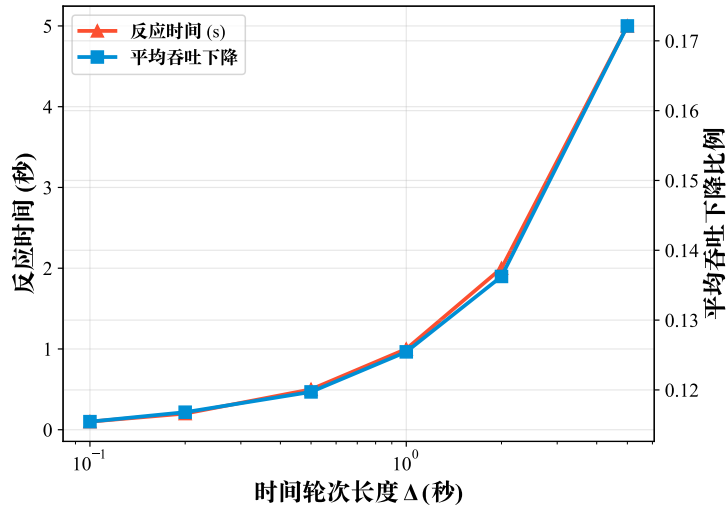
图 3-6 展示了在 $M = 100$ 、S1 方案下，FI/FO 估计误差（MAPE）与总内存随 m 的变化。 $m = 32$ 时 MAPE 约 4.4%， $m = 256$ 时降至 1.1%， $m = 1024$ 时进一步降至 0.5%。内存开销从 $m = 32$ 的约 20 KB 线性增长至 $m = 1024$ 的约 700 KB（ $K = 1000$ 时）。综合考虑精度与开销， $m = 256$ 是本文评估点下经验上较优的折中：MAPE 已降至 1.1%，足以区分攻击 FI（ ≥ 20 ）与良性 FI（ $1 \sim 5$ ）；按统一部署口径计算，总状态预算约为 200 KB，处于可编程交换机可承受范围内。

（3）epoch 长度 Δ

图 3-7 展示了在 $M = 100$ 、S1 方案下，反应时间与平均良性吞吐下降随 Δ 的变化。 $\Delta = 0.1$ s 时反应时间最短（约 0.2 s），但代理交互频率高达 10 次/秒，星上计算开销增加较多； $\Delta = 5$ s 时反应时间增至 10 s，对应良性吞吐下降约 24%。在 Δ 为 0.5 ~ 1.0 s 的推荐区间内，反应时间为 0.5 ~ 1.0 s（约 1 ~ 2 个 epoch），良性吞吐下降约 12 ~ 14%，系统在响应速度与计算开销之间取得较好平衡。本文选择

图 3-6 位图长度 m 对 FI/FO 估计误差与总内存的影响

$\Delta = 1.0$ s 作为默认值。

图 3-7 epoch 长度 Δ 对反应时间与良性吞吐下降的影响

三组消融结果表明， K 、 m 与 Δ 分别对应三种不同层面的资源约束： K 决定候选覆盖上限， m 决定候选内结构特征的估计精度， Δ 决定闭环时延与统计一致性。它们之间并非彼此独立，例如增大 K 会放大位图总开销，缩短 Δ 会提高 Agent 读写频率并加剧候选波动。因此，本文给出的 $K = 1000$ 、 $m = 256$ 、 $\Delta = 1.0$ s 是面向当前星座规模、链路容量与攻击设定得到的一组经验折中参数。若部署场景发生变化，这三者应被视为联合调参量，而不是彼此孤立的常数。

需要指出的是，地面网络中已经提出了若干针对 LFA 的检测与缓解方案。在 SDN 集中式方案中，Crossfire^[25] 利用网络范围内的流量重布局与 SDN 集中控制来检测并解除链路拥塞，Coremelt^[26] 利用 NetFlow 分析识别可疑通信对，LinkScope^[43]

通过主动探测检测目标链路拥塞，Woodpecker^[44] 和 RADAR^[45] 则分别从路径重路由和相关性分析角度实现 SDN 驱动 LFA 防御。在可编程数据面路线上，Ripple^[46] 提出了基于 P4 的去中心化 LFA 防御，Mew^[47] 进一步扩展了 P4 数据面的大规模 LFA 检测能力。然而，这些方案都难以直接应用于 LEO 场景：SDN 集中式方案依赖低时延控制通道与全局视图，而 LEO 的星地和星间控制链路存在时延与带宽限制；P4 方案虽然实现了数据面检测，但依赖地面交换芯片更充裕的片上资源（如 Tofino 2 的 80 MB SRAM），难以满足星载设备的 SWaP 约束。MS-SatShield 的优势在于将检测与缓解逻辑下沉到星载可编程交换机，并在约 200 KB 的统一部署状态预算下实现近实时在网缓解。

3.3.6 方法适用范围与外推限制

本文方法的第一类限制来自候选覆盖率。实验已经表明，当攻击分散度继续增大（如 $M \geq 500$ ）时，一级候选召回下降会成为主要瓶颈，多签名判别能力也随之受制于候选质量上限。该现象与 ICARUS 在高不确定路由、高分散流量下给出的资源上升但攻击仍可行这一结论并不冲突：二者共同说明，防御侧需要同时改进候选生成与候选内判别，而不能只优化后者^[24]。

在极端小流和强分散场景下，候选漏检会直接限制整体检测效果。由于二级评分只能作用于已经进入候选集的 key，若攻击 key 未被一级候选生成器覆盖，则后续多签名模块无法恢复该漏检。因此，最终 Recall 的上限受一级候选 Recall 约束。本文在 $M \geq 500$ 的实验中已经观察到这一现象：攻击流量被分散到大量 Decoy 后，每个目的地址上的聚合速率和 Fan-in 均下降，攻击 key 大量脱离候选集，性能瓶颈从候选内判别转移到候选覆盖率。

第二类限制来自路由策略与网络级目标的差异。本文默认单星本地检测与队列调度处置，未显式建模多路径随机负载分担下的概率攻击构造，也未直接评估区域级断连目标。ICARUS 的结果表明，增强路径多样性虽然可以提高攻击成本，但也可能引入较大的时延代价；因此，本文结论应理解为既定路由与本地观测条件下的检测与调度优化收益，而不应直接外推为全网路由韧性的充分条件^[7, 9, 24]。

3.4 本章小结

本章针对大规模低轨卫星网络中的链路洪泛攻击，提出了基于多签名在网检测与队列调度的 LFA 防御优化方法 MS-SatShield。该方法在 SatShield 的一级候选生成与队列调度框架中，引入 Fan-in/Fan-out 与持续性等结构性签名，用于弥补速率特征在退化攻击下的判别能力下降；同时通过“全量 sketch 粗筛 + 候选集显式

恢复 + 候选内不同对端数量统计”的两级设计，将数据面状态开销控制在星载可实现范围内。

实验结果表明，MS-SatShield 的主要收益来自两个方面。其一，在多 Decoy 分摊导致速率特征失真的场景中，FI/FO 能够恢复候选内的结构判别能力，使方法在 Rate-only 基线已失效时仍保持检测能力。其二，在脉冲式 on-off 攻击下，持续性特征并不改变静态样本的可分性，而是通过跨 epoch 的弱记忆平滑候选波动，主要改善攻击启停切换处的识别连续性与吞吐保护。消融实验表明，在本文评估设置下， $K \approx 1000$ 、 $m = 256$ 、 $\Delta = 1.0 \text{ s}$ 是检测精度与资源开销之间较合适的一组经验折中参数。

然而，本章方法仍然受限于单星、单链路视角。当攻击进一步碎片化时，一级候选覆盖率会重新成为主要瓶颈；即使本地节点已经完成检测与队列调度处置，攻击流仍可能在到达被攻击链路之前占用上游带宽。因此，本章解决的是“单节点如何在受限数据面上尽可能准确地识别退化攻击并完成队列调度优化”的问题，而第四章将进一步回答“如何通过多星协同与逐跳推回，把防御位置从被攻击链路前移到网络上游”的问题。

第四章 基于多星协同与逐跳推回的 LFA 防御优化

4.1 引言

上一章（第三章）讨论了单星、单节点视角下基于多签名在网检测与队列调度的 LFA 防御优化，利用可编程交换机的数据面线速能力实现了对可疑聚合 key 的实时识别与队列调度处置。这类星上本地闭环的优势很直接，即不必依赖地面控制器完成统计上报和策略下发，从而避开卫星到地面以及星间控制链路带来的时延开销。在 LEO 星座网络中，这一优势更容易体现。现有研究指出，卫星网络控制环路的延迟可达几十毫秒，而攻击流量能够在亚毫秒级完成自适应变化，因此依赖中心控制器的传统 LFA 防御很难跟上快速演化的攻击^[30]。

然而，只依赖被攻击链路附近的本地缓解仍有两方面不足。本地缓解往往发生在拥塞已经形成之后，攻击流量在到达目标瓶颈链路前，已经沿多条星间链路消耗了大量带宽和队列资源。这种末端丢弃会在网络内部制造额外拥塞风险，也会进一步加重对良性业务的影响。另一方面，低轨卫星网络的 LFA 攻击具有较强的全网性与漂移性。攻击者可以利用公开的轨道和拓扑信息，结合低时延路由偏好带来的路径可预测性，在全球范围内灵活选择 Bot 分布和攻击路径，使攻击证据以碎片化形式分散在多个卫星节点上^[24]。在这种情况下，即使单个节点具备本地检测能力，缺乏跨节点信息共享仍会限制整体缓解效果。各节点只能在本地丢弃已经到达的攻击流量，难以把抑制动作前推到更靠近攻击源头的上游节点，结果就是大量链路带宽被白白占用。

上一章已经说明，随着 LEO 网络承载的合法重流、回传流与业务连接持续增加，单纯依赖速率特征的本地检测会面临更高的误判风险；MS-SatShield 通过多签名检测缓解了这一问题。然而，当检测与处置仍局限在被攻击链路附近时，攻击流在到达处置位置之前对上游多跳链路造成的资源消耗依然存在，因此星座级自治仍需要跨卫星证据汇聚与更靠近源头的前移处置。

从系统条件看，LEO 星座网络本身具备开展多星协同防御的基础。典型星座的星间连接度较低，例如 Starlink Shell 1 中每颗卫星通常配置 4 条 ISL，单条 ISL 的容量可达 20 Gb/s 量级^[24]；同时，星座拓扑和最短路结构的变化发生在秒级时间尺度上，因此可以将系统近似为一系列近似静态的拓扑快照并分时处理^[24]。此外，卫星节点已具备在数据面执行轻量计算、在星上 Agent 执行局部控制的条件^[30]。基于这些前提，本章提出一种面向大规模 LEO 的抑制型邻居扩散协同与逐跳推回机制，在不依赖地面中心控制器的前提下，实现跨卫星的攻击证据聚合以及更靠近

源头的分布式防御优化，并通过区域限制、抑制扩散和速率配额来避免大规模网络中的信令风暴。

本章主要完成了三方面的工作。首先，给出本地证据抽取、协同证据聚合和逐跳推回执行组成的多星协同框架，将上一章的本地检测能力扩展到网络级威胁判断。其次，设计面向大规模星座的抑制型邻居扩散协议，在尽量保持协同收敛速度的同时降低控制信令开销。最后，在 Starlink Shell 1 拓扑和真实流量背景（CAIDA trace）下完成系统实现与实验评估，量化展示其相对 NoDefense、SatShield 和 MS-SatShield 三类星载基线的防御收益。至于 LinkScope、Woodpecker、RADAR、Ripple 与 Mew 等地面 LFA 防御方案，本文仍将它们保留为设计参照和假设差异的讨论对象，但不把它们作为正文图表中的数值基线^[24, 30, 43–47]。

4.2 问题定义与方法设计

4.2.1 问题模型与设计目标

（1）时变星座与拓扑快照假设

本章沿用第三章中的时变星座与 epoch 建模，将 LSN 表示为动态图 $G(t) = (V, E(t))$ ，其中卫星节点集合 V 视为固定，星间链路集合 $E(t)$ 随轨道运动而变化。由于最短路径变化主要发生在秒级时间尺度^[24]，本文仍采用离散 epoch 建模：在 epoch 长度 Δ 内近似认为拓扑与路由策略不变，并将第 τ 个 epoch 对应的快照记为 G_τ 。这一假设与 ICARUS 按快照预计算攻击流分配的对手能力一致^[24]，也与上一章及 SatShield 中星上 Agent 的周期更新机制相兼容^[30]。

与上一章不同的是，本章关心证据在邻近卫星之间的传播以及缓解动作沿路径的逐跳前移，因此需要把快照假设进一步理解为“在同一 epoch 内，协同消息传播、网络级聚合和推回规则安装都基于同一个一致的拓扑视图完成”。这使得 ROI 范围、上游邻居选择和软状态传播路径都具有明确语义；至于 epoch 临界处可能出现的路由切换，则由后文的软状态生存期 T_{life} 与跨 epoch 刷新机制吸收。

（2）威胁模型与推回收益

本章沿用第三章中的基本威胁模型与 key 定义，不再重复展开 Bot-Decoy 攻击流如何构造、FI/FO 如何由攻击分摊放大等内容。换言之，攻击者仍通过选择合适的 Bot 与 Decoy 通信对，使低速流量在目标瓶颈链路 e^* 附近汇聚；设目标链路容量为 C_{e^*} ，攻击聚合速率为 A ，背景良性流量为 B ，则拥塞条件近似写为 $B + A > C_{e^*}$ 。与上一章不同，本章新增关注的是：当攻击已经被本地检测识别后，若处置仍发生在拥塞点或其附近，那么攻击流在到达处置位置之前对上游多跳链路造成的带宽

和队列消耗仍然全部存在。这部分消耗在上一章的本地队列调度框架中并未被显式度量，因为末端丢弃只能保护被攻击链路下游的资源，无法追溯释放上游已经被占用的链路容量。

因此，本章不再把“是否识别出攻击 key”作为唯一收益指标，而是进一步关心“攻击流在被处置前一共白白占用了多少路径资源”。为从网络整体资源利用的角度量化这种消耗，本文将 W_{waste} 定义为路径积分意义下的无效带宽占用成本。具体来说，对 epoch τ 内的攻击相关 key 集合 $\mathcal{A}_{\tau}^{\text{atk}}$ ，记 key a 在被首次限速或丢弃之前所经历的链路集合为 $P_{\tau}(a)$ ，其在链路 $e \in P_{\tau}(a)$ 上的聚合速率为 $x_{a,e}(\tau)$ ，则有：

$$W_{\text{waste}}(\tau) = \sum_{a \in \mathcal{A}_{\tau}^{\text{atk}}} \sum_{e \in P_{\tau}(a)} x_{a,e}(\tau), \quad (4-1a)$$

$$[W_{\text{waste}}] = \text{Mb/s} \cdot \text{hop}. \quad (4-1b)$$

该定义对每个攻击 key 执行了一次沿路径的逐跳累加：key a 以速率 $x_{a,e}(\tau)$ 经过链路 e 时，该链路就被计入一次等效占用。这种逐跳累加使得同一 key 在不同链路上的占用被独立统计，从而反映攻击流在网络内部传播过程中的整体资源消耗。与传统的单链路吞吐或丢包率指标相比， W_{waste} 能够捕捉防御位置对网络全局资源利用效率的影响。量纲由式(4-1b)给出，为速率与跳数的乘积，因此与后文图示中的 $\text{rate} \times h$ 以及柱状图使用的 $\text{Mb/s} \cdot \text{hop}$ 量纲保持一致。

若进一步假设各 key 沿路径上的速率近似恒定（即路径中间没有发生部分限速），且攻击流的平均承载跳数为 $\bar{h}$ ，则式(4-1a)可近似写为：

$$W_{\text{waste}}(\tau) \approx \bar{h} \cdot A. \quad (4-2)$$

式(4-2)揭示了推回机制降低无效带宽占用的直接因素：在攻击总聚合速率 A 短期内难以改变的前提下，若能将丢弃或限速位置从被攻击链路 e^* 前移到更靠近攻击源头的上游节点，就等效于将 $\bar{h}$ 从全路径长度缩短到前移后的残留跳数，从而按比例降低 W_{waste} 。这一关系提供了从路径长度维度理解推回收益的简洁框架。若只关心被攻击链路上游的额外浪费，可以在式(4-1a)基础上减去目标链路上的聚合速率 A ；但为了与后文图示和实验统计保持一致，本文统一采用式(4-1a)给出的全路径积分口径。

在 Starlink Shell 1 中，ISL 容量约为 20 Gb/s ^[24]，而 ICARUS 与 SatShield 都表明，攻击者在数百到数千 Bot 规模下即可形成可观的攻击流量^[24, 30]。以 $\bar{h} = 5$ 跳为例，若攻击聚合速率 A 达到 10 Gb/s ，则 W_{waste} 可达 $50 \text{ Gb/s} \cdot \text{hop}$ ，意味着有等效于 2.5 条满载 ISL 的带宽资源被攻击流无效占用。在多条 ISL 同时受到攻击或攻击路径更长的情况下，这一数值还会进一步放大，并可能引发更广泛的链路排队

与连锁拥塞。推回收益能够通过 $\bar{h}$ 的缩减直接度量，这也是本章引入多星协同与 pushback 机制的直接动机。

(3) 设计目标

在上述模型下，本章主要关注三个目标。首先是时效性，协同判断与 pushback 传播需要在攻击自适应时间尺度之前完成闭环更新，避免再次落回中心控制器的高时延反馈环^[30]。其次是可扩展性，当星座规模达到数千甚至上万颗卫星时，协同机制不能采用全网广播式信令，控制开销应远小于 ISL 带宽，并且保持数据面可实现。最后是适应性，面对分散化、低速化、脉冲化和滚动目标等攻击策略，协同机制需要把碎片化证据汇聚成网络级信号，减少本地 Top- K 漏检对整体防御效果的影响。这个问题在第三章中已经通过退化设定作过论证。

4.2.2 CoopSatShield 总体架构

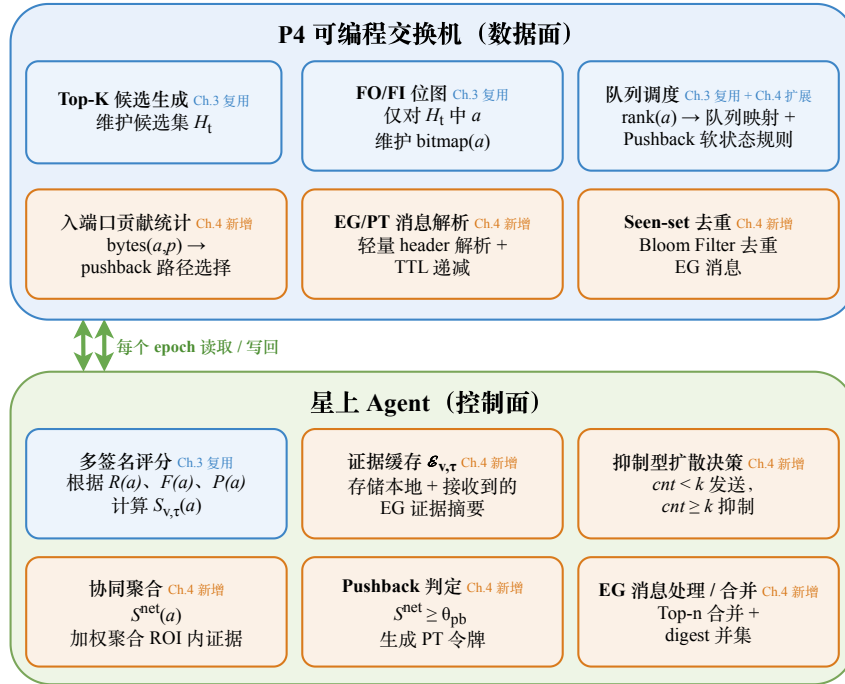


图 4-1 CoopSatShield 单节点内部架构

为实现上述目标，本章提出多星协同防御优化系统 CoopSatShield。系统由本地证据抽取（Local Evidence）、抑制型邻居扩散协同（Suppressed Gossip）和逐跳推回执行（Pushback Enforcement）三部分构成。图 4-1 展示单颗卫星节点 v 的内部架构，多节点协同拓扑将在后续图中单独给出。

图 4-1 所示的单节点内部结构分为上下两层，上层为面向 P4 可编程交换机^[63]

的数据面快路径，下层为星上 Agent（控制面，以 epoch 为周期）。

数据面包含 6 个模块，可分为复用模組与新增模組两部分：

- (1) **复用模組** (Ch.3 复用)：CMS 与候选缓存组成的一级候选生成，用于维护候选 key 集合及其近似频次；FI/FO 位图，用于记录候选 key 的扇入度或扇出度结构特征；队列调度，用于基于 rank 执行优先级队列映射与限速。
- (2) **新增模組** (Ch.4 新增)：入端口贡献统计，用于维护每个候选 key 在各入端口的字节统计 $\text{bytes}(a, p)$ ，供后续 pushback 选择上游；证据扩散消息/推回令牌解析，用于完成协同消息的轻量级头部解析与生存时间 (Time-To-Live, TTL) 递减；Seen-set 去重，用 Bloom Filter^[64] 实现 $O(1)$ 消息去重。

上述模块在设计语义上都可以映射到可编程交换机的寄存器阵列与 Match-Action 表^[61]，每包处理保持 $O(1)$ 复杂度。

控制面 Agent 包含 6 个模块：

- (1) 多签名评分 (Ch.3 复用)，计算候选 key 的多维可疑性评分；
- (2) 证据缓存 $\mathcal{E}_{v,\tau}$ ，存储感兴趣区域内接收到的 EG 消息；
- (3) 抑制型扩散决策，执行冗余抑制判断 ($\text{cnt} \geq k$ 时抑制发送)；
- (4) 协同聚合 S^{net} ，计算网络级加权评分；
- (5) Pushback 判定，比较 S^{net} 与阈值 Θ_{pb} 并触发推回；
- (6) EG 消息处理与合并，执行聚合操作并构造聚合消息。

两层之间通过 epoch 读取和写回接口连接。Agent 每个 epoch 从 CMS/候选缓存摘要、FI/FO 位图以及入端口贡献表中拉取数据，计算多签名评分后，再将 rank 及推回规则写回数据面。

本节复用第三章的本地多签名检测流水线，并将其输出进一步组织为可协同处理的证据单元。对协同计算而言，节点输出不仅要给出判定结果，还需要便于聚合、比较和传播抑制。基于这一要求，本节定义如下证据结构。

对卫星 v 在 epoch τ 内的某条出端口链路 $e = (v, u)$ ，定义其拥塞指示量为：

$$c_{v,e}(\tau) = \mathbf{1}[q_{v,e}(\tau) > Q_{\text{th}} \vee d_{v,e}(\tau) > D_{\text{th}}], \quad (4-3)$$

其中 $q_{v,e}(\tau)$ 表示该端口的队列占用（或排队时延估计）， $d_{v,e}(\tau)$ 表示端口丢包率， Q_{th} 和 D_{th} 为阈值。该指标直接来自数据面或交换机 traffic manager 的可观测量，符合星上实时性要求。为了便于后续权重计算，本章进一步定义归一化的拥塞等级：

$$\text{cong_level}_{v,e}(\tau) = \min\left(1, \max\left\{\frac{q_{v,e}(\tau)}{Q_{\text{th}}}, \frac{d_{v,e}(\tau)}{D_{\text{th}}}\right\}\right) \in [0, 1], \quad (4-4)$$

其中 Q_{th} 和 D_{th} 直接继承第三章的数据面拥塞告警配置，本章只复用其部署阈值，不把它们作为新的调参维度；若节点在同一 epoch 内观测到多条拥塞链路，则在

表 4-1 Evidence Gossip (EG) 消息字段设计

字段	含义
type	消息类型标识 (4-bit), 用于区分 EG 与 PT。
epoch	当前 epoch 编号, 用于软状态一致性与过期清理。
link_id	触发拥塞的链路标识 (如 (v, u) 或端口号)。
ttr	剩余传播跳数 (ROI 半径 R)。
nonce	消息唯一标识, 用于去重缓存 (Seen-set)。
digest	候选 key 集合摘要, 用于快速交集判断。
$\langle \hat{a}_i, \hat{s}_i \rangle_{i=1}^n$	Top- n 可疑 key 的哈希表示与量化评分。
cong_level	按式(4-4)归一化得到的拥塞等级。

表 4-2 Pushback Token (PT) 消息字段设计

字段	含义
type	消息类型标识 (4-bit), 用于区分 EG 与 PT。
epoch	触发 epoch 编号, 用于软状态一致性。
$R_{\uparrow}$	剩余推回跳数 (4-bit), 每跳递减直至 0 终止。
key_hash	目标 key 的 CRC32 短哈希 $\hat{a}$ (32-bit)。
rank	缓解优先级 (8-bit), 映射到队列与限速。
T_{life}	软状态生存期 (8-bit) 以 epoch 为单位, 到期自动清除。

式(4-8)中取相应 $\text{cong_level}_{v,e}(\tau)$ 的最大值作为节点级拥塞强度。

当 $c_{v,e}(\tau) = 1$ 时, 节点 v 从本地候选集 $\mathcal{H}_{v,\tau}$ 中提取与链路 e 最相关的 Top- n 可疑 key 集合 $\mathcal{A}_{v,e,\tau}$, 并为每个 $a \in \mathcal{A}_{v,e,\tau}$ 给出本地多签名评分 $S_{v,\tau}(a)$ (见第三章式 (3-11))。与此同时, 还输出该 key 在不同入端口上的贡献分布 $\pi_{v,\tau}(a)$, 供后续 pushback 选择上游:

$$\pi_{v,\tau}(a, p) = \frac{\text{bytes}_{v,\tau}(a, p)}{\sum_{p'} \text{bytes}_{v,\tau}(a, p')}, \quad (4-5)$$

其中 p 为 v 的入端口编号, $\text{bytes}_{v,\tau}(a, p)$ 为 epoch 内从端口 p 进入且匹配 key a 的字节数。由于 LEO 星间连接度较低 (典型为 4 条 ISL 加地面链路) [24], 这一分布统计可以在有限状态下实现, 也能直接用于识别攻击流主要来自哪个上游邻居。

4.2.3 抑制型邻居扩散协同机制

(1) 扩散抑制的动机

在大规模星座中, 朴素 Gossip 的做法是每个 epoch 内每个节点都向所有邻居发送摘要并继续转发, 这会使控制信令随网络规模快速增长。即使单条消息本身

很小，当 $|V|$ 达到数千时，持续扩散仍会带来不必要的链路占用和处理负担，更严重的是会在拥塞期间与数据业务争抢资源，出现防御信令反过来挤压业务的情况。LEO 网络的一个重要特征在于，攻击影响通常集中在被攻击链路及其上游汇聚路径附近，因此协同传播也应局部化、事件驱动，并且能够抑制重复传播。

本章用三种约束来控制信令扩散。首先采用事件驱动，只有当 $c_{v,e}(\tau) = 1$ 或接收到新的证据时才触发发送；其次采用区域限制，让每条证据消息携带 TTL，最多传播 R 跳，从而形成受影响区域 ROI；最后采用抑制扩散，借鉴传感网中的冗余抑制思想^[67]，当节点在一个发送周期内已经收到足够多重证据时，就自动抑制自身转发，避免扩散规模持续膨胀。这里的实现接近 Trickle 协议^[68] 的抑制思想，但并不沿用其倍增窗口机制，而是采用固定时隙 I 来简化星上实现。

(2) 协同消息类型与格式

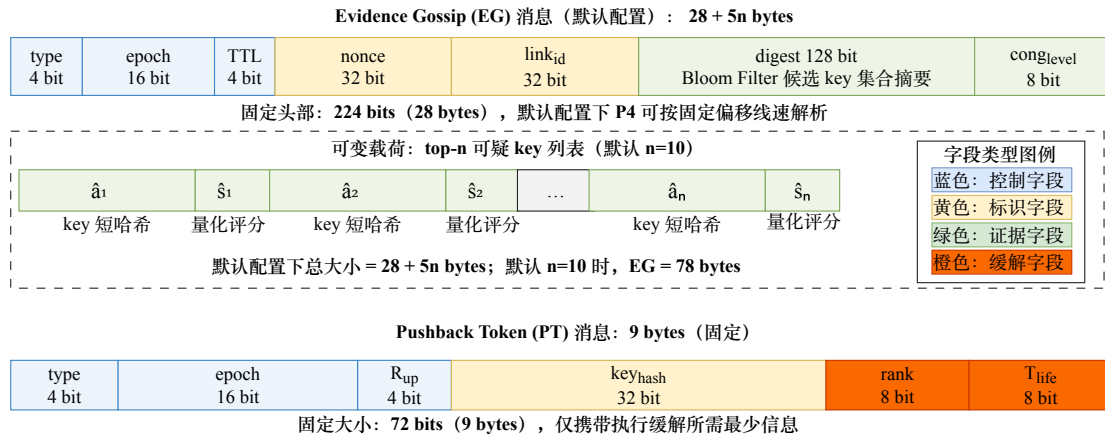


图 4-2 EG/PT 协同控制消息格式

本章定义两类星间控制消息：证据扩散消息（Evidence Gossip, EG）与推回令牌（Pushback Token, PT）。其中 EG 负责传播与聚合证据，PT 负责执行缓解推回。表 4-1 给出 EG 消息字段。

其中 $\hat{a}_i$ 为 key 的 CRC32 短哈希（32-bit），在实际 key 空间中碰撞率很低，同时能够节省带宽； $\hat{s}_i$ 为量化评分（8-bit，线性映射 $[0, 1]$ 至 $[0, 255]$ ），分辨率 $1/256$ 已足够完成排序和阈值判断。默认配置下，digest 采用 128-bit Bloom Filter^[64] 编码候选 key 集合，接收端可以用 $O(1)$ 代价判断本地候选是否与对方证据存在重叠，也就是对式(4-6)中 Ω 进行快速近似，从而避免对无关证据执行高代价合并。该默认配置对应图中的消息格式与大小统计。

图 4-2 用 bit-level 字段图展示了两类协同控制消息在默认配置下的具体格式。EG 消息的固定头部由 type（4-bit）、epoch（16-bit）、ttl（4-bit）、nonce（32-bit）、

link_id (32-bit)、digest (128-bit) 和 cong_level (8-bit) 组成, 共 224 比特, 即 28 字节, 可在 P4 交换机 parser 中以固定偏移完成线速解析; 可变载荷部分每个 entry 占 5 字节 ($\hat{a}_i$ 32-bit + $\hat{s}_i$ 8-bit), 因此 EG 消息总大小为 $28 + 5n$ 字节, 在默认 $n = 10$ 时为 78 字节。PT 消息更紧凑, 固定仅 72 bit (9 字节), 字段顺序为 type、epoch、 $R_{\uparrow}$ 、key_hash、rank 与 T_{life} , 只携带执行缓解所需的最少信息。图中的颜色用于区分控制字段、标识字段和证据/缓解字段, 便于读者从字段功能而非协议流程理解消息结构。

算法 4-1: 抑制型邻居扩散协同

输入:

- (1) 当前 epoch τ 的本地拥塞状态 $c_{v,e}(\tau)$ 与本地 EG 候选消息 M_v ;
- (2) 节点状态: 证据缓存 $\mathcal{E}_{v,\tau}$ 与发送窗口 I ;
- (3) 协同参数: ROI 半径 R 、抑制参数 k 、重叠阈值 θ_{Ω} 。

输出: 节点本轮的发送结果 O_v : 聚合 EG 消息 $\tilde{M}_v$ 或抑制决策 SUPPRESS。

```

/* 本地拥塞触发并启动抑制发送窗口 */
1 初始化发送结果  $O_v \leftarrow \text{SUPPRESS}$ ;
2 若存在链路  $e$  使  $c_{v,e}(\tau) = 1$ , 构造本地 EG 候选消息  $M_v$ ;
3 将  $M_v$  加入缓存  $\mathcal{E}_{v,\tau}$ , 并进入抑制发送窗口  $I$ ;
4 在窗口  $I$  内随机选择  $t_{\text{send}}$ ;

/* 监听邻居消息并合并等价证据 */
5 while  $t < t_{\text{send}}$  do
6   监听邻居 EG 消息  $M_u$ ;
7   if  $M_u$  未去重 (nonce 未见过) 且  $M_u.\text{ttl} > 0$  then
8     计算  $\Omega(M_u, v)$  (可由 digest 快速近似);
9     if  $\Omega(M_u, v) \geq \theta_{\Omega}$  or 本地存在拥塞链路 then
10      将  $M_u$  合并入  $\mathcal{E}_{v,\tau}$  (Top- $n$  合并 + digest 并集);
11 统计窗口  $I$  内等价证据数量  $\text{cnt}$ ;

/* 根据冗余程度决定转发或抑制 */
12 if  $\text{cnt} < k$  then
13   生成聚合消息  $\tilde{M}_v = \text{Aggregate}(\mathcal{E}_{v,\tau})$ ;
14   设置  $\tilde{M}_v.\text{ttl} \leftarrow \min(\text{ttl}_{\text{received}} - 1, R)$ , 向所有邻居发送 (不回发原入端口);
15    $O_v \leftarrow \tilde{M}_v$ ;
16 else
17   抑制发送 (suppress);
18 return  $O_v$ .
```

EG 与 PT 在载荷设计上的差异对应协同流程的两个阶段。EG 面向证据传播, 需要同时携带候选集合摘要、评分和拥塞上下文, 以支持相似性过滤与多源合并; PT 面向动作执行, 只保留 key 标识、缓解等级和生存期等执行字段, 以缩短控制

反馈路径并压低额外开销。这样设计后，证据阶段传递的是可比较、可聚合的信息，缓解阶段传递的是可直接下发的数据面控制信息。

(3) 抑制型扩散协议 (Suppressed Gossip Protocol)

设节点 v 在 epoch τ 内维护一个证据缓存 $\mathcal{E}_{v,\tau}$ ，存储 ROI 内观察到的 EG 消息摘要。对任意 $(\tau, \text{link_id})$ ，节点以固定时隙 $I = \Delta/4$ 执行一次抑制型扩散，即每个 epoch 内最多执行 4 轮扩散，且 $I \ll \Delta$ 足以保证在 epoch 结束前完成收敛。其基本过程是：节点先在 I 内随机选择一个发送时刻 t_{send} ；若在 t_{send} 之前已经监听到至少 k 条等价证据（同一 epoch、同一 link_id 且 digest 高度相似），则抑制本次发送；否则就在 t_{send} 发送聚合后的 EG 消息。通过随机化发送时机和冗余抑制，系统可以在保持传播速度的同时减少重复广播。

进一步地，为保证传播局部化，本章规定 EG 消息只有在满足相关性门限时才会继续转发。节点 v 收到来自邻居 u 的 EG 消息 M 后，计算其与本地候选集合 $\mathcal{A}_{v,\tau}$ 的重叠度：

$$\Omega(M, v) = \frac{|\mathcal{A}_{v,\tau} \cap \mathcal{A}(M)|}{|\mathcal{A}(M)|}, \quad (4-6)$$

其中 $\mathcal{A}(M)$ 表示消息中的 Top- n key 集合。只有当 $\Omega(M, v) \geq \theta_\Omega$ 或本地链路同样处于拥塞态 $c_{v,e}(\tau) = 1$ 时，节点才会将该证据纳入缓存并参与后续转发。这个机制借助本地观测的一致性过滤无关扩散，是控制信令规模的重要一环。

算法 4-1 给出了抑制型扩散协议的流程。与朴素 Gossip 相比，这里的重点不在于能否传播，而在于结合物理网络的局部性和低度数结构，把传播范围控制在 ROI 内，并尽量避免同一证据在区域内反复回旋。

函数 $\text{Aggregate}(\mathcal{E}_{v,\tau})$ 对缓存中的所有 EG 消息执行如下合并：(i) 同一 key a 若出现在多条消息中，则取 $\hat{s}(a) = \max_j \hat{s}_j(a)$ ，保留最强证据；(ii) 对 digest 按位 OR，得到 Bloom Filter 摘要并集，保证接收端能够判断聚合后候选集的覆盖范围；(iii) 若合并后的 key 集合超过 n ，则按 $\hat{s}(a)$ 降序保留前 n 个，保证消息大小恒定。这样处理后，消息仍然紧凑，同时优先传播高置信度证据。

算法 4-1 中的聚合发送步骤采用 $\text{ttl} \leftarrow \min(\text{ttl}_{\text{received}} - 1, R)$ ，从而保证每次转发时 TTL 至少递减 1，即使经过聚合节点也不会被重置为初始 TTL。因此，在 R 跳半径的 ROI 内，消息最多经历 R 次转发后就会终止，不存在无限传播风险。

在一个 epoch 内，节点对同一 $(\tau, \text{link_id})$ 的证据缓存 $\mathcal{E}_{v,\tau}$ 具有单调合并性。随着窗口内邻居消息到达，缓存中的候选集合只会扩展覆盖范围，或提高已有 key 的评分上界，不会因后续消息到来而撤销既有高置信证据。因此，即使相邻节点选择的随机发送时刻并不一致，各节点得到的聚合结果仍会逐步收敛到一组相近

高分 key。再结合基于 nonce 的去重和基于 epoch 的过期清理，重复接收带来的额外处理停留在常数级查表、少量缓存更新以及 Top- n 维护上，不会引入随传播轮次增长的状态负担。

参数 k 与 θ_Ω 分别刻画冗余抑制强度和相关性过滤强度，是协议行为的两个主要控制量。较小的 k 会使节点在窗口内更倾向于继续转发，从而加快扩散和局部收敛，但也会增加重复广播；较大的 k 则可进一步压缩控制开销，不过在证据较稀疏、拓扑分界较清楚的区域，可能降低相关证据的传播概率。 θ_Ω 越高，节点只接受与本地候选集合重叠度更高的消息，扩散范围更集中于受影响区域附近；适度降低 θ_Ω 则有助于跨越局部观测差异，吸收弱相关但仍有价值的证据。二者共同决定传播速度、空间局部性与信令规模之间的折中，也为后续实验中的参数整定提供依据。

4.2.4 协同判定与推回机制

(1) 协同证据聚合与判定

在 ROI 内，多个节点对同一攻击相关 key 会给出不同程度的本地评分 $S_{v,\tau}(a)$ 。为形成网络级判定，本章定义协同聚合评分：

$$S_\tau^{\text{net}}(a) = \sum_{v \in \mathcal{R}_\tau} \omega_v \cdot \tilde{S}_{v,\tau}(a), \quad (4-7)$$

其中 $\mathcal{R}_\tau$ 为在 epoch τ 内参与协同的 ROI 节点集合， $\tilde{S}_{v,\tau}(a)$ 为归一化后的本地评分， ω_v 为权重。权重的设计根植于物理可观测测量：拥塞越强、越接近被攻击链路的节点，其证据应更可靠。于是本章取：

$$\omega_v = \frac{(1 + \lambda \cdot \text{cong_level}_v) \cdot \exp(-\mu \cdot \text{dist}(v, e^*))}{\sum_{u \in \mathcal{R}_\tau} (1 + \lambda \cdot \text{cong_level}_u) \cdot \exp(-\mu \cdot \text{dist}(u, e^*))}, \quad (4-8)$$

其中 $\text{dist}(v, e^*)$ 表示 v 到被攻击链路 e^* 的跳数距离， λ 和 μ 为超参数，且所有参与节点的权重满足 $\sum_{v \in \mathcal{R}_\tau} \omega_v = 1$ 。 λ 控制拥塞等级对权重的放大程度， μ 控制距离衰减速率。实现时，为避免复杂指数运算，控制面 Agent 采用查表或分段线性近似完成权重计算；数据面只需提供 cong_level 与邻接关系。式(4-7)在本章中的主要用途是让 pushback 触发更平滑。它通过汇总 ROI 内的局部证据来压制单节点的瞬时误判，避免某处偶发异常立刻触发全区域推回。在后文的均匀 Bot 分布实验中，CoopSatShield 与 MS-SatShield 的检测 F1 保持一致，因此本章的证据更支持把它理解为网络级触发融合机制，而不是证明加权聚合在检测能力上一定优于简单的 max 或 avg 规则。

推回判定阈值 Θ_{pb} 的选取遵循如下工程原则：在预设验证场景下，控制误推

回率（即良性 key 被触发推回的概率）不超过 5%，并在此约束下选择一组可工作的触发点。后续实验统一采用 $\Theta_{pb} = 0.7$ 、 $\lambda = 1.0$ 、 $\mu = 0.5$ 。由于本章尚未单列 weighted / unweighted / max 三类聚合规则的消融对比，这些参数应理解为可工作的默认配置，而非全局最优解。

当存在 a 使 $S_{\tau}^{net}(a) \geq \Theta_{pb}$ 时，系统对该 key 触发逐跳推回；若多个 key 满足条件，则按 $S_{\tau}^{net}(a)$ 排序取前 n_{pb} 个执行，以保证数据面策略表规模可控。

（2）推回令牌与逐跳传播

算法 4-2: 逐跳推回 (Hop-by-hop Pushback)

输入:

- (1) 触发 key 集合 $\mathcal{A}_{\tau}^{pb}$ 及其缓解等级 $\text{rank}(a)$;
- (2) 各节点的入端口贡献分布 $\pi_{v,\tau}(a, p)$ 与本地软状态规则表 $\mathcal{R}^{pb}$;
- (3) 推回参数: 推回半径 $R_{\uparrow}$ 、上游分支数 r 、软状态生存期 T_{life} 。

输出: 更新后的本地软状态规则表 $\mathcal{R}^{pb}$ ，以及继续向上游传播的 PT 令牌集合 $\mathcal{T}^{\uparrow}$ 。

```

/* 被攻击节点按入端口贡献选择上游并发起 PT */
1 初始化待转发 PT 集合  $\mathcal{T}^{\uparrow} \leftarrow \emptyset$ ;
2 foreach  $a \in \mathcal{A}_{\tau}^{pb}$  do
3   在节点  $v$  生成 PT 令牌:  $\langle a, \text{rank}, \tau, R_{\uparrow}, T_{life} \rangle$ ;
4   计算上游贡献分布  $\pi_{v,\tau}(a, p)$ ，选取前  $r$  大端口  $\mathcal{P}_v^{\uparrow}(a)$ ;
5   向  $\mathcal{P}_v^{\uparrow}(a)$  对应邻居发送 PT;
6   将对应 PT 记入  $\mathcal{T}^{\uparrow}$ ;

/* 上游节点安装软状态并递归继续推回 */
7 if PT 未过期且  $R_{\uparrow} > 0$  then
8   在本地软状态规则表  $\mathcal{R}^{pb}$  中安装/刷新  $a$  的缓解项:  $\mathcal{R}^{pb}[a].\text{rank} \leftarrow \text{PT.rank}$ ，有效期  $T_{life}$ ;
9    $R_{\uparrow} \leftarrow R_{\uparrow} - 1$ ;
10  基于  $\pi_{u,\tau}(a, \cdot)$  继续向上游前  $r$  大邻居传播;
11  将更新后的 PT 记入  $\mathcal{T}^{\uparrow}$ ;
12 else
13   丢弃该 PT;
14 return  $\mathcal{R}^{pb}$  和  $\mathcal{T}^{\uparrow}$ 。
    
```

pushback 的重点在于，它不依赖全局路由逆推，而是利用逐跳可观测的上游贡献来实现近似反向路径传播。具体来说，被攻击链路附近节点 v 对 key a 计算其入端口贡献分布 $\pi_{v,\tau}(a, p)$ （式(4-5)），再选择贡献最大的前 $r = \min(2, d - 1)$ 个上游端口集合 $\mathcal{P}_v^{\uparrow}(a)$ ；在 $d = 4$ 的 LEO 星间链路结构中， $r = 2$ 。随后，节点向对应上游邻居发送 PT 令牌。令牌携带软状态生存期 T_{life} 与剩余推回跳数 $R_{\uparrow}$ ，上游节点收到后立即在本地数据面安装短时效缓解规则，如队列降权或限速，并继续沿

相同思路向更上游传播。记本地推回软状态规则表为 $\mathcal{R}^{pb}$ 。由于 LEO 网络连接度较低、拓扑变化发生在秒级^[24]，这种逐跳机制在每个 epoch 内都具有统一语义；同时，软状态到期后会自动失效，从而避免拓扑变化后留下过期规则。算法 4-2 给出了 pushback 传播流程。

PT 中的 rank 字段映射到三级缓解强度：(i) rank = High：匹配 key 的流量被调度至最低优先级队列 Queue 0，带宽上限设为链路容量的 5%，实现基本阻断；(ii) rank = Med：调度至 Queue 1，带宽上限为 20%；(iii) rank = Low：调度至 Queue 2，仅降低调度优先级但不硬限速。数据面通过 match-action 表项实现：以 key 哈希 $\hat{a}$ 匹配入向流量，命中后修改队列标记字段并施加对应 meter 速率。当同一 key 在同一 epoch 内收到多个 PT（来自不同下游节点或路径），取最严格的 rank 值（即 High > Med > Low）；若新 PT 的 rank 弱于已安装规则，则忽略。跨 epoch 时，新 PT 刷新 T_{life} 计时器，确保软状态的时效性。

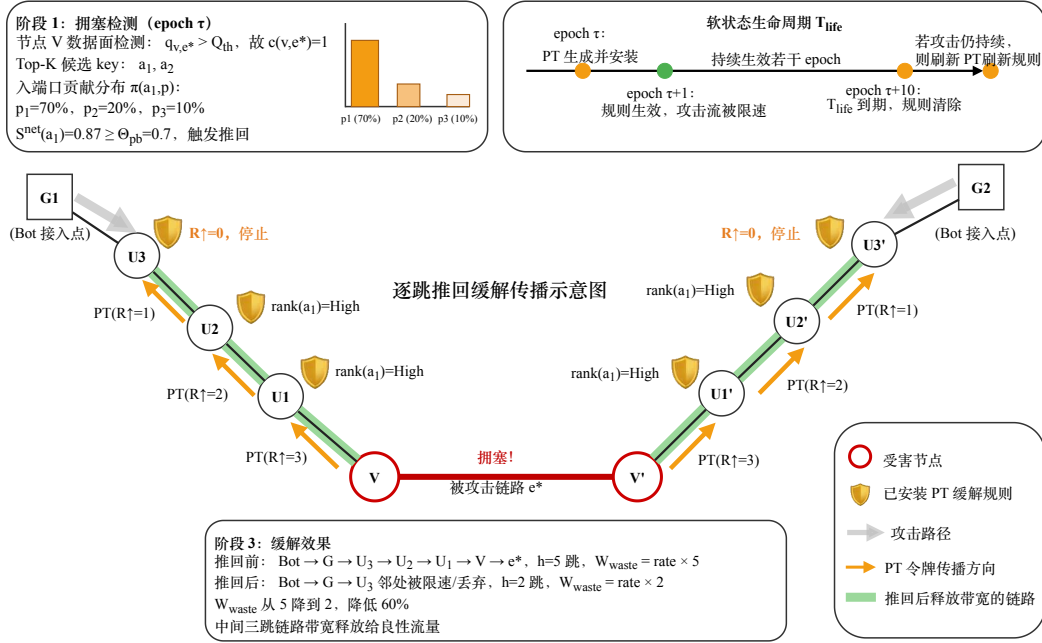


图 4-3 逐跳推回缓解传播示意

图 4-3 直观展示了逐跳推回的主要过程，并额外给出了软状态生命周期时间线。图中的被攻击链路 e^* 由两个端点节点 V 与 V' 构成，示意推回可以沿链路两侧各自的上游方向并行传播。

阶段 1 (图左上角) 展示被攻击节点 V 的拥塞检测过程：数据面检测到被攻击链路 e^* 的队列占用超过阈值 ($q_{v,e^*} > Q_{th}$ ，即 $c(v,e^*) = 1$)，并输出 Top-K 候选 key 的入端口贡献分布 $\pi(a_1, p)$ 。图中小型条形图显示 key a_1 在三个入端口的贡献比例分别为 70%、20% 和 10%，指明主攻击方向。当协同聚合评分 $S^{net}(a_1) = 0.87 \geq \Theta_{pb} = 0.7$

时，系统触发推回。

阶段 2（图中部）展示 PT 令牌的逐跳反向传播。为与被攻击链路 $e^* = (V, V')$ 的双端结构对应，图中给出了沿链路两侧并行推进的两条上游分支：左侧为 $V \rightarrow U_1 \rightarrow U_2 \rightarrow U_3$ ，右侧为 $V' \rightarrow U'_1 \rightarrow U'_2 \rightarrow U'_3$ 。在每一侧，受害链路端点生成携带 $\text{rank}(a_1) = \text{High}$ 的 PT，并使剩余推回跳数 $R_{\uparrow}$ 按 $3 \rightarrow 2 \rightarrow 1 \rightarrow 0$ 逐跳递减；各接收节点在本地数据面安装或刷新对应的软状态缓解规则，直至 $R_{\uparrow} = 0$ 时停止继续传播。图中盾牌图标标注了已安装 PT 规则的节点，绿色高亮链路表示攻击流被更早抑制后释放出的中间链路带宽。

阶段 3（图下方）定量对比推回前后的效果：按本文统一采用的路径积分无效占用成本口径，推回前攻击流沿一侧入口路径经历 $h = 5$ 跳后到达 e^* ，因此 $W_{\text{waste}} = \text{rate} \times 5$ ；推回后，攻击流已在更靠近入口的 U_3/U'_3 邻近位置被限速或丢弃，仅经历 $h = 2$ 跳，故 $W_{\text{waste}} = \text{rate} \times 2$ ，对应图中给出的 60% 降幅。图右上角的时间线则补充说明软状态生命周期 T_{life} ：PT 在 epoch τ 生成并安装，epoch $\tau + 1$ 开始生效并持续若干 epoch，在 epoch $\tau + 10$ 到期清除；若攻击仍持续，则新的 PT 会刷新规则，避免拓扑变化后残留过期状态。

（3）动态拓扑下的稳定性与误伤控制

需要说明的是，CoopSatShield 在动态 LEO 拓扑中并不追求强一致性判定，而是采用基于 epoch 的软一致性机制。EG 与 PT 消息均携带 epoch 标识，节点只接受当前或相邻 epoch 内的协同消息，过期消息直接丢弃。协同证据缓存以 epoch 为单位清理，推回规则也以软状态形式安装，并由生存期 T_{life} 控制自动过期。因此，当路由切换导致入端口贡献分布发生变化时，旧规则不会长期保留，而会在后续 epoch 中被新的观测结果刷新或清除。

基于入端口贡献的推回方向识别也不是独立的攻击判定依据。入端口贡献 $\pi_{v,\tau}(a, p)$ 只用于在已经触发协同判定之后选择上游传播方向；是否触发推回仍需同时满足本地拥塞、多签名可疑度、协同评分和推回阈值等条件。这样可以降低正常热点流量仅因从某个入端口汇聚而被直接推回的风险。对于负载波动或正常重流汇聚，系统通过较高的推回阈值 Θ_{pb} 、有限推回半径 $R_{\uparrow}$ 、上游分支数 r 以及软状态生存期共同限制误伤范围。

为避免规则在阈值附近频繁安装和撤销，推回规则采用短时软状态刷新机制：新的 PT 只会刷新或加强已有规则，过期规则由 T_{life} 自动清理；当同一 key 收到多个 PT 时，取更严格的 rank，而不是反复在多个等级之间切换。若后续工程部署中观测到明显抖动，还可以进一步采用双阈值迟滞策略，即使用 Θ_{on} 触发推回、使用较低的 Θ_{off} 解除推回。本文实验采用单阈值配置，主要验证协同推回的收益和开

销，未将迟滞机制作为独立优化项展开。

因此，本文结论应理解为在 epoch 内拓扑和路由近似稳定条件下的协同缓解收益。若路由在一个 epoch 内频繁切换，或正常热点流量与攻击流量在速率、Fan-in 和路径方向上高度重叠，仍可能出现短时误推回或规则抖动。这类情况属于本文方法的适用边界，后续可通过更细粒度的路由变化实验和误推回率评估进一步分析。

4.2.5 信令开销与可实现性分析

(1) ROI 限制下的信令规模上界

设星座网络的平均度数为 d （典型 LEO 星间链路结构中 $d \approx 4$ ）^[24]。对任意 EG 消息，TTL 为 R 时，其在最坏情况下可触达的节点数不超过一个 d 叉树的 R 层节点规模：

$$|\text{ROI}(R)| \leq 1 + d \cdot \frac{(d-1)^R - 1}{d-2}, \quad d > 2. \quad (4-9)$$

当 $d = 4$ 时，上界为 $|\text{ROI}(R)| \leq 2 \cdot 3^R - 1$ 。例如 $R = 4$ 时，最坏触达规模不超过 161 个节点，远小于 Starlink Shell 1 的 1584 节点总规模^[24]。因此，TTL 本身已经把协同传播限制在受影响的局部区域内，避免全网泛洪。

(2) 抑制机制下的消息量与带宽开销

在抑制型扩散中，节点在一个发送窗口内若监听到不少于 k 条等价证据即抑制发送，从而减少重复广播。对单个协同事件 $(\tau, \text{link_id})$ 而言，设 ROI 内有效参与节点数为 N_R ，且每个节点在该事件上每 epoch 至多发送一次聚合 EG 消息，则 EG 消息总数满足：

$$M_{\text{EG}}(\tau, \text{link_id}) \leq N_R. \quad (4-10)$$

进一步地，若该事件对应的 EG 与 PT 消息大小分别为 L_{EG} 、 L_{PT} 字节，PT 消息数为 $M_{\text{PT}}(\tau, \text{link_id})$ ，则其单事件控制带宽开销上界为：

$$B_{\text{ctrl}}^{(\tau, \text{link_id})} \leq \frac{M_{\text{EG}}(\tau, \text{link_id}) \cdot L_{\text{EG}} + M_{\text{PT}}(\tau, \text{link_id}) \cdot L_{\text{PT}}}{\Delta}. \quad (4-11)$$

若同一 epoch 内同时出现多条独立拥塞链路，则总控制开销可写为对所有事件的求和。结合式(4-9)与 Starlink ISL 20 Gb/s 容量级^[24]，当 R 取小常数（本章默认 $R = 4$ ）时， B_{ctrl} 相对 ISL 带宽占比可控制在远低于 1% 的量级。后续实验进一步验证了控制开销与 R 、 k 、 Δ 的敏感性（见第4.3.2节），并观察到抑制机制使得在攻击高峰期也不会出现信令风暴。

(3) 数据面实现可行性

实现上，本章遵循数据面负责常数复杂度快路径、控制面负责聚合与决策的分工。数据面新增逻辑主要包括协同消息的轻量 header 解析与 TTL 递减、Seen-set 去重（可由 Bloom Filter 近似实现）、key 的入端口贡献统计 $\text{bytes}_{v,\tau}(a,p)$ ，以及 pushback 规则的执行与软状态清理。这些操作都可以由可编程交换机的寄存器阵列与 match-action 表实现^[61]，每包处理仍保持 $O(1)$ 复杂度，满足线速约束。复杂的聚合评分和抑制决策则交由星上 Agent 完成，避免数据面承担过重计算负担。

4.3 仿真与分析

4.3.1 实验设置

本章的仿真实验基于与第三章相同的仿真框架实现。该框架集成了星座拓扑构建、最短路计算、epoch 级事件驱动与数据面行为模拟，拓扑构建与路由策略参考 Hypatia 仿真平台^[60]的快照路由模型^[8]。评估采用 Starlink Shell 1 拓扑，该拓扑包含 1584 颗卫星（72 轨道面 $\times$ 22 颗/面），ISL 为典型的 4 链路连接结构，ISL 带宽 20 Gb/s，上下行链路容量为数 Gb/s 量级^[24]。epoch 长度取 $\Delta = 1\text{ s}$ ，与卫星拓扑快照更新周期对齐^[24]。表 4-3 汇总了本章实验使用的主要参数。为贴近真实网络背景流量，本文采用 CAIDA UCSD Anonymized Internet Traces 2018 数据集（OC-192 链路，采集于 Equinix-Chicago 监测点）构造良性流量：原始 trace 按比例缩放至 20 Gb/s ISL 容量级别，并按人口加权的城市对分布模型将源-目的 IP 映射到 LEO 地面终端对，映射参数与第三章中的设置一致^[30]。攻击流量则采用 ICARUS 式 LFA 策略生成，攻击者利用公开轨道与拓扑信息选择 Bot-Decoy 对并计算使目标链路拥塞的流量分配^[24]；同时，在更强的对手模型下引入低速化、多 Decoy、脉冲化和滚动目标等策略，用来检验协同机制对碎片化证据的聚合能力。

为避免实验设置中声明的指标多于正文实际汇报的指标，本章只统计与后续图表直接对应的四类结果：路径积分无效带宽占用成本 W_{waste} ；攻击期间的良性吞吐，以及两类响应时延，本地检测生效时延 $T_{\text{det}} = t_{\text{first-local-rule}} - t_{\text{attack-start}}$ 和 pushback 生效时延 $T_{\text{pb}} = t_{\text{first-pushback-rule}} - t_{\text{attack-start}}$ ；检测质量，即攻击相关 key 识别的 Precision、Recall 和 F1，定义同第三章式 (3-19a)–(3-19c)；以及协同开销，即 EG/PT 消息数与字节开销。除非特别说明，下文提到的吞吐都指攻击持续阶段的 epoch 平均值，不包含 warmup 与恢复阶段。

表 4-3 本章实验主要参数设置

参数	含义	取值
Δ	epoch 长度	1 s
I	抑制扩散时隙	$\Delta/4 = 0.25$ s
R	ROI 半径 (EG TTL)	4 跳
$R_{\uparrow}$	推回半径 (PT 跳数)	3 跳
k	抑制参数	3
n	EG 携带的 Top- n key 数	10 (默认)
r	推回上游分支数	$\min(2, d - 1) = 2$
θ_{Ω}	相关性门限	0.3
Q_{th}, D_{th}	拥塞告警阈值	继承第三章部署配置
Θ_{pb}	推回判定阈值	0.7
λ	拥塞权重放大因子	1.0
μ	距离衰减因子	0.5
T_{life}	PT 软状态生存期	10 epoch

表 4-4 量化对比基线设置

方案	说明
NoDefense	无防御, 先进先出/等价多路径默认转发。
SatShield	单星 Top- K 过滤 + 队列调度缓解 ^[30] 。
MS-SatShield	第三章多签名增强, 但无协同、无 pushback。
CoopSatShield	本章方案: 抑制型扩散协同 + 逐跳 pushback。

4.3.2 结果与分析

本节围绕所提协同防御机制的实际表现展开分析, 分别从网络级带宽收益、良性吞吐恢复、协同控制开销以及碎片化攻击下的表现四个方面进行评估。

(1) 逐跳推回带来的网络级收益

图 4-4 以分组柱状图展示了在不同碎片化程度 $M \in \{1, 10, 100, 1000\}$ 下, 各方案对路径积分无效带宽占用成本 W_{waste} 的影响。实验结果可以分为三种情形来看。

当 $M \leq 10$ 时, SatShield 与 MS-SatShield 表现接近, 均可将 W_{waste} 降至 NoDefense 的 7.2% ($M = 1$) 和 19.8% ($M = 10$)。这是因为此时攻击 key 的聚合速率远高于良性长尾, Rate-only 已能检出, 多签名不会带来额外增益。CoopSatShield 则通过推回把丢弃位置继续前移到上游 $R_{\uparrow} = 3$ 跳附近, 使 W_{waste} 在 $M = 1$ 时进一步降到 3.0%。这里的收益主要来自 pushback 缩短攻击流实际承载路径, 而不是协同证据聚合; 在 $M = 1$ 时, 单节点已经足以检出唯一的攻击 key。这说明即使

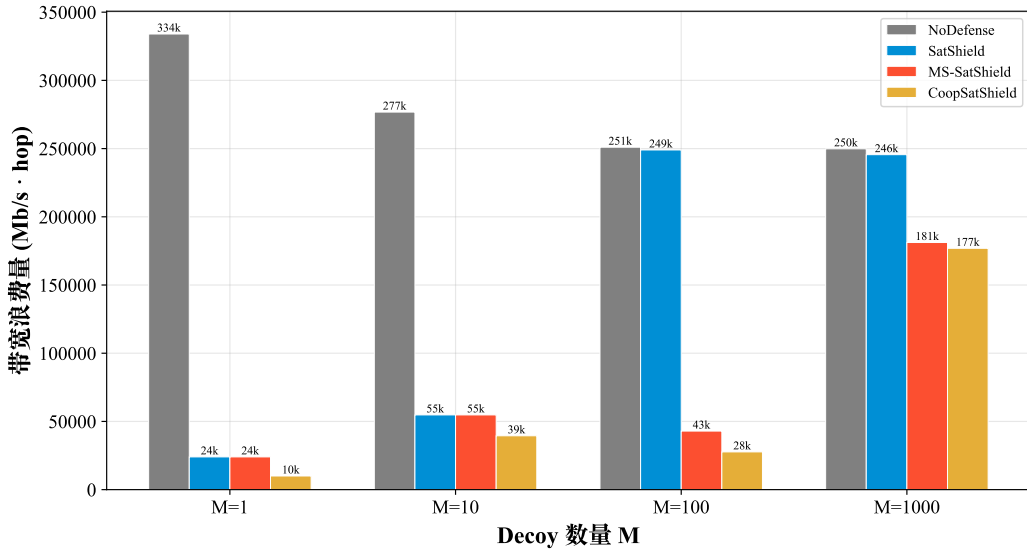


图 4-4 不同碎片化程度下的无效带宽占用对比

不存在强碎片化，pushback 本身也具有独立的带宽节约价值。

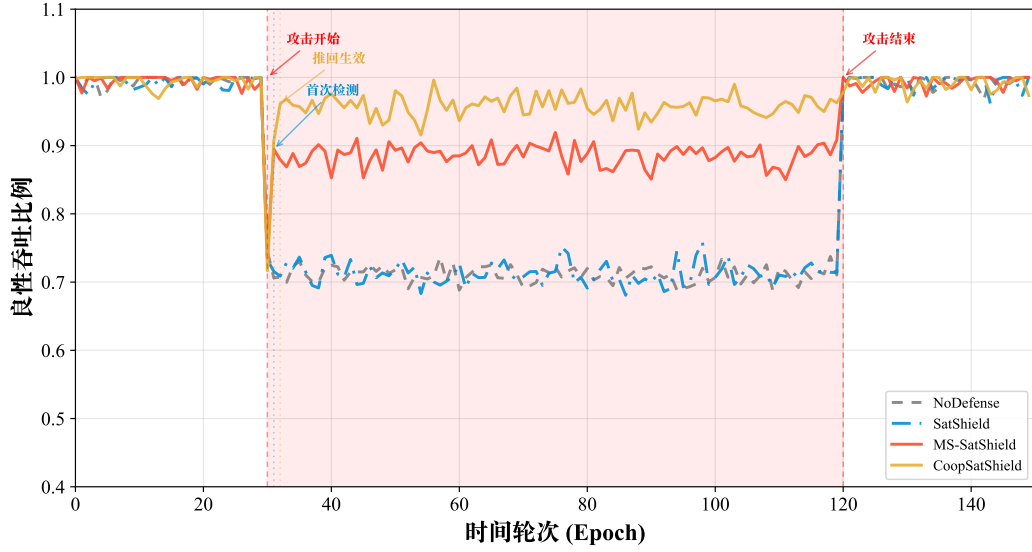
当 $M = 100$ 时，SatShield 的 Rate-only 检测已经基本失效 ($F1 = 0.017$)， $W_{\text{waste}} = 248,995 \text{ Mb/s} \cdot \text{hop}$ ，与 NoDefense 的 251,005 只差 0.8%，几乎等同于无防御。这一结果与第三章图 3-3(b) 中的分离段分析一致，因为在 $M = 100$ 时速率特征已经落入良性长尾。MS-SatShield 依靠 FI/FO 特征仍能保持检测能力 ($F1 = 0.909$)，使 W_{waste} 降至 NoDefense 的 17.1%；CoopSatShield 再通过推回把这一比例进一步压低到 11.0%，相较 MS-SatShield 额外释放了 6.1 个百分点的链路带宽。

当 $M = 1000$ 时，攻击 key 大量脱离 Top- K 候选集 ($\text{Top-}K \text{ Recall} \approx 0.30$)，所有方案的 W_{waste} 都上升较多。MS-SatShield 与 CoopSatShield 仍然优于 SatShield，分别占 NoDefense 的 72.5% 和 70.8%，但改善幅度已经有限，这反映的正是第三章中指出的候选集覆盖率瓶颈。

这些结果与式(4-2)描述的推回收益机理是一致的，即推回把攻击流的平均承载跳数 $\bar{h}$ 从全路径长度压缩到 $R_{\uparrow}$ 跳量级。同时，不同 M 值下的渐进退化设定也延续了第三章中的分析路径：先是 Rate-only 仍然有效，再到需要多签名补偿，最后进入候选集覆盖率受限的区间。

(2) 良性吞吐与反应时间

图 4-5 给出了 $M = 100$ 攻击场景下良性吞吐随时间 (epoch) 变化的全过程。实验将攻击窗口设置为 epoch 30 至 epoch 120 (共 90 个 epoch)，图中粉色背景标出攻击持续阶段，竖虚线分别对应攻击开始、首次检测和推回生效三个时刻。按上一节的定义，本地检测生效时延为 $T_{\text{det}} = 1 \text{ epoch}$ ，pushback 生效时延为 $T_{\text{pb}} = 2 \text{ epoch}$ 。

图 4-5 良性吞吐随时间变化 ($M = 100$)

攻击前 (epoch 0~29): 所有方案的良性吞吐都维持在 1.0 附近, 符合网络正常运行状态。

攻击触发与检测延迟 (epoch 30): 由于检测算法至少需要 1 个 epoch 的统计积累, 攻击开始后的第一个 epoch 内所有方案都无法检出攻击流, 良性吞吐统一跌至 NoDefense 水平 ≈ 0.71 。这一短暂下探反映的是在网检测的内在时滞, 对所有基线都成立。

首次检测生效 (epoch 31): SatShield 的 Rate-only 检测在 $M = 100$ 下已失效 ($F1 = 0.017$), 因此吞吐仍维持在 NoDefense 水平。MS-SatShield 依靠多签名检测能力 ($F1 = 0.909$), 通过高优先级队列保留机制 (80% 带宽保留) 将攻击期间的良性吞吐恢复到 0.886, 相对 NoDefense 提升了 17.5 个百分点。CoopSatShield 在这一时刻暂时与 MS-SatShield 处于同一水平, 因为检测已经生效, 但推回尚未传播完成。

推回生效 (epoch 32+): 当 Gossip 消息完成 ROI 内扩散后, CoopSatShield 的 pushback 规则开始在上游节点生效, 约 86% 的已检测攻击流在上游被直接丢弃, 中间链路带宽因此得到释放。稳态下, CoopSatShield 的吞吐恢复至 0.959, 相较 MS-SatShield 进一步提升了 7.3 个百分点。由于推回覆盖率并未达到 100%, 仍有约 14% 的攻击流沿非推回路径到达目标链路, 因此稳态吞吐停留在 0.959 而不是理想的 1.0, 这体现了分布式推回在工程实现上的实际限制。

攻击结束 (epoch 120 后): 所有方案的吞吐都迅速恢复到 1.0 附近。整体来看, CoopSatShield 的吞吐恢复表现为两个连续阶段: 先由检测触发的队列调度提供基础保护 (+17.5 pp), 再由推回带来的上游丢弃继续提供增量收益 (+7.3 pp), 最

终使良性吞吐在攻击期间维持在约 0.96。

(3) 协同开销与无信令风暴验证

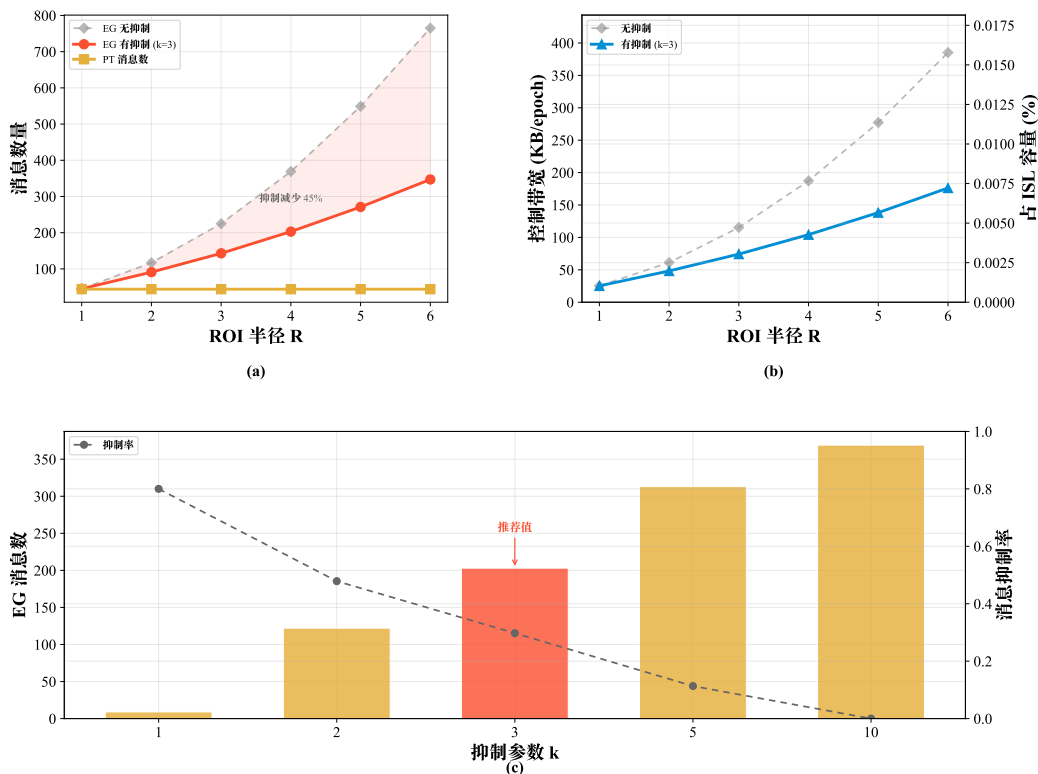


图 4-6 协同控制开销与抑制机制效果。(a)EG 消息数随 R 变化；(b) 控制带宽随 R 变化；(c) 抑制参数 k 的影响

本章最关心的工程风险之一，是大规模网络中的信令风暴。图 4-6 展示了协同开销和抑制机制的效果。

图 4-6(a) 比较了有抑制 ($k = 3$) 和无抑制两种情况下，EG 消息数随 ROI 半径 R 的变化。无抑制时，EG 消息数从 $R = 1$ 的 45 条增长到 $R = 6$ 的 765 条，接近超线性增长；加入抑制后，同一区间只从 45 条增长到 347 条。在 $R = 4$ 时，抑制机制把消息数从 369 条压低到 203 条，减少了 45%。PT 消息数则维持在 44 条，因为它只沿推回路径单向传播，不受 R 影响。图中的浅红色填充区域直观展示了抑制带来的收益。

图 4-6(b) 进一步将消息数换算为控制带宽 (KB/epoch)，并用右侧 y 轴给出其占 ISL 容量 (20 Gb/s) 的百分比。这里的字节统计采用了更保守的 EG 载荷配置 $n = 100$ ，并将 EG 与 PT 消息一并计入，因此 104 KB/epoch 应理解为压力条件下的上界估计，而不是默认 $n = 10$ 配置下的运行时均值。在推荐配置 $R = 4$ 时，该口径下的控制带宽仅占 ISL 容量的 0.004%；即使在 $R = 6$ 的最大测试半径下，占

比也不超过 0.008%。这说明协同开销本身很低，也与式(4-9)和式(4-11)的理论分析一致。

图 4-6(c) 展示了抑制参数 k 对 EG 消息数与抑制率的影响。 $k = 1$ 时抑制率高达 80%，但消息只有 9 条，覆盖不足； $k = 10$ 时抑制率降到 0%，基本等同于无抑制。本文推荐的 $k = 3$ （图中红色标注）在覆盖与开销之间取得了较好的平衡，此时抑制率为 30%，消息数为 203 条。基于当前结果，可以把 $k = 3$ 视为工程上可工作的默认配置；它对检测质量的影响仍需在后续补充实验中与 $k = 10$ 等配置进一步比较。

(4) 碎片化攻击下的表现

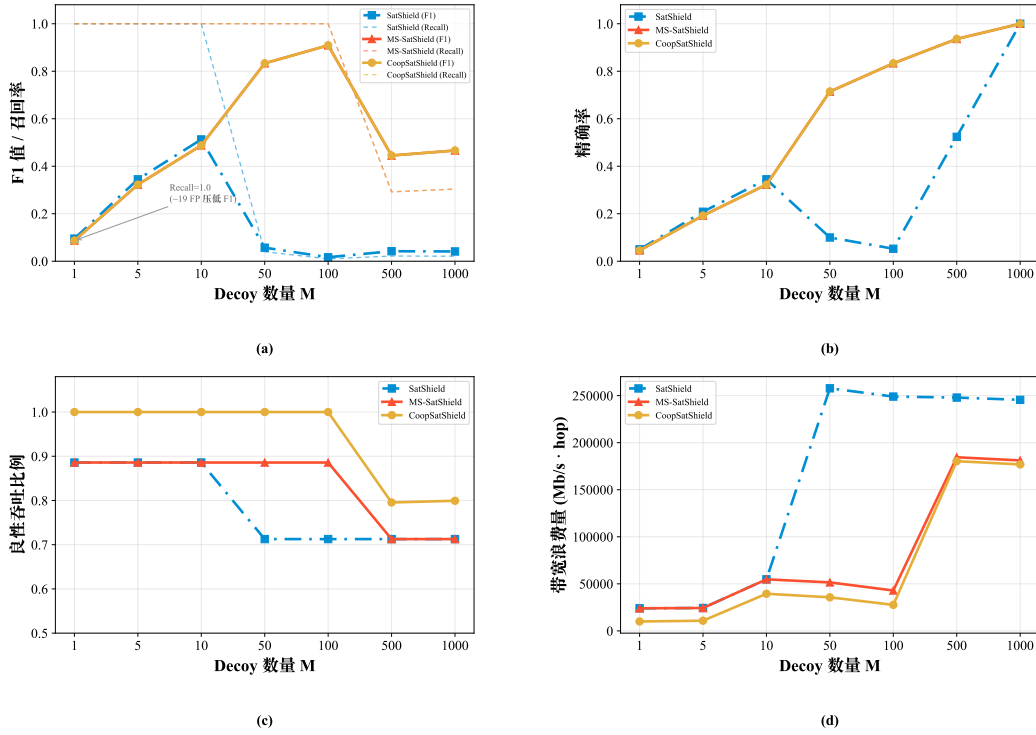


图 4-7 碎片化攻击下的表现。(a)F1 与 Recall；(b)Precision；(c) 良性吞吐恢复；(d) 无效带宽占用

图 4-7 评估了各方案在碎片化攻击 ($M \in \{1, 5, 10, 50, 100, 500, 1000\}$) 下的表现。其中，图 4-7(a)–(d) 分别给出检测效果 (F1 与 Recall)、检测精度 (Precision)、良性吞吐恢复以及无效带宽占用的实验结果。

F1 的倒 U 型变化 (图 4-7(a))：MS-SatShield 与 CoopSatShield 的 F1 随 M 呈现倒 U 型，而不是简单的单调递减，这一变化大致可以分成三段。

当 $M \leq 10$ 时，所有方案的 Recall 都保持在 1.0 (虚线所示)，说明攻击 key 已全部捕获；但 F1 仍然偏低，例如 $M = 1$ 时约为 0.09。原因并不在于漏检，而在于存在

约 19 个良性重流 key 被持续误判为可疑，这些 key 的聚合速率为 474 ~ 606 Mb/s，超过 $\tau \cdot \text{rate}_{p_{99}} = 242.5 \text{ Mb/s}$ 。这些良性重流 key 源自 CAIDA trace 中的大象流，其速率特征在统计上与低碎片化攻击 key 重叠，导致多签名评分器无法仅凭速率将其排除。由图 4-7(b) 可见， $M = 1$ 时 Precision 只有 0.05 (TP=1, FP $\approx$ 19)，因此 F1 偏低主要是 Precision 不足造成的。这里需要强调，F1 较低并不意味着检测无效：在 $M = 1$ 时，唯一的攻击 key（聚合速率 10,024 Mb/s）仍被正确识别，且其速率远高于所有良性 key，在缓解阶段会被优先调度到最低优先级队列，因此对良性业务的实际保护效果几乎不受误报影响。图 4-7(c) 也表明 SatShield 与 MS-SatShield 的吞吐都恢复到了 0.886，CoopSatShield 通过推回则进一步恢复到 1.0。

当 M 为 50 ~ 100 时，攻击 key 数量已经多于误报数，FI/FO 特征开始能够区分攻击流量与良性流量。这是因为随着碎片化程度增大，每个攻击 key 虽然速率下降，但它们仍然共享来自同一组 Bot 的源地址集合，在扇入度和扇出度分布上呈现出与良性流量不同的结构模式，使得多签名评分器能够借助速率以外的维度完成识别。 $M = 100$ 时，MS-SatShield 的 Precision 升至 0.833，Recall 仍保持 1.0，F1 达到峰值 0.909。同一区间内，SatShield 已经失效， $M = 50$ 时 F1 仅为 0.057， $M = 100$ 时进一步降到 0.017，因为此时攻击 per-key 速率已经接近良性 p_{99} ，仅依赖速率特征的 Rate-only 方案已无法完成区分。这一结果与第三章图 3-3(b) 中的分离段分析保持一致。

当 $M \geq 500$ 时，攻击 key 的单 key 速率降至 20 ~ 35 Mb/s，大量 key 脱离 Top- K 候选集 (Top- K Recall $\approx$ 0.30)，即 CMS 与候选缓存在有限容量下已经无法将绝大多数低速攻击 key 选入后续评分流水线。MS-SatShield 的 Precision 反而提高到 0.936 ~ 1.0，这是因为能够进入候选集的攻击 key 往往是碎片化 key 中速率相对较高的那部分，它们在多签名评分下几乎都能被正确识别；但 Recall 只有 0.30，说明大部分攻击流量已经逃逸到候选集之外，最终使 F1 降至 0.445 ~ 0.466。

CoopSatShield 与 MS-SatShield 的 F1 一致（图 4-7(a) 中两条 F1 实线重合）。原因在于二者共享同一检测流水线，即同一个多签名评分器。在当前实验采用的 Bot 均匀分布场景下，各节点的本地 Top- K 命中率已经足够高，协同证据聚合不会改变单节点的检测输出。因此，本章协同机制的价值主要体现在缓解层面。Gossip 负责跨节点共享攻击信息，pushback 则据此把缓解动作从被攻击链路前推到更上游的入口，使得攻击流在穿越中间链路之前就被限速或丢弃，从而释放原本被无效占用的中间链路容量，这正对应了引言中提出的上游带宽浪费问题。图 4-7(c) 显示，在低到中等碎片化范围内，CoopSatShield 的良性吞吐明显高于 MS-SatShield。若统一采用与图 4-5 一致的攻击期间稳态平均吞吐口径，则 $M \leq 50$ 时其恢复值接近 1.0， $M = 100$ 时仍为 0.959，高于 MS-SatShield 的 0.886；图 4-7(d) 则显示，

CoopSatShield 的 W_{waste} 在各个 M 值下都保持最低，这也与图 4-4 中的渐进退化分析一致。当 $M \geq 500$ 时，所有方案的吞吐都开始退化，CoopSatShield 也下降到 0.80，反映出候选集覆盖率瓶颈最终会传导到缓解层面。需要补充说明的是，在 Bot 分布极不均匀的特定场景下，例如攻击流量集中经过少数中继节点、而这些节点的本地候选集又无法覆盖全部相关 key 时，协同聚合仍然可能提高检测 Recall；但在本章采用的均匀 Bot 分布假设下，这种效果没有显现。

4.3.3 适用范围与讨论

(1) **量化比较范围。**本章正文中的数值比较严格限定在 NoDefense、SatShield、MS-SatShield 与 CoopSatShield 四类星载方案之内。LinkScope、Woodpecker、RADAR、Ripple 与 Mew 等地面 LFA 防御机制仍然有参考价值，但在统一的 LEO 实现完成之前，它们更适合作为设计假设和控制闭环差异的讨论对象，而不适合直接写入本章实验结论。

(2) **协同聚合的作用层次。**在当前均匀 Bot 分布设定下，协同聚合评分 S^{net} 并未改变检测 F1，因此本章能够支持的主要结论是：加权聚合有助于平滑 push-back 触发，但没有表现出提高检测能力的作用。若后续需要进一步说明式(4-7)与式(4-8)相对于简单 avg 或 max 规则的必要性，最直接的办法是补充聚合规则消融以及 Θ_{pb} 、 λ 、 μ 的敏感性实验。

(3) **推回覆盖率与候选集瓶颈。**CoopSatShield 在 $M = 100$ 时仍只能将稳态良性吞吐恢复到 0.959，说明近似反向路径推回与局部 ROI 覆盖仍然受制于拓扑分叉和路径不确定性；当 $M \geq 500$ 时，一级候选集覆盖率下降又会进一步把检测瓶颈传导到缓解层面。就本章的证据而言，可以较明确地说明推回能够前移缓解位置并降低网络内部成本，但还不能说明它在任意碎片化强度下都能基本消减攻击影响。

4.4 本章小结

本章面向大规模低轨卫星网络中的链路洪泛攻击，将研究范围从单星本地闭环防御进一步扩展到多星协同与逐跳推回的网络级防御。针对 LEO 网络中控制环延迟高、拓扑变化快、攻击证据分散等约束，本文提出了 CoopSatShield。该方法在第三章本地多签名检测的基础上，将检测输出组织为可协同处理的证据单元；通过事件驱动、ROI 限制和冗余抑制构造轻量级邻居扩散机制；再结合逐跳 pushback 与入端口贡献分布，将缓解动作从被攻击链路附近前移到更上游的位置，从而把本地检测能力扩展为跨节点的协同处置能力。

实验结果表明, CoopSatShield 的主要收益体现在两个方面。其一, 在攻击证据分散到多个节点、单节点局部处置不足以抑制网络内部资源浪费的场景中, 协同机制能够将碎片化本地观测汇聚为更平滑的网络级触发信号, 并借助 **pushback** 将丢弃与限速位置前移, 因此相较仅在被攻击链路附近执行缓解的方案, 更能降低网络内部的无效带宽占用并改善良性业务保护。其二, 在工程实现层面, 事件驱动、ROI 限制与抑制扩散的组合, 使协同开销始终受控, 没有出现信令风暴, 说明该方法在大规模星座中具备继续扩展的可行性。整体上看, 第四章解决了“如何在多星之间传播和聚合攻击证据, 并将防御位置从被攻击链路前移到网络上游”的问题。

不过, 本章方法仍有一些限制。首先, 在本文采用的均匀 Bot 分布设定下, 协同聚合主要提升的是缓解层面的效果, 而对单节点检测输出本身的改变有限, 因此其主要增益仍体现在 **pushback** 所带来的网络级资源节约。其次, 当攻击进一步碎片化时, 一级候选覆盖率仍然会重新成为主要瓶颈, 说明多星协同不能替代本地候选生成质量。最后, 本文默认协同邻居可信, 尚未讨论虚假 EG/PT 消息注入、跨轨道面更大范围协同以及与地面控制系统联动等问题, 这些都可以作为后续工作继续展开。

第五章 总结与展望

5.1 全文总结

大规模低轨卫星巨型星座凭借低时延、广覆盖的通信能力，正在成为下一代天地一体化信息网络的基础设施。然而，星间链路带宽有限、卫星轨道参数公开等特性，也使 LEO 星座网络面临链路洪泛攻击威胁。围绕如何在星载可编程数据面的严格 SWaP 约束下构建低时延、低资源消耗且能够应对 LFA 退化策略的在网防御体系，本文以在网计算为主线，研究了基于多签名在网检测与队列调度的 LFA 防御优化，以及基于多星协同与逐跳推回的 LFA 防御优化。全文的主要研究成果如下。

(1) **基于多签名在网检测与队列调度的 LFA 防御优化 (MS-SatShield)**。针对单节点在 P4 数据面上进行 LFA 检测时面临的特征退化问题，本文在第三章提出了基于多签名在网检测与队列调度的 LFA 防御优化方法。其主要设计是级联式两级检测流水线：第一级采用 Count-Min Sketch 频率估计与固定容量候选缓存相结合的方式，在固定内存下维护全局近似频次并恢复候选 key 集合；第二级在候选集上部署双缓冲位图结构，在线提取扇入度、扇出度和历史持续性等多维结构特征签名。通过仅对候选集合维护 FI/FO，该方法将去重计数状态成本控制在 $O(K \cdot m)$ ，契合星载平台的 SWaP 约束，并通过队列优先级调度将检测结果转化为低误伤处置动作。实验结果表明：在多 Decoy 分摊退化设定下 ($M = 100$)，仅依赖聚合速率的基线方案 F1 降至 0.017，而引入 FI/FO 后 MS-SatShield 的 F1 恢复至 0.908；在脉冲式 on-off 攻击下，持续性特征通过加性奖励与候选集扩展机制将 Recall 提升至 0.999，良性吞吐恢复至 0.885；消融实验表明， $K \approx 1000$ 、 $m = 256$ 、 $\Delta = 1.0$ s 是检测精度与资源开销之间较优的经验折中。

(2) **基于多星协同与逐跳推回的 LFA 防御优化 (CoopSatShield)**。针对单星检测视野有限和被动就地丢弃模式的不足，本文在第四章提出了基于多星协同与逐跳推回的 LFA 防御优化方法。该方法包含两个组件：抑制型流言传播协议和逐跳推回机制。前者通过事件驱动、区域限制和基于 Trickle 思想的冗余抑制三重约束，在受控信令开销下实现局部区域内多颗卫星之间的攻击证据交叉验证与聚合；后者利用入端口贡献分布传播推回令牌，无需全局逆路由即可将攻击抑制点从受害链路前推至上游入口，减少攻击流在网络中的无效传输。实验结果表明：在 $M = 100$ 时，CoopSatShield 将路径积分意义下的无效带宽占用成本降至无防御方案的 11.0%，相比 MS-SatShield 进一步降低 6.1 个百分点；攻击期间良性吞吐稳态

维持在 0.959，相比 MS-SatShield 提升 7.3 个百分点；在推荐配置 $R = 4$ 、 $k = 3$ 下，按保守 $n = 100$ 统计的控制带宽仅占 ISL 容量的 0.004%，抑制机制使 EG 消息数比无抑制基线减少 45%，表明协同通信不会引发信令风暴。

上述两项研究共同构成了面向大规模 LEO 卫星网络的在网安全防御体系，形成了从单节点在网计算到多节点协同在网计算的整体框架。在理论层面，本文揭示了 LFA 退化策略与多维特征签名之间的约束关系，即攻击者难以在压缩速率特征的同时压缩 FI/FO 特征，为多签名检测提供了依据。在方法层面，本文提出了级联式两级检测架构和抑制型邻居扩散协同框架，将检测与缓解逻辑下沉至星载可编程数据面，规避了星地控制环路的时延瓶颈。在系统层面，数据面总状态开销控制在约 200 KB，控制信令带宽不超过 ISL 容量的 0.004%，满足星载环境的资源约束。

5.2 未来展望

本文围绕低轨卫星网络中的链路洪泛攻击防御开展了研究，并在单星检测、多星协同和逐跳推回等方面取得了较好的实验结果。随着星座能力持续提升，未来的研究还需要考虑新型业务载荷和更复杂网络组织形态带来的新需求。基于这一判断，后续工作可重点从以下两个方向继续展开。

(1) **面向多业务与在轨计算载荷的防御扩展。**未来低轨卫星网络将不再只是单纯的宽带转发网络，而会同时承载公众接入、企业专网、政府业务以及在轨 AI 处理和空间侧算力协同等任务^[10, 23, 32]。这意味着网络中的保护对象将从传统链路和队列进一步扩展到模型分发路径、在轨推理结果、跨星算力调度和业务切片保障等新型载荷。相应地，后续研究可在本文框架基础上进一步考虑面向多类业务与计算任务的分级检测、差异化缓解与联合保障机制，使在网防御能够同时适应通信业务和计算载荷并存的运行环境。

(2) **面向跨区域与异构星座场景的协同扩展。**本文第四章主要围绕单一星座内的局部协同与逐跳推回展开，而未来低轨网络很可能朝着更大规模、跨区域互联、多轨道层协作以及异构星载平台并存的方向发展。在这类场景中，攻击证据和防御动作不一定局限于单一 ROI 或单一星座内部，而可能涉及跨轨道面、跨区域甚至跨星座的联合响应。因此，后续可进一步研究更大范围协同条件下的证据传播、区域间推回衔接和异构数据面协同机制，将本文提出的多签名检测与逐跳推回防御优化扩展到更复杂的网络组织形态中，以提升其在未来空间信息网络中的适用性和推广价值。

致 谢

青青园中葵，朝露待日晞。转眼间我已经在这个学校待了7年，7个春华秋实。时间过得好快，这7年里，我的弟弟从幼儿园的宝宝长到六年级的大孩子，妹妹从小学生变成高考生。也有很多亲密的人离开了我，我的爷爷、姥爷、么爸、小舅等。

我要感谢我的爸爸妈妈，感谢你们在这这么多年给我稳定的支撑、持续的指引、无尽的包容和爱护。无论是什么年纪，你们都在不断学习，不断去面对更大的挑战。我看着你们一步步从筚路蓝缕走到今天，认真地做好每一件小事。是你们给了我不断挑战的决心，不怕从头开始的勇气，以及真诚踏实的处事态度。

感谢亲爱的弟弟，感谢你给我补充“好奇心”的干细胞。很多人长大了之后就会失去对这个世界的好奇心，但是因为有你总是问我“为什么”，让我总是去深入思考一个事情背后的本质原因，你的提问让我养成了这种思考的习惯，让我无论在什么位置，都能像一个孩子一样好奇地去分析这个世界。希望你能健康茁壮成长。

我要感谢我在电子科技大学遇到的所有师长。我觉得是老师塑造了一个学校的风格，让从这里走出去的同学都有了求实求真、大气大为的底色。研究生阶段项目上合作的老师有孙健老师、段景山老师、贾蓉老师、杨宁老师等，他们从技术、表达等方面培养了我严谨认真的做事风格，感谢他们对我的耐心和宽容，感谢他们给我营造的一次次的锻炼机会。感谢吴畏虹老师，他带着我们做了很多项目，以为学生着想的态度帮助每一个学生做学业规划。感谢罗龙老师，她真诚聪明、经验丰富，带着年少的我们初入国际会议的舞台，感谢她的帮助给了我一段难忘的经历。感谢虞红芳老师，感谢她为项目组提供的丰富资源，感谢她为支撑学生自由发展所做的辛勤工作，她以宽容开放的态度在很多次谈话和讲话中激励了我，时间越长越能感觉到虞老师的高瞻远瞩、高屋建瓴与良苦用心。最后我要感谢我的导师孙罡老师，感谢他，感谢他在我毕设工作中的尽心尽力，提出了无微不至的建议，“认真、负责、严谨与追求细节”，是孙老师这次以及之前很多次项目中对我的言传身教；感谢他的课程，进而吸引我加入这个团队，感谢他为团队、为我们所有学生的奔波与辛劳。

感谢我的女朋友赵舒心，这三年因为有你整个学校的色彩都不再一样，有了你之后，我好像就没有了后顾之忧，可以全心全意地向前奔跑。我记得在图书馆，你为我讲解一道道最优化的数学题目，一起准备期末考试；在我难以专注的时候，陪伴我听我梳理源代码；在我疲惫不想睁开眼睛时，我可以安心地让你拉着我走。

我们一起经历了很多大大小小的挑战，即使我们的小舟行驶在的是波涛汹涌的大海上，我也没有一丝沮丧，我深深地相信这个小船可以平稳坚定地驶到前方。

感谢这三年里我遇到的每个同学，他们都是非常的优秀。感谢何奕晨和冯伊凡，我们一起在项目和科研上通力合作，忙碌但是充实，又有着悠闲的快乐。感谢伍东旭、郭家豪等同学、冯文佼、谢景昭、马崇喜等师兄师姐、李政廉等师弟师妹的陪伴，希望你们都前途似锦。

感谢我在阿里云实习遇到的每个人，我的 leader 和 mentor 们，他们以深厚的技术积累和经验，为我抚平了内心的焦虑、拓展了我未来的路。感谢我足球队的队友们，你们是我的另一个世界。

出生在这个时代从大部分角度来看都是幸运的，感谢有 AI 的存在，让我这样的学生，能够有比以前高很多倍的效率去学习新的知识，能够探索世界与自己更多的可能性，能够跟随科技更快速地发展。尽管太过强大的 AI 难免会给人未知的焦虑，我仍然抱有 AI 乐观主义。

千里之行、始于足下，还需要珍惜生命的每一分钟，珍惜和每个人相处见面的机会，珍惜每一次挑战自己的可能，永远不放弃对这个世界的好奇心。

参考文献

- [1] Kodheli O, Lagunas E, Maturo N, et al. Satellite Communications in the New Space Era: A Survey and Future Challenges[J]. IEEE Communications Surveys & Tutorials, 2021, 23(1): 70–109.
- [2] Wang S, Li Q. Satellite computing: Vision and challenges[J]. IEEE Internet of Things Journal, 2023, 10(24): 22514–22529.
- [3] Bhattacharjee D, Aqeel W, Bozkurt I N, et al. Gearing up for the 21st century space race[C]. The 17th ACM Workshop on Hot Topics in Networks, 2018: 113–119.
- [4] Chaudhry A U, Yanikomeroglu H. Laser intersatellite links in a starlink constellation: A classification and analysis[J]. IEEE Vehicular Technology Magazine, 2021, 16(2): 48–56.
- [5] Giuliani G, Klenze T, Legner M, et al. Internet backbones in space[J]. ACM SIGCOMM Computer Communication Review, 2020, 50(1): 25–37.
- [6] Singla A, Chandrasekaran B, Godfrey P B, et al. The internet at the speed of light[C]. The 13th ACM Workshop on Hot Topics in Networks, 2014: 1–7.
- [7] Handley M. Delay is not an option: Low latency routing in space[C]. The 17th ACM Workshop on Hot Topics in Networks, 2018: 85–91.
- [8] Bhattacharjee D, Singla A. Network topology design at 27,000 km/hour[C]. The 15th ACM International Conference on Emerging Networking Experiments and Technologies, 2019: 341–354.
- [9] Handley M. Using ground relays for low-latency wide-area routing in megaconstellations[C]. The 18th ACM Workshop on Hot Topics in Networks, 2019: 125–132.
- [10] Starlink. Direct to cell[OL]. 2026, <https://www.starlink.com/business/direct-to-cell>.
- [11] Starlink. Stargaze: SpaceX’s space situational awareness system[OL]. 2026, <https://starlink.com/updates/stargaze>.
- [12] Federal Communications Commission. Authorization and order: Space exploration holdings, llc, request for deployment and operating authority for the spacex gen2 ngso satellite system[OL]. 2026-01-09, <https://docs.fcc.gov/public/attachments/DA-26-36A1.pdf>.
- [13] Amazon Staff. Amazon leo mission updates: Next launch with ula set for march 29[OL]. 2026, <https://www.aboutamazon.com/news/innovation-at-amazon/project-kuiper-satellite-rocket-launch-progress-updates>.
- [14] Amazon. Amazon leo[OL]. 2026, <https://www.aboutamazon.com/what-we-do/devices-services/amazon-leo>.

- [15] European Commission. IRIS² – the european commission awards the concession contract to spacerise consortium[OL]. 2024-12, <https://digital-strategy.ec.europa.eu/en/news/iris2-european-commission-awards-concession-contract-spacerise-consortium>.
- [16] Eutelsat. Responsible space[OL]. 2026, <https://www.eutelsat.com/group/sustainability-esg/responsible-space>.
- [17] Eutelsat Group. Our company[OL]. 2025, <https://prod.eutelsat.com/en/group.html>.
- [18] Telesat. Telesat advances telesat lightspeed terrestrial network with new quebec and saskatchewan landing station sites[OL]. 2026-03, <https://www.telesat.com/press/press-releases/telesat-advances-telesat-lightspeed-terrestrial-network-with-new-quebec-and-saskatchewan-landing-station-sites/>.
- [19] Xinhua. Chinese commercial satellite constellation completes network connection test[OL]. 2025-01-02, <https://english.news.cn/20250102/1971ae23a7154d15bff056e2e0372db3/c.html>.
- [20] Xinhua. Economic watch: China’s commercial satellites accelerate global connectivity[OL]. 2025-12-12, <https://english.news.cn/20251212/3d7831df01b84a56af7a121dca6cef9f/c.html>.
- [21] SpaceNews. China’s guowang launch raises questions about satellite purpose and transparency[OL]. 2025-01, <https://spacenews.com/chinas-guowang-launch-raises-questions-about-satellite-purpose-and-transparency/>.
- [22] SpaceNews. China adds new satellites to guowang constellation, eyes accelerated launch rate[OL]. 2025-07, <https://spacenews.com/china-adds-new-satellites-to-guowang-constellation-eyes-accelerated-launch-rate/>.
- [23] Xinhua. China demonstrates AI computing power in outer space with satellite network breakthrough[OL]. 2026-02-13, <https://english.news.cn/20260213/f697fc260d66410398395307dd27443b/c.html>.
- [24] Giuliani G, Ciussani T, Perrig A, et al. ICARUS: Attacking low Earth orbit satellite networks[C]. The 2021 USENIX Annual Technical Conference, 2021: 317–331.
- [25] Kang M S, Lee S B, Gligor V D. The Crossfire attack[C]. IEEE Symposium on Security and Privacy, 2013: 127–141.
- [26] Studer A, Perrig A. Coremelt: An attack on the internet core through legitimate traffic[C]. European Symposium on Research in Computer Security, Springer, 2009: 209–225.
- [27] Rasti R, Murthy M, Weaver N, et al. Temporal lensing and its application in pulsing denial-of-service attacks[C]. IEEE Symposium on Security and Privacy, 2015: 187–198.
- [28] Lu T, Ding X, Shang J, et al. DoSat: A DDoS Attack on the Vulnerable Time-Varying Topology of LEO Satellite Networks[C]. Applied Cryptography and Network Security, Springer Nature Switzerland, vol. 14584, 2024: 265–282.

- [29] Jiang W. Software defined satellite networks: A survey[J]. Digital Communications and Networks, 2023, 9(6): 1243–1264.
- [30] Jiang W, Jiang H, Xie Y, et al. SatShield: In-Network Mitigation of Link Flooding Attacks for LEO Constellation Networks[J]. IEEE Internet of Things Journal, 2024, 11(16): 27340–27355.
- [31] Liu Z, Namkung H, Nikolaidis G, et al. Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric DDoS attacks with programmable switches[C]. The 30th USENIX Security Symposium, 2021: 3829–3846.
- [32] Amazon Staff. Amazon leo debuts new gigabit-speed “Ultra” antenna, begins enterprise preview[OL]. 2025-11, <https://www.aboutamazon.com/news/amazon-leo/amazon-leo-satellite-inter-net-ultra-pro>.
- [33] Kianpisheh S, Taleb T. A Survey on In-Network Computing: Programmable Data Plane and Technology Specific Applications[J]. IEEE Communications Surveys & Tutorials, 2023, 25(1): 701–761.
- [34] Sapio A, Abdelaziz I, Aldilajjan A, et al. In-Network Computation is a Dumb Idea Whose Time Has Come[C]. The 16th ACM Workshop on Hot Topics in Networks, 2017: 150–156.
- [35] Hauser F, Häberle M, Merling D, et al. A survey on data plane programming with P4: Fundamentals, advances, and applied research[J]. Journal of Network and Computer Applications, 2023, 212: 1–53.
- [36] Werner M. A dynamic routing concept for ATM-based satellite personal communication networks[J]. IEEE Journal on Selected Areas in Communications, 1997, 15(8): 1636–1648.
- [37] Wood L. Internetworking with satellite constellations[M]. University of Surrey (United Kingdom), 2001.
- [38] Yue P, An J, Zhang J, et al. Low earth orbit satellite security and reliability: Issues, solutions, and the road ahead[J]. IEEE Communications Surveys & Tutorials, 2023, 25(3): 1604–1652.
- [39] Usman M, Qaraqe M, Asghar M R, et al. Mitigating distributed denial of service attacks in satellite networks[J]. Transactions on Emerging Telecommunications Technologies, 2020, 31(6): 1–12.
- [40] Guo W, Xu J, Pei Y, et al. A distributed collaborative entrance defense framework against DDoS attacks on satellite internet[J]. IEEE Internet of Things Journal, 2022, 9(17): 15497–15510.
- [41] Meng F, Yan X, Zhang Y, et al. Mitigating DDoS attacks in LEO satellite networks through bottleneck minimize routing[J]. Electronics, 2025, 14(12): 1–19.
- [42] Hu T, Bai L, Yin Z. Pathweaver: Enhancing LEO satellite networks resilience via topology design and randomized routing against link flooding attacks[J]. Computer Networks, 2026, 277: 1–15.

- [43] Xue L, Ma X, Luo X, et al. LinkScope: Toward detecting target link flooding attacks[J]. IEEE Transactions on Information Forensics and Security, 2018, 13(10): 2423–2438.
- [44] Wang L, Li Q, Jiang Y, et al. Woodpecker: Detecting and mitigating link-flooding attacks via SDN[J]. Computer Networks, 2018, 147: 1–13.
- [45] Zheng J, Li Q, Gu G, et al. Realtime DDoS defense using COTS SDN switches via adaptive correlation analysis[J]. IEEE Transactions on Information Forensics and Security, 2018, 13(7): 1838–1853.
- [46] Xing J, Wu W, Chen A. Ripple: A programmable, decentralized link-flooding defense against adaptive adversaries[C]. The 30th USENIX Security Symposium, 2021: 3865–3881.
- [47] Zhou H, Hong S, Liu Y, et al. Mew: Enabling large-scale and dynamic link-flooding defenses on programmable switches[C]. IEEE Symposium on Security and Privacy, 2023: 3178–3192.
- [48] Ilha A D S, Lapolli A C, Marques J A, et al. Euclid: A Fully In-Network, P4-Based Approach for Real-Time DDoS Attack Detection and Mitigation[J]. IEEE Transactions on Network and Service Management, 2021, 18(3): 3121–3139.
- [49] Alcoz A G, Strohmeier M, Lenders V, et al. Aggregate-based congestion control for pulse-wave DDoS defense[C]. The ACM SIGCOMM Conference, 2022: 693–706.
- [50] Wu J, Pan H, Cui P, et al. Patronum: In-network volumetric DDoS detection and mitigation with programmable switches[C]. European Symposium on Research in Computer Security, 2024: 187–207.
- [51] P4 Language Consortium. P4 becomes an independent project, hosted by the linux foundation[OL]. 2024-01, <https://p4.org/p4-becomes-an-independent-project-hosted-by-the-linux-foundation/>.
- [52] P4 Language Consortium. Specifications[OL]. 2026, <https://p4.org/specifications/>.
- [53] P4 Language Consortium. Intel’s tofino P4 software is now open source[OL]. 2025-01, <https://p4.org/intels-tofino-p4-software-is-now-open-source/>.
- [54] P4 Language Consortium. Cisco joins the P4 consortium as a premier member in 2025[OL]. 2025-04, <https://p4.org/cisco-joins-the-p4-consortium-as-a-premier-member-in-2025/>.
- [55] P4 Language Consortium. Xsight labs transforms ethernet switch market with unprecedented open architecture[OL]. 2025-03, <https://p4.org/xsight-labs-transforms-ethernet-switch-market-with-unprecedented-open-architecture/>.
- [56] NVIDIA. DOCA target architecture[OL]. 2026, <https://docs.nvidia.com/doca/sdk/doca-target-architecture/index.html>.
- [57] AMD. Amd pensando pollara 400 AI NIC[OL]. 2025, <https://www.amd.com/en/products/network-interface-cards/pensando.html>.

- [58] P4 Language Consortium. 2025 P4 workshop wrap-up[OL]. 2025-11, <https://p4.org/2025-p4-workshop-wrap-up/>.
- [59] Walker J G. Satellite constellations[J]. Journal of the British Interplanetary Society, 1984, 37: 559–571.
- [60] Kassing S, Bhattacharjee D, Águas A B, et al. Exploring the “internet from space” with hypatia[C]. The ACM Internet Measurement Conference, 2020: 214–229.
- [61] Bosshart P, Gibb G, Kim H S, et al. Forwarding metamorphosis: Fast programmable match-action processing in hardware for SDN[C]. The ACM SIGCOMM Conference, 2013: 99–110.
- [62] McKeown N, Anderson T, Balakrishnan H, et al. OpenFlow: Enabling innovation in campus networks[J]. ACM SIGCOMM Computer Communication Review, 2008, 38(2): 69–74.
- [63] Bosshart P, Daly D, Gibb G, et al. P4: Programming protocol-independent packet processors[J]. ACM SIGCOMM Computer Communication Review, 2014, 44(3): 87–95.
- [64] Bloom B H. Space/time trade-offs in hash coding with allowable errors[J]. Communications of the ACM, 1970, 13(7): 422–426.
- [65] Cormode G, Muthukrishnan S. An improved data stream summary: The count-min sketch and its applications[J]. Journal of Algorithms, 2005, 55(1): 58–75.
- [66] Whang K Y, Vander-Zanden B T, Taylor H M. A linear-time probabilistic counting algorithm for database applications[J]. ACM Transactions on Database Systems, 1990, 15(2): 208–229.
- [67] Demers A, Greene D, Hauser C, et al. Epidemic algorithms for replicated database maintenance[C]. The Sixth Annual ACM Symposium on Principles of Distributed Computing, 1987: 1–12.
- [68] Levis P, Patel N, Culler D, et al. Trickle: A self-regulating algorithm for code propagation and maintenance in wireless sensor networks[C]. USENIX Symposium on Networked Systems Design and Implementation, 2004: 15–28.

攻读硕士学位期间取得的成果

发表论文：

- [1] **Gong K**, He Y, Feng Y, et al. Demo: KLONET-SAT: A Comprehensive Platform for Satellite Network Education[C]. The ACM SIGCOMM Conference: Posters and Demos, 2024: 113-115.
- [2] Zhang J, Ma T, **Gong K**, et al. DTNET: Toward A Digital-Twin Computer Networks Education[C]. The 39th Youth Academic Annual Conference of Chinese Association of Automation, 2024: 585–588.
- [3] 贾蓉, 张进, **龚科成**, 等. 天地一体化网络虚拟仿真实验设计 [J]. 实验室研究与探索, 2026,45(01):85-91.

参与项目：

- [1] 华为合作项目：基于虚拟化技术的大规模网络创新试验平台, 2023 年 9 月–2024 年 12 月。
- [2] 国家重点研发计划项目：基于源地址伪造的安全威胁分析及态势感知，项目编号：2021YFB01001，2023 年 6 月–2024 年 11 月。
- [3] 中国电科 10 所项目：基于 SDN 的异构网验证测试软件，2025 年 4 月–2025 年 7 月。

所获奖励：

- [1] 阿里云实习生 AI 大赛冠军，2025 年 8 月。
- [2] 第十九届“挑战杯”大学生学术科技作品竞赛全国一等奖，2024 年 11 月。
- [3] IEEEXtreme 18.0 极限编程大赛全球排名前 3%，2024 年 10 月。
- [4] IEEEXtreme 17.0 极限编程大赛全球排名前 6%，2023 年 10 月。
- [5] 信息与通信工程学院研究生学术论坛最佳海报展示奖，2024 年 12 月。
- [6] 电子科技大学学业奖学金一等奖，2025 年 10 月。
- [7] 电子科技大学学业奖学金一等奖，2024 年 10 月。
- [8] 电子科技大学学业奖学金一等奖，2023 年 10 月。
- [9] “复杂网络协议分析虚拟仿真实验”获评虚拟仿真教学国家一流课程，2025 年 12 月。

- [10] “天地一体化网络虚拟仿真实验”获得第六届中国计算机教育大会计算机类教学和实验案例特等奖，2024 年 12 月。
- [11] “卫星互联网虚拟仿真实验”获得第四届全国高校电子信息类专业课程实验教学案例设计竞赛西部赛区二等奖、全国三等奖，2024 年 11 月。